# NN Builder

User manual: using the application, designing a neural network,
and the demo architectures explained

The NN Builder project
https://github.com/varunjha089/NN-Builder

5 November 2026 (version 0.3 and the changes after it; see CHANGELOG.md)

**Abstract**

NN Builder is a desktop application for designing neural networks on a canvas, the way one draws a block diagram. The design is checked while it is drawn: every connection shows the shape of the tensor it carries, every block shows its output size and parameter count, and mistakes are explained in plain words on the block itself. A finished design becomes a complete, readable PyTorch training script that also loads the data (a built-in dataset, Hugging Face or Kaggle), and the application runs it on a GPU somewhere else: a free Kaggle notebook, a college GPU server over SSH, or an Amazon EC2 instance (including spot instances, which may be interrupted). The computer the application runs on only designs: it never trains, never downloads datasets and needs no PyTorch. This manual explains the application screen by screen, walks through designing a network from scratch, gives the arithmetic behind every number the application shows, explains the fifteen demo architectures (from LeNet-5 to a one-billion-parameter Vision Transformer, plus one original design), and ends with troubleshooting. References point to the original papers, on arXiv wherever they are published there.

## Contents

**1 What NN Builder is** **5**

1.1 The idea . . . 5
1.2 The workflow at a glance . . . 5
1.3 What it is not . . . 5

**2 Installing, updating and the window** **5**

2.1 Installing . . . 5
2.2 Automatic updates . . . 6
2.3 The top bar . . . 6
2.4 Menus and keyboard shortcuts . . . 6
2.5 Day and night themes, Settings . . . 7

**3 Projects** **7**

3.1 Creating, opening, saving . . . 7
3.2 Demo architectures are unsaved until you save them . . . 8
3.3 What a project folder contains . . . 8

**4 The Design screen** **9**

4.1 Layout . . . 9
4.2 Working with blocks . . . 9
4.3 The layer table . . . 10
4.4 Famous architectures and the skip-connection tool . . . 11

**5 Designing a network, step by step** **11**
5.1 When something is wrong . . . 12

**6 The arithmetic behind the numbers** **13**
6.1 Output size . . . 13
6.2 Computation . . . 13
6.3 Parameters . . . 13
6.4 Receptive field . . . 14
6.5 Memory needed for training . . . 14

**7 Reference of all blocks** **14**
7.1 Your own layers in PyTorch (Custom block) . . . 16
7.2 Transformer options: masks, positions, LLaMA-style blocks, cross-attention . . . 16
7.3 Frozen layers, LoRA, experts and early exits . . . 17
7.4 Several GPUs . . . 17
7.5 Transformers in NN Builder . . . 17
7.6 Segmentation . . . 18

**8 Writing designs as text: the NNB language** **18**
8.1 A first design, line by line . . . 18
8.2 The design's name: `net` . . . 19
8.3 Settings: words and numbers . . . 19
8.4 Connections: `<-` . . . 20
8.5 Labels . . . 20
8.6 Names and arithmetic: `let` . . . 20
8.7 Repeating a group: `repeat` . . . 20
8.8 A loop over values: `for` . . . 21
8.9 Functions: `def` . . . 21
8.10 Several inputs, and reaching into a group . . . 22
8.11 Every block and how its values are set . . . 23
8.12 The rules in brief . . . 29
8.13 Mistakes and what the application says . . . 29
8.14 How the text and the canvas stay in step . . . 30
8.15 A design that uses everything . . . 30

**9 The Dataset tab** **30**
9.1 Built-in datasets . . . 31

9.2 Hugging Face . . . 32
9.3 Kaggle . . . 32
9.4 Segmentation data . . . 32
9.5 Image size and classes . . . 32
9.6 Satellite images: Sentinel-2, Sentinel-1 and other bands . . . 32
9.7 Your own files, e.g. Sentinel-1/2 from Google Earth Engine . . . 33
9.8 Several classes per image (multi-label) . . . 34
9.9 Numbers per image (regression) . . . 35
9.10 Several outputs (multi-task) . . . 35
9.11 Object detection (boxes) . . . 36
9.12 Time series of satellite images . . . 36
9.13 Shared weights: Siamese networks and change detection . . . 37
9.14 Sentinel-1 and Sentinel-2 together (fusion) . . . 37

**10 Kinds of data, step by step** **38**
10.1 Built-in sets (MNIST, Fashion-MNIST, CIFAR-10) . . . 38
10.2 Pictures from Hugging Face . . . 38
10.3 Pictures in class folders (Kaggle or your own files) . . . 38
10.4 Segmentation: images and masks . . . 40
10.5 Satellite images: Sentinel-2, Sentinel-1, any bands . . . 40
10.6 A large exported scene with a label image (Google Earth Engine) . . . 40
10.7 Several classes per image (multi-label) . . . 43
10.8 Two sensors: Sentinel-2 and Sentinel-1 together (fusion) . . . 43

**11 The Code tab: what the generated script does** **43**

**12 The Connection tab** **48**
12.1 Machines . . . 48

**13 Training** **48**
13.1 Where to train . . . 48
13.2 Training settings . . . 49
13.3 Unbalanced classes . . . 51
13.4 Check data before training . . . 51
13.5 Following a run . . . 51
13.6 Several seeds, comparing runs, exporting results . . . 52
13.7 Independence from the connection . . . 52
13.8 EC2 spot instances . . . 52
13.9 Can my GPU train this network? . . . 53
13.10 How large a design the application handles . . . 53

**14 The Results tab** **54**

**15 The demo architectures** **54**
15.1 LeNet-5 (1998) . . . 54
15.2 Small CNN for CIFAR-10 . . . 55
15.3 AlexNet (2012) . . . 55
15.4 VGG-16 (2014) . . . 56
15.5 GoogLeNet / Inception v1 (2014) . . . 56
15.6 ResNet-18 (2015) . . . 56
15.7 MobileNet v1 (2017) . . . 57
15.8 DenseNet, small CIFAR version (2017) . . . 57
15.9 Transformer encoder on image rows (2017) . . . 57
15.10 ViT-Base/16 (2020) . . . 58
15.11 ViT-Tiny for CIFAR-10 . . . 58
15.12 ViT-g/14, one billion parameters (2022) . . . 58
15.13 U-Net (2015) . . . 59
15.14 Attention U-Net (2018) . . . 59
15.15 MSG-Former: an NN Builder original . . . 60
15.15.1 The idea . . . 60
15.15.2 The multi-scale gated residual block . . . 60
15.15.3 The whole network . . . 61
15.15.4 Training it . . . 64
15.15.5 Experiments to run (ablations) . . . 64
15.15.6 Adapting it . . . 64
15.15.7 Status . . . 65

**16 Troubleshooting** **65**

**17 Privacy and security** **65**

**A Glossary** **66**

# 1 What NN Builder is

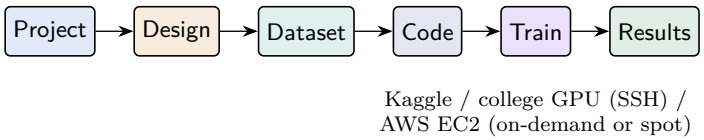
## 1.1 The idea

Building a neural network usually means writing code, running it, and finding out from an error message (or worse, from silently bad results) that a layer received a tensor of the wrong shape. NN Builder moves that discovery to the moment of design. It is inspired by visual programming tools such as MIT App Inventor: blocks are placed and connected, and the tool refuses or explains combinations that cannot work.

Three principles shape the application:

1. **The design computer only designs.** Shapes, parameter counts and the generated code are computed by an engine written in Rust inside the application, instantly and offline. No deep-learning library is installed locally, and no dataset is downloaded to the design computer.
2. **Training happens on a GPU elsewhere, automatically.** One button generates the script, uploads it, prepares the machine if needed, starts the run, follows it, and brings the results back.
3. **A run does not depend on the connection.** On a server or an EC2 instance the run is started detached; it continues when the laptop sleeps, the network drops or the application is closed, and the application reconnects to it later.

## 1.2 The workflow at a glance




The tabs in the top bar follow this order. **Connection**, set apart on the right, holds the tokens and machines that training needs, and is usually visited once.

## 1.3 What it is not

NN Builder covers image classification and image segmentation. It does not (yet) train on text, audio or tabular data, and it does not run training on the design computer, even for a tiny network. The generated script, however, is ordinary PyTorch [29]: it can be copied, read, edited and run anywhere by hand.

# 2 Installing, updating and the window

## 2.1 Installing

Installers are published with every release on GitHub (`https://github.com/varunjha089/NN-Builder/releases/latest`):

- **macOS** (Apple silicon): the file ending in `_aarch64.app.zip`. Unzip it and move `NN Builder.app` to the Applications folder. The application is not signed by Apple, so the first start may need a right-click on the application and **Open**.
- **Windows**: `NN-Builder_X_x64-setup.exe`.

- **Linux**: `nn-builder_X_amd64.deb` or the portable `.AppImage`.

### 2.2 Automatic updates

Once a day (if enabled), the application asks GitHub for the latest release. When a newer version exists, a banner appears under the top bar with **Install and restart**. The download is checked against the SHA-256 checksum published with the release before anything is replaced. The switch is in **Settings** (and on the Projects page); **Help** → **Check for Updates...** checks at once.

### 2.3 The top bar

From left to right: the name, the **Project** button (the current project, or *No project*; it opens the Projects page), a **not saved** badge and a **Save...** button when the open project is an unsaved demo, the tabs **Design**, **Dataset**, **Code**, **Train**, **Results**, then on the right a summary ("*N params · ready*" in green, or "*design has problems*" / "*dataset incomplete*" in red), the day/night button (a moon in day mode, a sun in night mode), the **Settings** button, and **Connection** (a red dot means something is still missing there).

### 2.4 Menus and keyboard shortcuts

| Menu | Item (shortcut) | What it does |
|---|---|---|
| NN Builder | Settings... (`Cmd+,`) | opens the Settings page |
| File | New Project... (`Cmd+N`) | Projects page, cursor in the name field |
| | Open Project... (`Cmd+O`) | choose a project folder |
| | Save Project... (`Cmd+S`) | keeps an unsaved demo as a real project |
| | Close Project | back to the Projects page with no project open (an unsaved demo is discarded) |
| | Delete (Projects page) | asks in the row, then moves the project's folder (design, runs, results) to the Trash, from where it can be put back; a missing project is only taken off the list |
| | New from Demo ▷ | opens one of the fifteen demo architectures |
| | Import / Export Design... | reads or writes a design (`model.json`) |
| | Export train.py... (`Cmd+E`) | saves the generated training script |
| | Show Project in Finder (`Shift+Cmd+R`) | reveals the project folder |
| Edit | Undo (`Cmd+Z`), Redo (`Shift+Cmd+Z`) | undo/redo changes to the design (in a text field: the typing) |
| | Arrange Left to Right (`Cmd+L`) | lays the canvas out in columns by depth |
| View | Night Mode (`Shift+Cmd+N`) | switches the neon night theme |
| Connection | Open Connection (`Shift+Cmd+K`) | the Connection page |

| Menu | Item (shortcut) | What it does |
|---|---|---|
| Help | Install Tools... | installs the `kaggle` and `boto3` helpers |
| | Documentation (Cmd+?) | opens this manual (a PDF inside the application) |
| | Check for Updates... | asks GitHub for a newer release |

### 2.5 Day and night themes, Settings

The night theme draws panels dark and boxes and text as neon glows; every layer glows in the colour of its kind (Conv2D orange, pooling teal, normalisation violet, Dense blue, joins red, outputs green, transformer blocks teal and violet). In night mode the Design screen is split into three separate panels (blocks on the left, canvas in the middle, inspector on the right). Their background colours are chosen in **Settings** → **Night panel colours**: six presets (Midnight, Deep purple, Ocean, Carbon, Forest, Crimson) or a colour picker per panel, with a live preview; a warning appears for colours so light that neon text becomes hard to read. The theme, the panel colours and the automatic-update switch are all on the Settings page (Figure 1); Figure 2 shows the day theme.

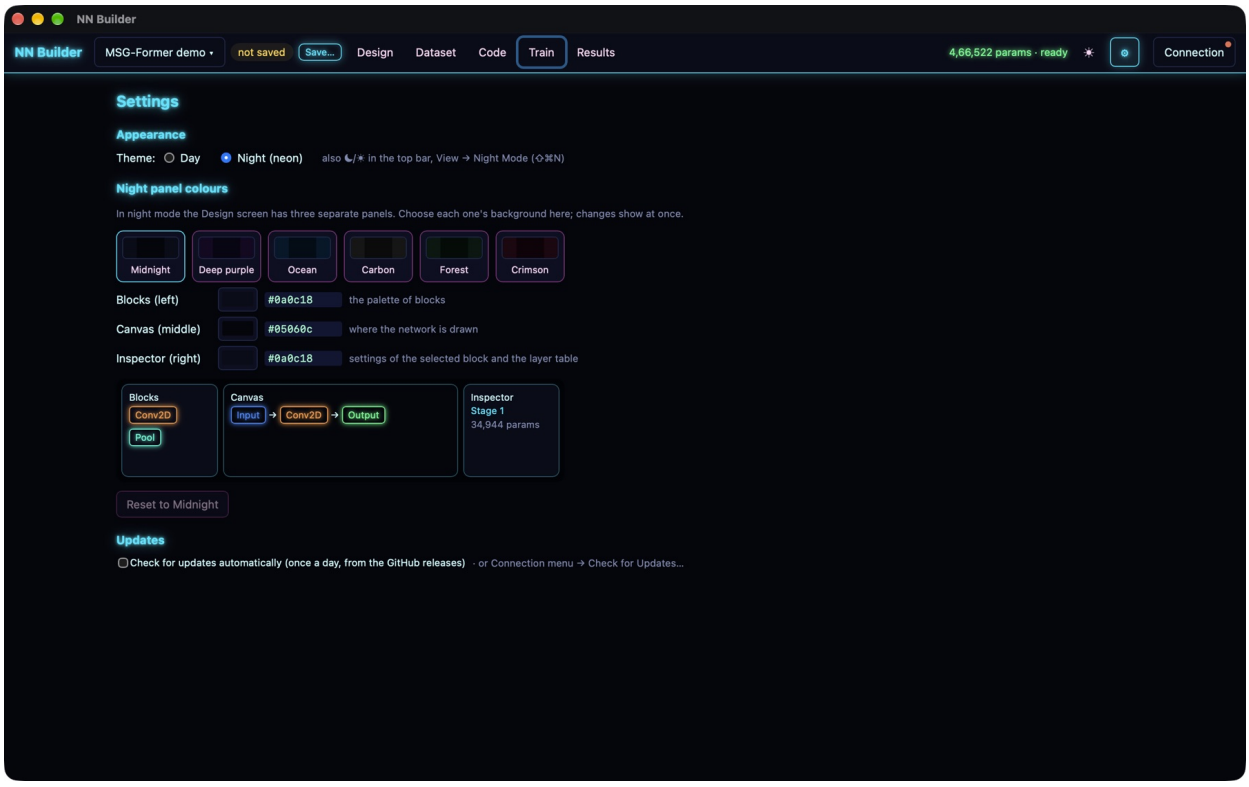



Figure 1: Settings: the theme, the six presets and a colour picker per panel for the night Design screen, a live preview, and the automatic-update switch.

## 3 Projects

### 3.1 Creating, opening, saving

A *project* is a folder. The Projects page has two cards side by side:

- **New project**: type a name and press **Create** (or Enter). The folder is made in the projects

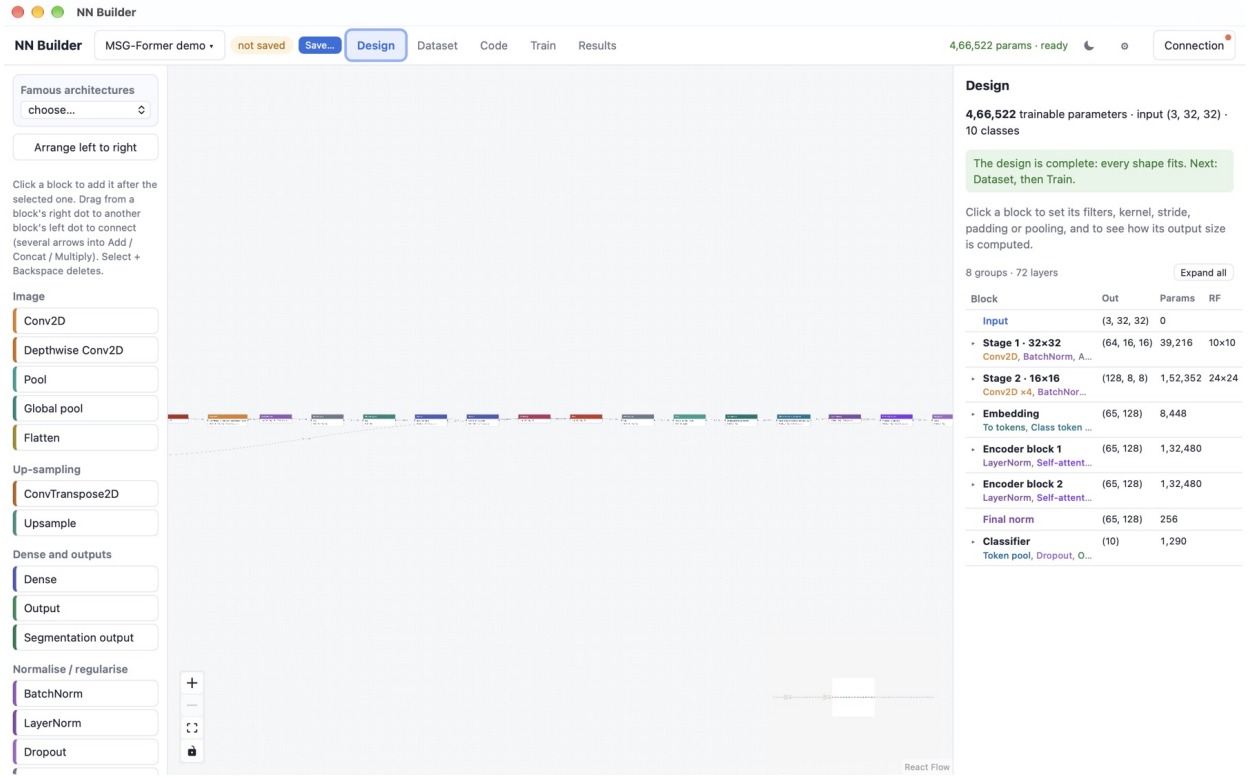



Figure 2: The Design screen in the day theme (MSG-Former, fitted to the canvas; the layer table on the right).

folder shown underneath (**change** picks another one).

- **Open**: the projects used before, each with its folder, and a field to open any project folder by its path.

Every change is saved as it is made; there is no "save" step for a normal project.

## 3.2 Demo architectures are unsaved until you save them

Below the two cards, **Start from a demo architecture** shows the fifteen demos as tiles (the section folds with its arrow). Clicking a tile opens that network on the canvas as an *unsaved* project: nothing is written to the projects folder. The top bar then shows **not saved** and **Save...**. Saving (**Save...** or `Cmd+S`) asks for a name and moves it into the projects folder as a normal project. An unsaved demo is discarded, without asking, when another demo or project is opened or created, and when the application quits. This keeps the projects folder free of "AlexNet demo 2, 3, 4...". A demo cannot be trained before it is saved, so that runs always belong to a kept project.

## 3.3 What a project folder contains

| | |
|---|---|
| `project.json` | name, creation date |
| `model.json` | the design: blocks (with their settings and positions) and arrows |
| `dataset.json` | the dataset choice |
| `train.json` | the training settings |
| `runs/<id>/` | one folder per training run (code, metrics, log, results, model) |

A design can also be exported on its own (**File** → **Export Design...**) and imported into another project.

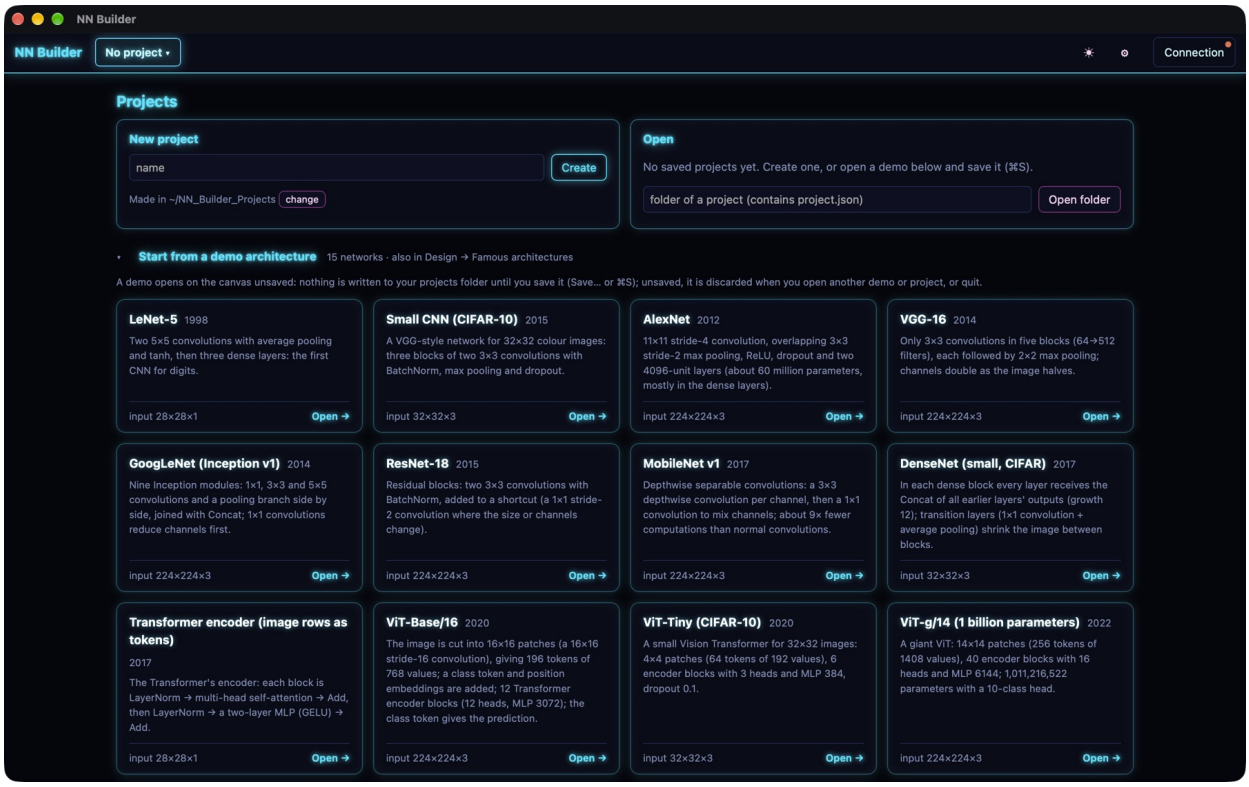



Figure 3: The Projects page (night theme): New project and Open side by side, then the demo architectures as tiles.

# 4 The Design screen

## 4.1 Layout

**Blocks (left).** The palette, grouped as *Image* (Conv2D, Depthwise Conv2D, Pool, Global pool, Flatten), *Up-sampling* (ConvTranspose2D, Upsample), *Dense and outputs* (Dense, Output, Segmentation output), *Normalise / regularise* (BatchNorm, LayerNorm, Dropout, Activation), *Transformer* (To tokens, Class token + position, Self-attention, Token pool) and *Branches* (Add, Concat, Multiply). Above them: the **Famous architectures** picker and **Arrange left to right**.

**Canvas (middle).** The network, read from left to right. Each block shows its title (with × for an error and a warning sign for a warning), a one-line summary of its settings, and its output shape with its parameter count. Each arrow shows the shape of the tensor it carries. Zoom and pan with the trackpad or the controls; a minimap sits in the corner.

**Inspector (right).** With a block selected: its settings, its shape formula with its own numbers, its receptive field, the sliding-window animation (convolution and pooling), and the skip-connection tool. With nothing selected: the whole design's parameter count, its problems, and the layer table.

## 4.2 Working with blocks

- **Add:** click a block in the palette; it is added after the selected block (and connected to it).
- **Connect:** drag from a block's right dot to another block's left dot. Add, Concat and Multiply accept several arrows; all other blocks take one.

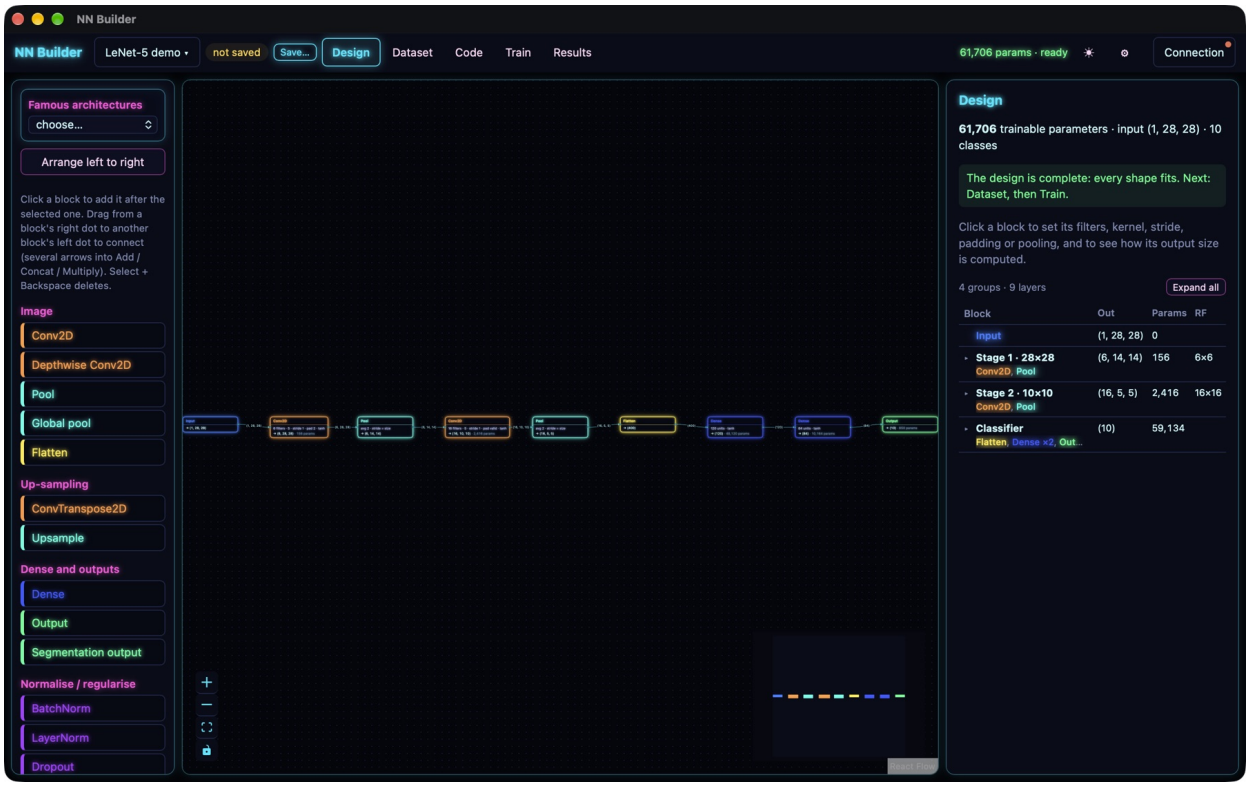



Figure 4: The Design screen with LeNet-5: blocks on the left, the network on the canvas (every arrow labelled with its tensor shape, every block with its output and parameters), and on the right the whole design's summary and the grouped layer table.

- **Delete:** select a block or an arrow and press Backspace. The Input block cannot be deleted.
- **Move:** drag blocks; **Arrange left to right** (Cmd+L) places them in columns by depth (the longest path from the Input), stacking parallel branches in their column.
- **Undo / Redo:** Cmd+Z / Shift+Cmd+Z; changes made within 0.6 seconds of each other count as one step.
- **Copy / paste:** select blocks (Shift-click or drag a box with Shift) and use Cmd+C / Cmd+V: the copies, with the arrows between them, appear below the originals, ready to be connected.
- **Residual block after selected:** inserts Conv 3×3 → BatchNorm → ReLU → Conv 3×3 → BatchNorm → Add (with a skip from the selected block) → ReLU, keeping its size and channels.
- **Collapse repeated blocks (×$N$):** draws identical consecutive groups as one node, for example the 40 encoder blocks of ViT-g/14 as "Encoder block ×40" (287 blocks become 8). It is a read-only view; double-click the node or press **Show every block** to edit again.

### 4.3 The layer table

With nothing selected, the inspector lists the layers grouped into stages, each with its output shape, its parameter subtotal and the receptive field at its end:

- **Input**.
- **Image stages** by resolution ("Stage 2 · 27×27"); a pooling layer, a Dropout after it, and small vector layers inside a stage (a squeeze-and-excitation gate) stay with their stage; decoder stages that grow the image are marked with an up arrow.

- **Transformers**: *Embedding* (with a patch convolution before it), *Encoder block 1, 2, …*, *Final norm*.
- **Classifier**: the vector layers at the end.

Each group shows its contents in block colours ("Conv2D ×3, Pool") and opens with a click (**Expand all** opens all).

### 4.4 Famous architectures and the skip-connection tool

**Famous architectures** replaces the current design with a demo (the previous design is kept as `model.previous.json`) and adjusts the dataset's image size to the one the architecture was made for. The skip-connection tool (inspector, with a block selected) joins an earlier block's output to the selected block's output with *Add* (a residual connection, as in ResNet [9]) or *Concat* (as in DenseNet [15] and U-Net [32]). When the shapes differ, it inserts a projection: a 1×1 convolution with the right stride (images), a pooling layer, or a Dense layer (vectors).

## 5 Designing a network, step by step

This section builds a small convolutional network for CIFAR-10 [18] (32×32 colour photos, 10 classes).

**1. A project and the data first.** Create a project "CIFAR CNN". Open **Dataset**, choose **Built-in (torchvision)** and **CIFAR-10**. The Input block now says $(3, 32, 32)$: three colour channels of 32×32 pixels. (Choosing the data first means every shape shown while designing is the real one.)

**2. A convolution.** In **Design**, select the Input block and click **Conv2D**. In the inspector set **Filters** to 32 and choose the preset "3×3, stride 1 (keep size)". The block shows $\rightarrow (32, 32, 32)$ and 896 parameters ($3 \cdot 3 \cdot 3 \cdot 32$ weights + 32 biases). The arrow from Input shows $(3, 32, 32)$.

**3. Downsampling.** With the convolution selected, click **Pool**: max pooling 2×2, stride "same as pool size". Output $(32, 16, 16)$, no parameters. The inspector animates the 2×2 window sliding with stride 2: every window position gives one output value.

**4. Repeat, wider.** Add another Conv2D (64 filters, 3×3, same) and Pool: $(64, 8, 8)$. A third pair with 128 filters gives $(128, 4, 4)$. This is the common pattern: *the image gets smaller and deeper*.

**5. From image to vector.** A classifier needs one vector per image. Two choices:

- **Flatten**: $(128, 4, 4) \rightarrow (2048)$. Keeps the spatial layout, but the next Dense layer then has many weights.
- **Global pool** (average): $(128, 4, 4) \rightarrow (128)$, one number per channel [23]. Far fewer parameters, and it works for any image size.

Choose Global pool here.

**6. The head.** Add **Dense** (128 units, ReLU), **Dropout** (rate 0.3) [12, 36], then **Output**. The Output block reads the number of classes from the dataset (10) and has $128 \cdot 10 + 10 = 1290$ parameters. The top bar now says "*. . . params · ready*".

**7. Make it better.**

- Insert **BatchNorm** [16] after each convolution (set the convolution's activation to *none*, and add an **Activation** ReLU after the BatchNorm): faster, more stable training.
- Select the second convolution and use the skip-connection tool to add its input to its output (*Add*): a residual block. Where the channel counts differ, the tool adds the 1×1 projection for you.
- Look at the layer table: it now shows Stage 1 · 32×32, Stage 2 · 16×16, Stage 3 · 8×8, Classifier, each with its parameter subtotal.

**8. Check, then train.** The **Code** tab shows the generated script; **Train** starts it on Kaggle, a server or EC2 (Section 13).

### 5.1 When something is wrong

A block turns **red** when the design cannot work (training is blocked) and **yellow** when it works but probably not as intended. Blocks after a red block are greyed out ("fix the layer above"); blocks not connected to the Input are faded and ignored. The reasons are written on the block and in the inspector. The checks include:

| Level | Check (as the application words it, shortened) |
|---|---|
| error | exactly one Input; exactly one Output (or Segmentation output), and it must be last; no loops |
| error | the image is too small for the window ("use a smaller kernel, padding, or fewer pooling layers before this") |
| error | Dense / Output on an image: "add Flatten or Global pool before it" |
| error | an image block (convolution, pooling) after Flatten or after To tokens |
| error | Add with different shapes ("use a 1×1 Conv2D to match channels, or a stride to match sizes") |
| error | Concat with different heights/widths; Multiply with incompatible shapes |
| error | attention: the token dimension must divide evenly by the number of heads |
| error | BatchNorm on tokens (use LayerNorm); Output on tokens (add Token pool) |
| error | segmentation: the output must return to the mask size ("Upsample / ConvTranspose2D") |
| error | pooling padding more than half the window; output padding not smaller than the stride (PyTorch rules) |
| error | the dataset has no classes yet |
| warning | the stride does not divide the input evenly: the last rows/columns are ignored |
| warning | stride larger than the window: pixels between windows are skipped |
| warning | "same" padding with an even kernel cannot keep the size exactly |
| warning | a very large Dense layer after Flatten ("try Global pool instead") |
| warning | a block whose result is not used; Flatten on an input that is already flat |

# 6 The arithmetic behind the numbers

The engine computes everything below exactly as PyTorch does; on every change to the code, an automatic test on GitHub builds hundreds of random designs (and all demos) in real PyTorch and compares every shape and parameter count with the application's (most recently: 296 designs, 0 differences). A good visual guide to this arithmetic is Dumoulin and Visin [7].

## 6.1 Output size

For a convolution or pooling layer with input size $n$ (height or width), kernel $k$, stride $s$, padding $p$ and dilation $d$:

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - d\,(k-1) - 1}{s} \right\rfloor + 1. \tag{1}$$

- **same**: padding chosen so that, with stride 1, $n_{\text{out}} = n$ (for an odd kernel, $p = d(k-1)/2$). With stride $s > 1$ the application computes the explicit padding PyTorch needs (it refuses `padding="same"` with a stride), giving $n_{\text{out}} = \lceil n/s \rceil$.
- **valid**: $p = 0$.
- **a number**: that many pixels of zeros on each side.

Dilation [44] spreads the kernel: a 3×3 kernel with dilation 2 covers 5×5 pixels with 9 weights. For a transposed convolution (up-sampling):

$$n_{\text{out}} = (n-1)\,s - 2p + d\,(k-1) + p_{\text{out}} + 1, \tag{2}$$

so a 2×2 kernel with stride 2 exactly doubles the size. Upsample multiplies the size by its scale.

## 6.2 Computation

The Design screen's summary also gives the **multiply–accumulate operations (MACs) per image**: convolutions (weights × output positions), dense layers (per token on a sequence) and attention ($4Nd^2 + 2N^2d$); normalisation, activations, pooling and joins are counted as zero. ResNet-18 at 224 × 224 comes to the usual 1.8 G.

## 6.3 Parameters

| Block | Parameters | Example |
|---|---|---|
| Conv2D | $k_h k_w C_{\text{in}} C_{\text{out}} + C_{\text{out}}$ (bias) | 3×3, 64→128: 73 856 |
| Depthwise Conv2D | $k_h k_w C_{\text{in}} m + C_{\text{in}} m$ | 3×3, 512 channels: 5 120 |
| ConvTranspose2D | $k_h k_w C_{\text{in}} C_{\text{out}} + C_{\text{out}}$ | 2×2, 512→256: 524 544 |
| Dense / Output | $n_{\text{in}} n_{\text{out}} + n_{\text{out}}$ | 4096→4096: 16 781 312 |
| BatchNorm | $2C$ (scale and shift) | 64 channels: 128 |
| LayerNorm | $2D$ | 768 values: 1 536 |
| Self-attention | $4d^2 + 4d$ (query, key, value, output) | $d = 768$: 2 362 368 |
| Class token + position | $d + (N+1)\,d$ | 196 tokens, $d = 768$: 152 064 |
| Pool, Global pool, Flatten, Dropout, Activation, joins | 0 | |

Convolutions followed by BatchNorm are created without a bias in the demos (the BatchNorm shift makes it redundant), as torchvision does.

### 6.4 Receptive field

The receptive field of a block is the size of the input region that one of its output values can see. With $r_0 = 1$ and $j_0 = 1$ (the "jump", the distance in input pixels between neighbouring outputs), each layer with kernel $k$, dilation $d$ and stride $s$ gives

$$r_\ell = r_{\ell-1} + d\,(k-1)\,j_{\ell-1}, \qquad j_\ell = j_{\ell-1}\,s. \tag{3}$$

The inspector shows it for every block ("one output value here sees 99×99 input pixels"). A network should usually reach a receptive field about the size of the objects it must recognise; the *effective* receptive field is smaller and concentrated in the centre [25].

### 6.5 Memory needed for training

A useful estimate for training with Adam [17] in mixed precision [27] is about **16 bytes per parameter** before any images: 4 for the weights, 4 for the gradients and 8 for Adam's two running averages. On top come the activations kept for the backward pass, which grow with the batch size and the image size. Examples: ResNet-18 (11.2 M parameters) needs about 0.2 GB for its state; ViT-Base (86 M) about 1.4 GB; ViT-g/14 (1.01 B) about 16 GB, more than a 16 GB T4 has. If a run fails with "CUDA out of memory", halve the batch size; beyond that, gradient checkpointing [3] and sharding the optimizer state over several GPUs [30] are the standard remedies (not built into NN Builder).

## 7 Reference of all blocks

| Block | Shape in → out | Settings and notes |
|---|---|---|
| Input | $\to (C, H, W)$ | set by the dataset (channels, height, width) |
| Conv2D | $(C, H, W) \to (F, H', W')$ | filters, kernel (with a "square" link: unlink for height×width), stride, padding (same / valid / number), dilation, activation, bias; presets: 3×3 keep size, 3×3 stride 2 (halve), 1×1 (mix channels), 5×5, 7×7 stride 2 (ResNet stem) |
| Depthwise Conv2D | $(C, H, W) \to (Cm, H', W')$ | one filter per input channel (channel multiplier $m$); with a following 1×1 Conv2D it forms a depthwise separable convolution [13] |
| ConvTranspose2D | $(C, H, W) \to (F, H'', W'')$ | learnt up-sampling; filters, kernel, stride, padding, output padding |
| Pool | $(C, H, W) \to (C, H', W')$ | max or average; pool size; stride (default: same as size); padding |
| Global pool | $(C, H, W) \to (C)$ | average or max over the whole image |
| Upsample | $(C, H, W) \to (C, sH, sW)$ | scale, nearest or bilinear; no parameters |
| BatchNorm | same shape | normalises each channel over the batch [16]; images and vectors only |
| LayerNorm | same shape | normalises each sample [1]; on tokens, over the dimension; on an image, over (channels, height, width) |
| GroupNorm | same shape | normalises each image over groups of channels; independent of the batch, so good for small batches; the channels must divide by the number of groups (often 32) |

| Block | Shape in → out | Settings and notes |
|---|---|---|
| Dropout | same shape | rate: the fraction of values set to zero during training [36] |
| Flatten | $(C, H, W) \to (CHW)$ | |
| Dense | $(n) \to (u)$; tokens $(N, D) \to (N, u)$ | units, activation; on tokens it acts on each token (the transformer MLP) |
| Activation | same shape | ReLU, Leaky ReLU, GELU [11], SiLU [31], sigmoid, tanh |
| Add | equal shapes → same | element-wise sum: residual connections [9] |
| Concat | joins along channels | images need equal height and width; the arrow order is the channel order |
| Multiply | see notes | equal shapes; a one-channel map times an image (attention gate [28]); a per-channel vector times an image (squeeze-and-excitation [14]) |
| To tokens | $(C, H, W) \to (N, D)$ | *patches*: every pixel of the feature map becomes a token of $C$ values ($N = HW$), used after a patch convolution as in ViT [6]; *rows*: every image row is a token |
| Class token + position | $(N, D) \to (N{+}1, D)$ | a learnt class token in front, and learnt position embeddings added |
| Self-attention | $(N, D) \to (N, D)$ | heads, dropout; multi-head scaled dot-product attention [40] |
| Token pool | $(N, D) \to (D)$ | the class token, the mean of all tokens, or the last one (after an LSTM / GRU) |
| Cross-attention | $(N, D), (M, D_2) \to (N, D)$ | the first input's tokens attend to the second's (queries; keys and values); heads, dropout |
| RMSNorm | same shape | per token: divided by its root mean square, times a learnt scale (LLaMA) |
| SwiGLU | same shape | the gated MLP $W_2(\text{silu}(W_1 x) \cdot W_3 x)$; hidden units |
| Output | $(n) \to (K)$ | $K$ = number of classes; softmax/cross-entropy is in the training script |
| Segmentation output | $(C, H, W) \to (K, H, W)$ | a 1×1 convolution giving a class score for every pixel; must match the mask size |
| Early exit | $(n) \to (K)$ | an extra classifier part way through, trained with the same labels (its loss × weight); Results show what stopping confident images there would give |
| Mixture of experts | same shape | a router sends every token to its top $k$ of $E$ expert MLPs; a load-balancing loss keeps all in use |
| Detection output | $(C, h, w) \to (K{+}4, h, w)$ | anchor-free boxes (CenterNet [49]): per cell a score per class, the box's width and height and the centre's offset; the image must be a whole number of cells (Section 9.11) |
| Conv3D | $(C, T, H, W) \to (F, T', H', W')$ | a time series: filters, kernel (pixels), dates per window, stride (pixels), padding same / valid (Section 9.12) |
| Temporal pool | $(C, T, H, W) \to (C, H, W)$ | the dates of every pixel combined: mean, maximum, or attention (weights learnt per pixel, as in U-TAE [46]) |
| To sequence | $(C, T, H, W) \to (T, C)$ | the mean of every date's pixels: one vector per date |
| Conv1D | $(T, D) \to (T', F)$ | a convolution along a sequence: filters, kernel, stride, padding |

| Block | Shape in → out | Settings and notes |
|---|---|---|
| LSTM, GRU | $(T, D) \to (T, u)$ | recurrent layers along a sequence [48]; units, both directions ($2u$ out); Token pool *last* keeps the last step |
| Subtract | equal shapes → same | the first input minus the second, or $\lvert a - b \rvert$: the difference of two branches (Siamese change detection) |
| Shared weights | as the shared block | runs another block's layer, with the same weights, on its own input of the same shape; the weights count once (Siamese / twin networks, Section 9.13) |
| Custom (PyTorch) | declared by the class | runs a class from the design's own PyTorch code (an `nn.Module` given the input shape; a `# out:` line declares the output shape); the inspector edits the code; checked on the training machine (Section 7.1) |

## 7.1 Your own layers in PyTorch (Custom block)

When a layer is not in the block list (a new attention variant, a gate from a paper, a pixel shuffle), write it in PyTorch. Add a **Custom (PyTorch)** block (palette: *Your own*) and select it. The inspector shows the design's PyTorch code, shared by all its Custom blocks, and a **Class** list with the classes in it; **Insert an example** adds a squeeze-and-excitation class to start from. Each class is an `nn.Module` at the top level of the code:

- `__init__(self, c, h, w)` receives the shape of the block's input (`t, d` for tokens, `n` for a vector), so the class can size its layers;
- a line `# out: c * 2, h // 2, w // 2` in the class declares the output shape in those names (without it, the output has the input's shape). NN Builder uses it to check and size the rest of the design;
- `forward(self, x)` returns the output. `torch` and `nn` are imported; import anything else at the top.

The Mac never runs this code. It is copied into `train.py` above the network, and on the training machine the one-batch check compares each Custom block's real output shape with the declared one before any training, stopping with the block's name if they differ. The parameters of Custom blocks are counted there too (the Design tab's total leaves them out). In the Text tab the code is the `python """ ... """` section and the block is `custom CLASS` (Section 8.11).

The loss can be your own as well: **Loss: your own (Python)** in the Train tab opens a box with `def custom_loss(out, y):` (an example for the task is filled in). `out` is the network's output for a batch and `y` the labels as the data gives them (class numbers; 0/1 per class for multi-label; a class per pixel with $-100$ for unlabelled pixels; the standardised targets for regression); it returns one number. The validation loss uses it too.

## 7.2 Transformer options: masks, positions, LLaMA-style blocks, cross-attention

**Self-attention** has two switches: **causal** (each token sees only itself and the tokens before it, as in a decoder or for a time series that must not look ahead) and **rotary positions** (RoPE: the queries and keys are rotated by an angle that grows with the position, so attention depends on how far apart two tokens are; use it with **Positions: none** in Class token + position). **Positions** can also be fixed sines and cosines (no parameters). **RMSNorm** and **SwiGLU** give LLaMA-style blocks (RMSNorm → attention → Add, RMSNorm → SwiGLU → Add). **Cross-attention** joins two token streams:

the first arrow's tokens ask, the second's are looked at, e.g. Sentinel-2 patch tokens attending to Sentinel-1 tokens (mid fusion with attention instead of Concat), or one date's tokens attending to the others. Concat joins token sequences one after the other. In a time series, dates with no data at all (every pixel empty) are filled from the dates before and after, so attention needs no mask for them; a token embedding for text is not part of NN Builder (it trains on images).

### 7.3 Frozen layers, LoRA, experts and early exits

**Frozen.** Every block with weights has a **frozen** switch (text: the word `frozen`): its weights stay as they are, e.g. the first layers of a network started from ImageNet weights. The Design tab shows how many weights are trained and how many frozen.

**LoRA.** Dense and Self-attention have a **LoRA rank**. The layer's own weights are frozen and a low-rank change $B \cdot A$ (rank $r$, $B$ starting at zero) is learnt instead [50]: with ImageNet weights and **train only the new layers**, the loaded weights stay and only the adapters and the new head learn, a small fraction of the weights. Text: `dense 64 lora 8`, `attention heads 8 lora 4`.

**Mixture of experts.** The **Mixture of experts** block (tokens or vectors) holds $E$ small expert MLPs and a router; every token goes to its top $k$ experts, whose outputs are mixed by the router's weights (Switch, Mixtral). Many more weights at the cost of $k$ experts per token; a load-balancing loss (Switch Transformer, $\times 0.01$) keeps every expert in use. Text: `moe 8 top 2 hidden 256`.

**Early exits.** An **Early exit** after a Global pool part way through a classifier is an extra Output trained with the same labels (its loss times its **share**). Results list each exit's accuracy and what stopping every image at the first exit that is at least 90 % sure would give: the accuracy and the share of images leaving at each exit, i.e. how much computation could be saved. Text: `exit 0.3`.

### 7.4 Several GPUs

On a machine with several GPUs (a college server, a multi-GPU EC2 instance such as g4dn.12xlarge or g5.12xlarge, a Kaggle notebook with two T4s) the Train tab's **GPUs** can be *all the machine has* or a number. `train.py` then starts one worker per GPU itself (nothing else changes in how the run is started) and trains with PyTorch's DistributedDataParallel: every worker holds the whole network and takes its share of every batch, the gradients are averaged after each step, and BatchNorm counts the images of all GPUs. The batch size stays the total, so the training goes as on one GPU, in about $1/N$ of the time when the data loading keeps up. Worker 0 evaluates, writes the events, checkpoints and results; the stop button, the spot interruption notice and the timed checkpoints reach every worker (the run's process forwards them), all stop at the same step, and a resumed run continues at the same batch. With several GPUs a batch that does not fit is not split automatically (use a smaller batch or **save memory**); a data check uses one GPU. The Kaggle API cannot choose the T4×2 accelerator: a Kaggle run gets one GPU unless the notebook is set to two. A model too big for one GPU's memory (FSDP, splitting the weights over the GPUs) is not supported yet.

### 7.5 Transformers in NN Builder

A transformer works on a *sequence of tokens*, shape $(N, D)$: $N$ tokens of $D$ values. To use one on images:

1. cut the image into patches with a convolution whose kernel equals its stride (ViT: 16×16, stride 16, $D$ filters), giving $(D, H/16, W/16)$;
2. **To tokens** (patches) turns that map into $N = (H/16)(W/16)$ tokens;

3. **Class token + position** adds a learnt class token and position information (attention by itself ignores order);
4. repeat the *encoder block*: LayerNorm → Self-attention → Add (with the block's input), then LayerNorm → Dense (GELU, usually $4D$ units) → Dense ($D$) → Add. Normalising before each part ("pre-norm") trains more stably than after it [43];
5. a final LayerNorm, **Token pool** (class token), **Output**.

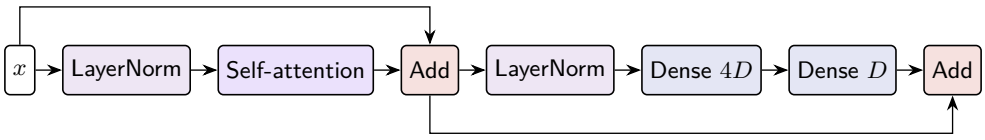



Figure 5: A pre-norm Transformer encoder block as it is built on the canvas.

### 7.6 Segmentation

For segmentation (Dataset → **Segmentation**) every pixel gets a class. The network must end with **Segmentation output** at the size of the masks, so an encoder that shrinks the image needs a decoder that grows it again (ConvTranspose2D or Upsample), as in U-Net [32]. The training script uses a per-pixel cross-entropy loss and reports pixel accuracy and the mean intersection over union (mIoU) of the classes.

## 8 Writing designs as text: the NNB language

Every design can also be written as text, in the **Text** tab. The text and the canvas are two views of the same design and are kept in step both ways: typing changes the canvas a moment after the text is valid, and a change on the canvas changes the text. This chapter teaches the language from the first line to functions and loops, then lists every block and every rule.

Why write text at all? A network of fifty blocks is fifty short lines, typed in a minute; every number of every block is visible at once; repeated parts are written once (`repeat 40 { ... }`); and the text can be copied into a paper, an e-mail or a chat and pasted back unchanged (**Copy**, or **File** → **Export Design...** as a `.nnb` file).

### 8.1 A first design, line by line

The smallest useful design is a chain of blocks. LeNet-5 (Section 15) for the $1 \times 28 \times 28$ MNIST digits:

lenet5.nnb

```
net "LeNet-5"
# MNIST digits, 1×28×28. One block per line: a label, the block, its settings.
1 input
2 conv 6 5x5 pad 2 tanh      # 6 filters, 5×5 kernel, padding 2 keeps 28×28
3 avgpool 2x2                # 28×28 -> 14×14
4 conv 16 5x5 valid tanh     # no padding: 14 -> 10
5 avgpool 2x2                # 10 -> 5
6 flatten                    # 16×5×5 = 400 values
7 dense 120 tanh
8 dense 84 tanh
9 output                     # one score per class (10 for MNIST)
```

- `net "LeNet-5"` gives the design a name (optional; see Section 8.2). It is not a block: it makes nothing on the canvas.
- Each further line is one block: a **label** (`1`, `2`, ...), the **block** (`conv`, `avgpool`, ...) and its **settings** in any order. Left-out settings take the same defaults as the canvas (a `conv` without a kernel is $3 \times 3$, without an activation it uses ReLU, ...).
- A block takes its input from **the line above** unless it says otherwise.
- `#` starts a comment, to the end of the line. Blank lines and indentation do not matter.
- The first block is `input`: its shape comes from the Dataset tab, not from the text. `output` gets as many classes as the dataset has.

Typed into the Text tab, this draws the nine blocks on the canvas, each with its label in its header, and the shapes and parameters are those of Section 6: 61 706 parameters.

## 8.2 The design's name: `net`

```
net "NAME"
```

A line `net` followed by a name in double quotes names the design, for example `net "LeNet-5"` or `net "ResNet-18, width 32"`. It is optional and is usually the first line. What it does:

- it says what the text is when the text travels on its own: in a `.nnb` file, pasted into a paper, an e-mail or a chat;
- the generated `train.py` carries it in its first line, after the project's name (`My project (design LeNet-5): generated by NN Builder ...`), so a script says which design it trains;
- it stays when the text is printed again (**Format**, or after a change on the canvas).

It does not rename the project (the project's name is set when the project is created or saved, and shown in the top bar), and it does not create a block. The name is any text without a double quote; if there are several `net` lines, the last one counts. It may only stand at the top level, not inside braces.

## 8.3 Settings: words and numbers

Settings are written the way you would say them. For a convolution, the first plain number is the number of filters, `5x5` is the kernel, and words set the rest. In general (capitals are your values, `[ ]` is optional, `|` means one of):

```
LABEL conv FILTERS HxW [stride S] [pad P | same | valid] [dil D] [ACTIVATION] [nobias]
      ACTIVATION: relu | leaky_relu | gelu | silu | sigmoid | tanh | none
```

For example:

```
2 conv 64 3x3                       # 64 filters, 3×3, stride 1, padding "same", ReLU
3 conv 128 3x3 stride 2 pad 1 gelu # stride 2, padding 1, GELU
4 conv 128 1x1 valid none nobias   # no padding, no activation, no bias
5 conv 64 3x3 dil 2                # dilation 2
6 conv filters=64 kernel=7x7 stride=2 padding=3   # any setting as name=value
```

Every setting also has a *name=value* form (the names are those of the inspector), useful when a value is computed (`kernel=s+2`, below). A word that a block does not know is an error, never ignored: `conv 64 stide 2` says "`'stide' means nothing for conv (did you mean 'stride'?)`". A value outside its range (a kernel of 40, 100 000 filters) is an error on its block.

### 8.4 Connections: `<-`

A block with a different input, or with several (Add, Concat, Multiply), names them after `<-`:

```
1 input
2 conv 64 3x3
3 conv 64 3x3 none
4 add <- 2, 3          # residual: the output of block 2 plus that of block 3
5 relu
6 conv 32 1x1 <- 2     # a side branch from block 2 (not from the line above)
7 concat <- 5, 6       # channels joined in this order: 5's first, then 6's
```

A block may only use blocks above it (a network computes each block once, from the blocks before it), so the text can never contain a loop. `<- none` makes a block without input (shown as not connected).

### 8.5 Labels

Labels are names, not positions: `1, 2, 5, 9` is fine, and so is a word (`stem conv 32 3x3`, then `<- stem`). The label may also be left out: `conv 32 3x3` on its own gets the next free number. Two lines may not have the same label. The labels are what the canvas shows in each block's header and what the layer table uses, so canvas, text and table name blocks the same way.

### 8.6 Names and arithmetic: `let`

`let` gives a value a name; every number may be an expression of names and numbers with `+ - * /`, `//` (whole-number division), `%` (remainder), `^` (power) and parentheses:

```
let w = 32
let depth = 6
2 conv w 3x3
3 conv w * 2 3x3 stride 2
4 dense 2 ^ 10 gelu        # 1024
```

Changing `w` changes every block that uses it: the quickest way to try a wider or narrower network.

### 8.7 Repeating a group: `repeat`

Identical parts are written once. A `repeat` line has a label, a count and the group in braces; the passes are chained (each pass takes the previous pass's output), and the line's output is the last pass's last block:

vit_small.nnb

```
1  net "Small ViT"
2  # CIFAR-10, 3×32×32: 4×4 patches give 64 tokens of d values; depth encoder blocks.
3  let d = 192
4  let depth = 6
5
6  1 input
7  2 conv d 4x4 stride 4 valid none     # patch embedding: (192, 8, 8)
8  3 to_tokens patches                  # (64, 192)
9  4 cls_pos                            # class token + positions: (65, 192)
10 5 repeat depth {                     # one pre-norm Transformer encoder block per pass
11     a ln
```

```
    b attention heads 3 drop 0.1
    c add <- in, b                  # residual: the pass's input + attention
    d ln
    e dense d * 2 gelu              # MLP ratio 2
    f dense d none
    g add <- c, f
  }
6 ln
7 token_pool cls
8 output
```

Inside the braces:

- blocks have **local labels** (`a`, `b`, ...); on the canvas they become `5.1.a` ... `5.6.g` (line, pass, block);
- `in` is the group's input (for the first pass, the block above the `repeat` line);
- `i` is the pass number, 1, 2, ...: `conv 32 * i 3x3` gets wider in every pass;
- the count may be a name (`repeat depth`); `repeat 12 as enc` names the group for readers.

This ViT has 49 blocks and 1 806 538 parameters at $3 \times 32 \times 32$; ViT-g/14 is the same pattern with `repeat 40` and 16 lines in all.

### 8.8 A loop over values: `for`

`for` repeats a group once per value, chained like `repeat`. A value can be a tuple, for several names at once:

```
5 for c in 64, 128, 256 { a conv c 3x3 }
6 for c, s in (64, 1), (128, 2), (256, 2) {
    x conv c 3x3 stride s
    y bn
  }
```

`i` is also the pass number in a `for`.

### 8.9 Functions: `def`

A function is a group of blocks with values, written once and used many times. It is defined with `def`, its parameters may have defaults, and a line calls it like a block:

resnet_small.nnb

```
1  net "Small ResNet"
2  # CIFAR-10, 3×32×32. A function for the residual block, a loop over the stages.
3  let w = 32
4
5  # two convolutions and a shortcut. With s = 2 the first convolution is 4×4 / stride 2 / padding 1, wh
6  # (kernel = s + 2), and the shortcut averages 2×2 patches before its 1×1 convolution (ResNet-D); with
7  def basic(c, s = 1) {
8    a conv c kernel=s+2 stride s pad 1 none nobias
9    b bn
10   r relu
11   d conv c 3x3 none nobias
12   e bn
13   p avgpool s <- in               # the shortcut starts from the block's input
14   q conv c 1x1 none nobias
15   n bn
16   j add <- e, n
```

```
  o relu
}

1 input
2 conv w 3x3 none nobias
3 bn
4 relu
5 for c, s in (w, 1), (w * 2, 2), (w * 4, 2) {   # three stages: 32×32, 16×16, 8×8
    x basic(c, s)
    y basic(c)                                    # s takes its default, 1
  }
6 global_pool
7 output
```

- `basic(c, s)` passes the values by position; `basic(c)` leaves `s` at its default; `basic(c=64, s=2)` names them.
- Inside the function, `in` is the input of the call (here the line above the call); the last block is the call's output.
- A function may call other functions (up to 16 deep: a function that calls itself is reported).
- Functions may be defined anywhere in the text, also below their first use.

This network has six residual blocks (66 blocks on the canvas, 791 338 parameters): about forty lines of text.

### 8.10 Several inputs, and reaching into a group

A call (or a `repeat` / `for`) can take several inputs: `<- 3, 2.3.e` gives the call two, called `in` and `in2` inside. A block made inside a group or a call is named by its path: `2.3.e` is line 2, pass 3, block `e` (for a call: line, then block, as in `7.u`). Together they make U-Net's skip connections:

unet_small.nnb

```
net "Small U-Net"
# Segmentation, 3×64×64 with masks. Functions with two inputs (in, in2) and skips into a repeat by pa
def double(c) {
  a conv c 3x3
  b conv c 3x3
}
def up(c) {
  u convT c 2x2 stride 2 none    # doubles the size
  j concat <- u, in2             # in2: the skip from the encoder
  k double(c)
}

1 input
2 repeat 3 {                     # encoder: 16, 32, 64 filters, each halving the size after
    e double(16 * 2 ^ (i - 1))
    p maxpool 2x2
  }
3 double(128)                    # bottleneck at 8×8
4 up(64) <- 3, 2.3.e             # 2.3.e = line 2, pass 3, block e (its output: 64 filters at 16×16)
5 up(32) <- 4, 2.2.e
6 up(16) <- 5, 2.1.e
7 seg_output
```

Line 2 builds the encoder as three passes whose width depends on the pass number ($16 \cdot 2^{i-1}$: 16, 32, 64); lines 4–6 each take the block above and the matching encoder output (25 blocks, 482 050 parameters for $64 \times 64$ images and two classes).

## 8.11 Every block and how its values are set

### How a value is written

A block's line is its word (`conv`, `dense`, ...) followed by its values in any order. Each value can be written in a short form, and every value also in a long form *name*=*value*:

| Form | Example | Sets |
|---|---|---|
| a plain number | `conv 64`, `dense 128`, `dropout 0.3` | the block's *main number* (filters, units, rate, ...: see each block; a second plain number is an error) |
| a size `HxW` | `3x3`, `5x3`, `2x2` | the kernel (convolutions) or the window (pools); height × width |
| x*n* | `x2` | a multiplier (depthwise) or a scale (upsample) |
| a word and its value | `stride 2`, `pad 1`, `dil 2`, `heads 8`, `drop 0.1` | that setting |
| a word alone | `gelu`, `none`, `same`, `nobias`, `max`, `bilinear` | a choice (activation, padding mode, bias off, pool mode, ...) |
| *name*=*value* | `filters=64`, `kernel=7`, `stride=2x1`, `bias=false` | any setting by its name (the names are those of the inspector); a single number for a size means a square (`kernel=7` is 7 × 7) |

Wherever a number goes, an expression may go (`conv w * 2 3x3`, `kernel=s+2`). Left-out values take the defaults below, which are also the canvas's. A value outside its range, a word the block does not have, or a second plain number is an error on its line (Section 8.13); nothing is ever changed silently. Block words are not case-sensitive (`Conv2D 16 3x3` works).

Each block starts with its general form: capitals are your values, `[ ]` is optional, `|` means one of, and `ACTIVATION` is one of `relu`, `leaky_relu`, `gelu`, `silu`, `sigmoid`, `tanh`, `none`. Every value can also be given as *name*=*value*, the label may be left out, and every block except `input` may end with `<- INPUT` (by default the line above). In the tables: *short* is how the value is written briefly, *name* its long form.

### `input` — the images

```
LABEL input [GROUP]
```

`1 input`: its shape (channels, height, width) comes from the Dataset tab; it takes no input. A design has one, except with two band groups (Dataset tab, Section 9.14: e.g. Sentinel-2 and Sentinel-1), where it may have one Input per group, each starting its own branch (mid fusion):

```
1 input 1                  # band group 1 (Sentinel-2)
2 conv 32 3x3
3 input 2                  # band group 2 (Sentinel-1)
4 conv 16 3x3
5 concat <- 2, 4           # the two branches joined
6 global_pool
7 output
```

| value | short | name | default | allowed |
|---|---|---|---|---|
| band group | `1` | `group` | 0 | 0 (all bands), 1, 2 |

### `conv` (also `conv2d`) — 2-D convolution

```
LABEL conv FILTERS HxW [stride S] [pad P | same | valid] [dil D] [ACTIVATION] [nobias]
```

```
2 conv 64 3x3                                # 64 filters, 3×3, stride 1, "same" padding, ReLU
3 conv 128 3x3 stride 2 pad 1 gelu nobias
4 conv 32 5x3 dil 2 valid none
5 conv filters=64 kernel=7 stride=2 padding=3 activation=silu bias=false
```

| Value | short | name | default | range | meaning |
|---|---|---|---|---|---|
| filters | the plain number: `64` | `filters` | 32 | 1–65 536 | output channels |
| kernel | `3x3`, `5x3` | `kernel` | $3 \times 3$ | 1–32 | window size |
| stride | `stride 2` | `stride` | 1 | 1–32 | step (`stride=2x1` for different steps) |
| padding | `same`, `valid`, `pad 1` | `padding` | same | 0–10 | `same` keeps the size; `valid` = 0 |
| dilation | `dil 2` | `dilation` | 1 | 1–6 | spacing inside the window |
| activation | `relu` `leaky_relu` `gelu` `silu` `sigmoid` `tanh` `none` | `activation` | relu | | applied after the convolution |
| bias | `nobias` (`bias`) | `bias` | true | | off before a BatchNorm |

### `dwconv` (also `dwconv2d`) — depthwise convolution

```
LABEL dwconv [xM] HxW [stride S] [pad P | same | valid] [ACTIVATION] [nobias]
```

One filter per input channel (MobileNet). `dwconv 3x3`, `dwconv x2 3x3 stride 2 pad 1 none nobias`, `dwconv multiplier=3 kernel=5x5`.

| Value | short | name | default | range | meaning |
|---|---|---|---|---|---|
| multiplier | `x2` | `multiplier` | 1 | 1–8 | output channels = input channels × multiplier |
| kernel, stride, padding, activation, bias | as for `conv` | | | | (no dilation; no plain number) |

### `convT` (also `convtranspose`) — transposed convolution

```
LABEL convT FILTERS HxW [stride S] [pad P] [outpad O] [ACTIVATION]
```

Makes the image larger (U-Net's decoder). `convT 64 2x2 stride 2 none` doubles height and width; `convT 32 3x3 stride 2 pad 1 outpad 1` too.

| Value | short | name | default | range | meaning |
|---|---|---|---|---|---|
| filters | the plain number | `filters` | 32 | 1–65 536 | output channels |
| kernel | `2x2` | `kernel` | $2 \times 2$ | 1–32 | |
| stride | `stride 2` | `stride` | 2 | 1–32 | the enlargement |
| padding | `pad 1` | `padding` | 0 | 0–10 | a number only (no `same`) |
| output padding | `outpad 1` | `output_padding` | 0 | 0–7 | extra rows / columns at the end |
| activation | as for `conv` | `activation` | relu | | (no bias setting) |

#### `maxpool`, `avgpool` (also `pool`) — pooling

```
LABEL maxpool | avgpool HxW [stride S | stride size] [pad P]
```

`maxpool 2x2` (halves the size), `maxpool 3x3 stride 2 pad 1`, `avgpool 3x3 stride 1 pad 1`, `maxpool 3` ($3 \times 3$), `pool avg size=2 stride=size`.

| Value | short | name | default | range | meaning |
|---|---|---|---|---|---|
| mode | the word `maxpool` / `avgpool`, or `max`, `avg` | `mode` | max | | maximum or average of the window |
| size | `2x2`, or the plain number `3` | `size` | $2 \times 2$ | 1–8 | window |
| stride | `stride 1`, `stride size` | `stride` | size | 1–32 | `size`: the same as the window |
| padding | `pad 1` | `padding` | 0 | 0–4 | a number only |

#### `global_pool` (also `gap`) — one value per channel

```
LABEL global_pool [avg | max]
```

`global_pool` (average over the whole image), `global_pool max`. Value: mode, `avg` / `max` (`mode=`), default avg. From $(C, H, W)$ to $(C)$.

#### `upsample` — enlarge without weights

```
LABEL upsample [xN] [nearest | bilinear]
```

`upsample x2`, `upsample 2`, `upsample x4 bilinear`. Values: scale, x$n$ or the plain number (`scale=`), default 2, range 2–8; mode, `nearest` / `bilinear` (`mode=`), default nearest.

#### `bn`, `ln`, `gn` — normalisation

```
LABEL bn | ln
LABEL gn GROUPS
```

`bn` (also `batchnorm`) and `ln` (also `layernorm`) have no values. `gn 8` (also `groupnorm`, `groups=8`): GroupNorm with 8 groups, default 8, range 1–65 536; the number must divide the channels.

#### `dropout`

```
LABEL dropout RATE
```

`dropout 0.3`, `dropout rate=0.5`. Value: rate, the plain number (`rate=`), default 0.25, range 0–0.95: the fraction of values set to zero during training.

#### `flatten`

```
LABEL flatten
```

`flatten`. No values: $(C, H, W)$ becomes one vector of $C \cdot H \cdot W$ values (before `dense`).

#### `dense` (also `linear`) — fully connected

```
LABEL dense UNITS [ACTIVATION]
```

`dense 128`, `dense 10 none`, `linear units=256 activation=tanh`. Values: units, the plain number (`units=`), default 128, range 1–1 048 576; activation, as for `conv` (`activation=`), default relu. On tokens it acts on every token separately (a Transformer's MLP).

### Activation blocks

```
LABEL relu | leaky_relu | gelu | silu | sigmoid | tanh
```

A line with only an activation word is an activation block: `relu`, `leaky_relu`, `gelu`, `silu`, `sigmoid`, `tanh`. Also `activation gelu`, `act fn=silu` (the name of the value is `fn`).

#### `add`, `concat` (`cat`), `mul` (`multiply`) — joining

```
LABEL add | concat | mul <- INPUT, INPUT [, INPUT ...]
```

No values; the inputs come after `<-`: `add <- 2, 5` (same shapes, summed: residual), `concat <- 3, 7` (channels joined in this order; same height and width), `mul <- 4, 6` (element-wise; a vector of $C$ values may multiply a $(C, H, W)$ image: an attention gate or squeeze-and-excitation). Two or more inputs.

### Transformer blocks: `to_tokens`, `cls_pos`, `attention`, `cross_attention`, `rmsnorm`, `swiglu`, `token_pool`

```
LABEL to_tokens [patches | rows]
LABEL cls_pos [cls | nocls] [learned | sinusoidal | nopos]
LABEL attention [heads] HEADS [drop RATE] [causal] [rotary]
LABEL cross_attention [heads] HEADS [drop RATE] <- ASKING, LOOKED_AT       (also xattn)
LABEL rmsnorm                                                          (also rms)
LABEL swiglu HIDDEN
LABEL moe EXPERTS [top K] [hidden UNITS]
LABEL exit [WEIGHT]                  an early exit (classification): needs a vector
... frozen                           any block with weights: not trained;  dense / attention ... lora
LABEL token_pool [cls | mean | last]
```

```
3 to_tokens patches        # (C, H, W) -> H·W tokens of C values;  "rows": H tokens of C·W va
4 cls_pos                  # a learnt class token in front + learnt positions;  "nocls": posi
5 attention heads 8 drop 0.1      # also: attention 8, attn heads=8 dropout=0.1
6 token_pool cls           # the class token's vector;  "mean": the mean of all tokens
```

`to_tokens`: mode `patches` / `rows` (`mode=`), default patches. `cls_pos`: `cls` / `nocls` (`cls=true` / `false`), default cls. `attention` (also `attn`): heads, the plain number or `heads 8` (`heads=`), default 4, range 1–512, must divide the token width; dropout, `drop 0.1` (`dropout=`), default 0, range 0–0.9; `causal`: each token sees only itself and those before it; `rotary` (also `rope`): rotary positions (RoPE) inside the attention, each head's width must be even. `cls_pos`'s positions: `learned` (default), `sinusoidal` (fixed sines and cosines; even width), `nopos` (none; `pos=`). `cross_attention`: the first input's tokens are the queries, the second's the keys and values (their widths may differ); the output has the first input's shape. `rmsnorm`: per token, LayerNorm without centring (LLaMA). `swiglu`: the gated MLP $W_2(\text{silu}(W_1x) \cdot W_3x)$ with HIDDEN units (`units=`, default 128), back to the token width. `token_pool`: `cls` / `mean` / `last` (`mode=`), default cls. Concat joins token sequences one after the other (the same width).

```
1 input
2 conv 64 4x4 stride 4 valid none
3 to_tokens
4 cls_pos nopos
5 rmsnorm
6 attention heads 4 rotary       # a LLaMA-style block: RMSNorm, rotary attention, RMSNorm, S
7 add <- 4, 6
8 rmsnorm
9 swiglu 256
10 add <- 7, 9
11 token_pool cls
12 output
```

**output, seg_output**

```
LABEL output [HEAD] | seg_output | detect
```

`output` gives one score per class of the dataset (classification); `seg_output` one score per class for every pixel (segmentation; it must be at the size of the masks); `detect` (`detect_output`) the boxes of an object-detection dataset, per cell of its input map (the image must be a whole number of cells). A network has one of them, at the end of its path, unless the Dataset tab lists *extra outputs* (multi-task): then each has its own `output` with its number, `HEAD` (also `head=`; default 0, the main task):

```
1 input
2 conv 32 3x3
3 global_pool
4 output                   # the main task: its classes
5 dense 32 <- 3
6 output 1                 # extra output 1 (e.g. biomass and height: two numbers)
7 output 2 <- 3            # extra output 2 (e.g. dry / wet)
```

**Time series: conv3d, temporal_pool, to_sequence, conv1d, lstm, gru**

```
LABEL conv3d FILTERS KxK [time T] [stride S] [same | valid] [ACTIVATION] [nobias]
LABEL temporal_pool [attention | mean | max]          (also tpool)
```

```
LABEL to_sequence                                          (also seq)
LABEL conv1d FILTERS [KERNEL] [stride S] [same | valid] [ACTIVATION] [nobias]
LABEL lstm UNITS [bi]  |  gru UNITS [bi]
```

For a time-series dataset, whose Input is (bands, dates, height, width). `conv3d`: filters (default 16), the pixel kernel (3x3), `time` = dates per window (`tkernel=`, default 3), stride over the pixels, padding same (odd kernels) or valid. `temporal_pool`: to an image (`mode=`, default attention). `to_sequence`: one vector per date. `conv1d`: filters and kernel (the first and second number; default 64 and 3), `stride`, padding. `lstm` / `gru`: units (default 64), `bi` for both directions (`bidirectional=true`). `token_pool last` keeps the last step.

```
1 input                     # (4, 6, 32, 32): 4 bands at 6 dates
2 conv3d 16 3x3 time 3
3 temporal_pool attention   # (16, 32, 32): one image, the dates weighted per pixel
4 conv 32 3x3
5 global_pool
6 seq <- 1                  # (6, 4): one vector per date
7 lstm 16 bi
8 token_pool last
9 concat <- 5, 8
10 output
```

### `subtract (sub)`, `shared` — comparing branches, shared weights

```
LABEL subtract [abs] <- A, B
LABEL shared OTHER
```

`subtract`: the first input minus the second (`abs`: $|a-b|$; `abs=true`); equal shapes. `shared OTHER`: the layer of block `OTHER` (its label) again, with the same weights, on this block's input (a Siamese network); the label may be a whole path or one of the same function call or pass. Its input must have the shape that block's input has.

```
1 input 1                   # the "before" images (band group 1)
2 conv 16 3x3
3 conv 16 3x3
4 input 2                   # the "after" images: the same bands (band group 2)
5 shared 2                  # block 2's convolution, with its weights
6 shared 3
7 subtract abs <- 3, 6      # what changed
8 global_pool
9 output
```

### `custom` — your own PyTorch class

```
LABEL custom CLASS
python """   ...  """          the design's PyTorch code: one section, anywhere (usually at the top)
```

A layer the block list does not have is written in PyTorch. The code is a section that begins with a line `python """` and ends with a line holding only `"""` (inside it, write docstrings on one line or with `'''`); every class at its top level is an `nn.Module` whose `__init__` receives the input's shape (`c, h, w` for an image, `t, d` for tokens, `n` for a vector), and a comment line `# out:` declares the output's shape in those names (whole-number arithmetic: `+ - * // %`); without it the output has the input's shape. `custom CLASS` uses a class:

```
python """
class Down(nn.Module):
    # out: c * 2, h // 2, w // 2
    def __init__(self, c, h, w):
        super().__init__()
        self.conv = nn.Conv2d(c, c * 2, 3, stride=2, padding=1)

    def forward(self, x):
        return torch.relu(self.conv(x))
"""
1 input
2 conv 16 3x3
3 custom Down              # (16, 32, 32) -> (32, 16, 16), from the # out: line
4 custom Down              # (32, 16, 16) -> (64, 8, 8): one class, used twice (two separate
5 global_pool
6 output
```

The Mac never runs the code: it checks the rest of the design with the declared shapes, and the training machine's one-batch check compares every Custom block's real output with its declaration before training (a wrong `# out:` line stops the run there). The parameters of these classes are counted on the training machine. On the canvas the Custom block's inspector edits the same code.

### 8.12 The rules in brief

```
net "name"                                    name of the design (optional)
let NAME = EXPR                               a named value
def NAME(P, Q = EXPR) { LINES }               a function (P, Q: parameters; Q has a default)
LABEL BLOCK SETTINGS [<- REF, REF]            one block; without <- it takes the line above
LABEL NAME(EXPR, Q=EXPR) [<- REF, REF]        a function call (inside: in, in2, ...)
LABEL repeat EXPR [as NAME] [<- REF] { LINES }       passes chained; i = pass number
LABEL for A, B in (EXPR, EXPR), ... [<- REF] { LINES }
python """ ... """                           the design's PyTorch classes (for custom CLASS)
REF:  a label | in | in2 | a path like 5.2.b    EXPR: numbers, names, + - * / // % ^ ( )
```

### 8.13 Mistakes and what the application says

A text with a problem is not applied: the canvas keeps the last valid design, the status says "not applied", the line number in the gutter turns red and the message below the text names the line (click it to jump there). Typical messages:

| Written | Message |
|---|---|
| `conv 64 3x3 stide 2` | 'stide' means nothing for conv (did you mean 'stride'?) |
| `convv 64 3x3` | unknown block 'convv' (did you mean 'conv'?) |
| `add <- 2, 7` (no line 7 above) | '7': there is no block '7' above this line (a block may only use blocks above it) |
| two lines labelled `2` | the label '2' is used twice |
| `dense 10 pad 1` | 'pad' means nothing for dense |
| `conv 8 3x3 filtrs=4` | conv has no setting 'filtrs' (its settings: activation, bias, dilation, filters, kernel, padding, stride) |
| `conv 8 3x3 99` | what is the number 99? write it as name=value (e.g. filters=99) |

| Written | Message |
|---|---|
| a `{` without `}` | a '{' is not closed with '}' |
| `f(1, 2)` for `def f(c)` | 'f' takes 1 value(s), more were given |
| `repeat k` without `let k` | repeat: 'k' is not a number |
| `repeat 5000 { ... }` too large | 5000 passes × 7 blocks = 35000 blocks, more than the 13000 a design may have (Settings → Design size ...). At most 1856 passes fit here. |

Shape problems (an Add of different shapes, a kernel larger than the image) are found by the same checks as on the canvas and listed under the text with the line of their block; click *block 5.2.a* to see it selected on the canvas. Cmd-click on any label in the text also selects its block.

### 8.14 How the text and the canvas stay in step

- **Typing:** a quarter of a second after the last key, a valid text replaces the design (one undo step). Blocks keep their place on the canvas when their label and kind stay the same; new blocks are placed next to their input.
- **Moving blocks** on the canvas keeps the text exactly as written.
- **Changing a setting** of a block that has its own line rewrites only that line (its label, inputs and comment stay).
- **Adding, removing or connecting** blocks of plain lines changes only those lines: a new block gets the next free number and goes after its input; a line that took "the line above" gets an explicit `<-` when that changed; a block inserted in front of a function call or loop becomes its input. Names, functions, loops and comments stay.
- **Changing a block that a function or loop made** would change every call, so the text is printed again from the blocks: identical consecutive groups become `repeat`, the rest plain lines. Cmd+Z brings the hand-written text back.
- **Format** prints the text again from the blocks on purpose; **Copy** copies it.

In the layer table, the blocks of one `repeat`, `for` or call line form one group named after the line.

### 8.15 A design that uses everything

`doc/examples/everything.nnb` in the repository is a hybrid CNN and Transformer written only as text: functions calling functions with defaults and named values, a function with two inputs, `for` with tuples, `repeat` with `i`, `let` and every operator, paths into loop passes, a left-out label, comments, and every block kind except the segmentation output and the time-series blocks (those are in `doc/examples/timeseries.nnb`), including a Custom block with its `python` section and a shared-weights twin, cross-attention, RMSNorm, SwiGLU and rotary attention, a mixture of experts, an early exit, a LoRA adapter and a frozen layer (97 blocks, 217 396 parameters of which 16 832 frozen, no warning at $3 \times 32 \times 32$). The test suite builds it and every example of this chapter, and GitHub Actions compares its layers with real PyTorch.

## 9 The Dataset tab

Only the *description* of a dataset is read on the design computer; the images are downloaded on the GPU machine when training starts (and cached there in `~/.nn_builder/data`).

**What data NN Builder reads.** Every kind, with a picture of such data and the Dataset tab set up for it, is in Section 10.

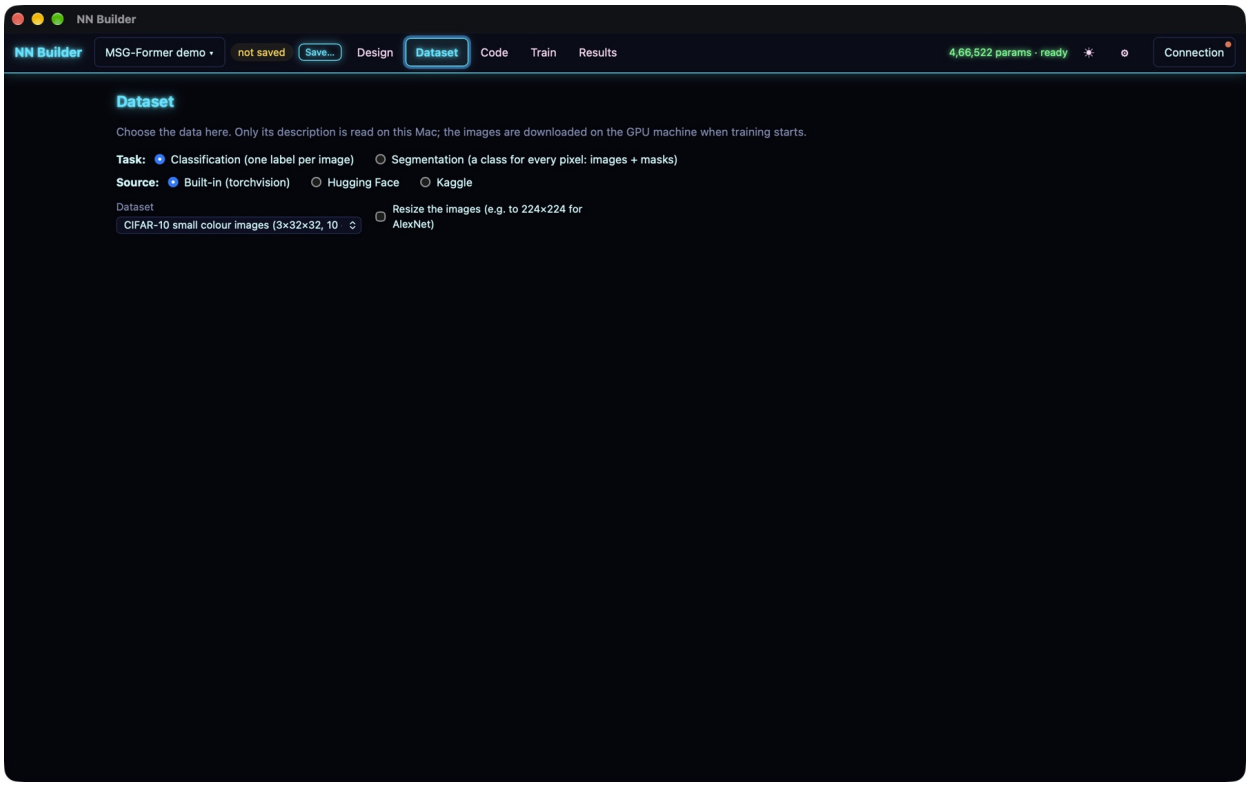



Figure 6: The Dataset tab with built-in CIFAR-10, as MSG-Former opens (the demo picks the built-in set that fits its input, so nothing is resized).

| Data | Sources | Task and files |
|---|---|---|
| MNIST, Fashion-MNIST, CIFAR-10 | built-in | classification |
| Pictures (PNG, JPEG, …; grey or colour) | Hugging Face, Kaggle, own files | classification: a label column, or one folder per class |
| Pictures + masks | Hugging Face, Kaggle, own files | segmentation: a mask column, or images and masks with the same names |
| Satellite / multispectral images (Sentinel-2, any bands) | Kaggle, own files | GeoTIFF or `.npy`, 1–32 bands; classification, multi-label, segmentation |
| Radar (Sentinel-1 VV, VH) | Kaggle, own files | as above; linear or dB |
| Large exported scenes + label image (Google Earth Engine) | own files, Kaggle | segmentation: cut into tiles |
| Several classes per image (BigEarthNet) | own files, Kaggle, Hugging Face | multi-label: a CSV (or a list column) |
| Two sensors for the same places (S1 + S2) | own files, Kaggle | fusion: two band groups, early or mid |

## 9.1 Built-in datasets

MNIST (handwritten digits, $1 \times 28 \times 28$) [21], Fashion-MNIST (clothing, $1 \times 28 \times 28$) [42] and CIFAR-10 (photos, $3 \times 32 \times 32$) [18], ten classes each, downloaded by torchvision on the GPU machine. **Resize the images** enlarges them for networks made for larger inputs (for example 224×224 for AlexNet); the number of channels can be changed as well (grey images repeated into three channels).

## 9.2 Hugging Face

Type a dataset id (`owner/name`) and press **Read info**: the application reads its configurations, splits, columns and first rows from the public datasets-server API, without downloading [22]. Then choose:

- **Config** and **Training split** (with its number of rows);
- **Test split**: a separate split, or "10% of the training split" (a fixed, seeded split);
- **Image column** and **Label column**: the column that holds the pictures (an *Image* feature) and the one that holds the class (a *ClassLabel* feature, whose class names fill the class list).

Gated or private datasets need a Hugging Face token (Connection tab), which the run can copy to the machine.

**Only image datasets work.** A text corpus such as `HuggingFaceFW/fineweb-edu` (web pages and their metadata) has no image column and no class column: the image and label columns then point at unrelated text, the class list stays empty, and the design is marked as having problems. Use datasets such as `uoft-cs/cifar10`, `ylecun/mnist` or `food101`. The Dataset tab now says so at once when a dataset has no image column.

After **Read info**, the tab also shows the **class balance** of the training split (counted by Hugging Face's dataset viewer) with a warning when the largest class has more than three times the images of the smallest, and **Show 8 example images** fetches just eight pictures to check that the right columns are chosen (the dataset itself is still downloaded only on the GPU machine).

## 9.3 Kaggle

Type the dataset slug (`owner/name`); with the Kaggle token stored, the application lists its files and recognises the common layout of *one folder per class*. Set the training folder (and optionally a test folder; otherwise 10% of the training images are kept for testing) and the class names (the folder names, in order). On Kaggle the dataset is attached to the notebook; on a server or EC2 it is downloaded with the Kaggle tool, which needs the token on that machine (the Train tab offers to copy it there).

## 9.4 Segmentation data

Hugging Face: an image column and a **Mask column**. Kaggle: an **Images folder** and a **Masks folder** whose files have the same names. The class list maps mask values to classes (value 0 is the first class; with two classes any non-zero pixel is the second).

## 9.5 Image size and classes

**Image height**, **Image width** and **Channels** set the shape of the Input block; images are resized to it. The class names, one per line in label order, set the Output block's size.

## 9.6 Satellite images: Sentinel-2, Sentinel-1 and other bands

Multispectral and radar images have more than three bands and values that are not 0–255 colours, so they are read as arrays, not as pictures. A Kaggle dataset with one folder per class of GeoTIFF (`.tif`, `.tiff`) or `.npy` files works, as in EuroSAT-MS. Choose it as usual (**Read files**, the training folder), then under **Image bands**:

- **Sentinel-2 (13 bands)**: the files' band order is filled in as EuroSAT-MS stores it (B01 … B12, then B8A; correct it if the dataset's description says otherwise). Tick the bands the

network should see, or use a preset: all 13, the ten 10 m and 20 m bands, true colour (B04 B03 B02), or colour and near infrared. Each band shows its wavelength and resolution.

- **Sentinel-1 radar (VV, VH)**: the two polarisations; tick **convert to decibels** when the files hold linear backscatter (values around 0.01–0.5), since a network learns radar much better in dB.
- **other bands**: type the bands of the files in their order and tick the ones to use.

The Input block then has as many channels as bands are ticked (up to 32), and the first convolution's parameters follow (13 bands into a $3 \times 3$ convolution with 16 filters: $13 \cdot 9 \cdot 16 + 16 = 1\,888$). On the GPU machine `train.py`:

- reads each file (bands first or last), picks the ticked bands by name, converts Sentinel-1 to dB if asked, and resizes to the image size of the Dataset tab;
- measures the mean and spread of every band on up to 256 training images and normalises each band with its own values (kept in `band_stats.json`, so a resumed run normalises the same way);
- augments with horizontal / vertical flips and quarter turns (a satellite image has no "up"), when flips or rotation are ticked in the Train tab;
- shows the result pictures as composites: true colour (B04 B03 B02) for Sentinel-2, VV / VH / VV−VH for Sentinel-1, else the first three bands, each stretched between its 2nd and 98th percentile.

**ImageNet weights with bands.** **Start from ImageNet weights** (Train tab) also works for satellite images: the first convolution's red, green and blue filters go to B04, B03 and B02 (matched by name), every other band gets the mean of the three, and all are scaled by $3/C$ for $C$ bands so the layer's output keeps its size; the rest of the network loads as usual. The images are still normalised per band (not with ImageNet's colour statistics).

**Hugging Face.** A Hugging Face dataset whose image column holds band arrays (an array feature, or GeoTIFF / `.npy` bytes) is read the same way: choose the bands under **Image bands** (type the band order, since it cannot be read on the Mac). Classification, multi-label (a list column) and segmentation (a mask column) work; two band groups need files (own files or Kaggle).

Several classes per image (Section 9.8) and Sentinel-1 with Sentinel-2 (Section 9.14) are below. GitHub Actions trains on generated Sentinel-2 (mixed `.tif` and `.npy`) and Sentinel-1 folders on every change and checks the bands, the statistics and the accuracy; it also trains on every layout of the next section.

## 9.7 Your own files, e.g. Sentinel-1/2 from Google Earth Engine

Sentinel data is usually exported from Google Earth Engine (GEE) to Google Drive and downloaded, so it is not on Kaggle or Hugging Face. Choose **Source: My own files**, then the folder (**Choose...**) and **Read folder**. The app lists the files, finds class folders, and reads the first GeoTIFF's header: its size, number of bands and, when GEE or GDAL wrote them, the band names, which fill **Bands in the files** by themselves (nothing is opened beyond the header, and nothing in the folder is changed).

**Where the files are. on this Mac**: when a run starts, the folder is copied to the training machine (`~/.nn_builder/data/local/nnb-<folder>-<code>`, with rsync, so only the first time and after a change), or, for Kaggle, uploaded under your account as a *private* Kaggle dataset of the same name and attached to the notebook (again only when it changed; the folders go up as zip files, which Kaggle unpacks). **already on the training machine**: give its path there (e.g. a large export copied to the college server yourself); nothing is copied, and Kaggle cannot be used.

**In the cloud.** The training machine (or the Kaggle notebook) fetches the files itself, so a large Earth Engine export never passes through the Mac: `gs://bucket/folder` (Earth Engine's

Cloud Storage export: in the export script choose **Export to: Cloud Storage**; a private bucket needs a service-account key, Connection tab, which is copied to an SSH or AWS machine with the other tokens; Kaggle notebooks read public buckets only), a Google Drive folder or file shared as "anyone with the link" (fetched with `gdown`: at most 50 files per folder, a zip is unpacked), or `rclone:remote:path` for rclone set up on the machine (any of its 70 storage services, Drive included). The files are kept in the machine's data folder; a later run fetches only what is new.

**Classification.** One subfolder per class (`forest/`, `water/` ...) of `.tif` or `.npy` patches, or ordinary pictures. An optional test folder; else 10 % of the images.

**Segmentation: images + masks.** An images folder and a masks folder with the same file names (a mask may add `_mask`); masks may be PNG, GeoTIFF or `.npy` with class numbers 0, 1, 2 ...

**Segmentation: a large image and a label image.** What GEE exports for a region: one image (possibly split into several files named `scene-0000000000-0000000000.tif` ...) and a label image on the same grid with the same file names (in another folder). The images are cut into tiles of the image size; a tile is used when at least half of its pixels have data and at least one has a class. **Test tiles**: 10 % at random (quick, but neighbouring tiles are alike, so the score is optimistic), the eastern tenth of every image with one column of tiles left out between it and the training tiles (*strip*: no test tile touches a training tile), or a test region exported separately (*files*: the most honest). The arrays are kept as `.npy` files in the data folder and read with memory mapping, so a scene larger than the memory works.

GEE's conventions are handled: band names B1 ... B9 are the same as B01 ... B09 everywhere in the app (**Earth Engine SR (12, B1 ... B12)** sets that band order; the surface-reflectance collection has no B10); empty (masked) pixels written as NaN are left out of the band statistics and filled with the band's mean; label values that are not a class (no data, 255, negative) are left out of the loss and of the scores; and Sentinel-1 from `COPERNICUS/S1_GRD` is already in dB: such files are recognised by their negative values and not converted again, even with **convert to decibels** ticked.

## 9.8 Several classes per image (multi-label)

In benchmarks such as BigEarthNet a patch can be forest *and* water *and* farmland at once. Choose **Task: Multi-label**. The labels come from a CSV file (own files or Kaggle): one row per image, a file column (`file`, `image`, `name`, `path`, `patch_id` ...; with or without the extension, relative to the CSV's folder or the images folder) and either a labels column with the classes separated by `;` `,` `|` or spaces, or one 0/1 column per class. **Read labels** reads the class names and how many images have each. From Hugging Face, choose a column that is a list of class labels. Rows that name a missing file are left out (with a warning). A `split` column (`train` / `validation` / `test`) is used as given instead of a random split.

**BigEarthNet v2 as it is downloaded.** Each patch is a folder of single-band GeoTIFFs (`<patch>/<patch>_B02.tif` ..., the 20 m and 60 m bands at their own sizes) below a tile folder, and the labels are in `metadata.parquet` (a list of class names per patch, the split, and `s1_name`, the matching Sentinel-1 patch). Choose **BigEarthNet (19 classes, metadata.parquet)**, set the **Images folder** to the Sentinel-2 folder, and for fusion add Sentinel-1 as the second band group with its folder: the patch folders are found below them, every band is brought to the largest band's size, and `s1_name` pairs the two sensors. Parquet tables are read on the training machine only (the class names come from the preset or are typed).

The network ends in the ordinary Output block, one score per class, but every class is its own yes/no: the score goes through a sigmoid, a class is predicted when it is at least 50 %, and the loss is binary cross-entropy summed over the classes (label smoothing pulls the targets towards 0.5). The run reports micro F1 (every image–class decision together; shown where accuracy usually is),

macro F1 (the classes averaged, so rare classes count as much as common ones), mean average precision (mAP: how well each class's scores rank its images, independent of the threshold) and exact match (every class of an image right). Results lists precision, recall, F1 and AP per class, and the sample pictures mark each class as right, wrong or missed.

### 9.9 Numbers per image (regression)

To predict a quantity instead of a class (above-ground biomass, canopy height, crop yield, soil moisture of a plot), choose **Task: Regression**. The numbers come from a CSV file next to the images (own files or Kaggle): one row per image, the same file column as for multi-label, and one numeric column per target. **Read labels** lists the columns that hold a number in every row, with their range; tick the targets (one or several). From Hugging Face, tick the numeric columns of the table. Rows whose target is empty or not a number are left out with a warning; a `split` column is used as given.

The Output block gives one number per target (no softmax). Before training the run measures every target's mean and spread on the training images, and the network learns the targets standardised (mean 0, spread 1), so targets of very different sizes train equally well. **Loss** in the Train tab: *mean squared error* (MSE, the default; large errors count most), *L1* (mean absolute error; less swayed by a few wild values) or *Huber* (squared for errors below one standard deviation of the target, linear above). The scores are in the targets' own units: $R^2$ (the share of the targets' variation the network explains: 1 is perfect, 0 is no better than always predicting the mean; shown where accuracy usually is, and what early stopping on "validation $R^2$" watches), MAE (mean absolute error) and RMSE (root mean squared error), per target and averaged. Results shows a predicted-against-true scatter plot per target and sample images with both numbers. `model.pt` outputs the standardised numbers; `target_stats.json` next to it holds each target's mean and standard deviation (value = output × std + mean). Flips and rotations are fine for quantities that do not depend on direction. A number for every pixel (a canopy height map) is not supported yet.

### 9.10 Several outputs (multi-task)

One network can predict several things about the same image at once: the land-cover class *and* the biomass, or the crop type *and* whether the field is irrigated. The shared layers then learn features useful for all of them, which often helps the smaller tasks. In the Dataset tab, **Extra outputs (multi-task)** → **Add an extra output**, for each one:

- a **name** (shown in the results), what it **Predicts** (one class, several classes, or numbers), and its column(s) of the labels table: the class column; the labels column (several classes; empty when they are 0/1 columns); or the number columns;
- for classes, the class names in label order; its **Loss** (cross-entropy or focal; binary cross-entropy; MSE, L1 or Huber); and its **Weight**: the total loss is the main task's plus each extra output's times its weight.

The values come from the labels table next to the images (own files or Kaggle: the same CSV / TSV / Parquet as for multi-label and regression; with the main task *classification* the image's class is its folder, or the **Main task's class column**), or from columns of a Hugging Face dataset. Rows without a usable value for an output are left out, with a warning.

In the Design tab each output gets its own Output block: select an Output and choose **Output for** (the main task, 1, 2, …; in text `output 1`). Usually they share the body of the network and each has a small branch of its own (a Dense or two) before its Output. The network returns all outputs; the run reports the main task as usual (and its score decides early stopping and the best epoch) and every extra output in a table: accuracy, micro F1 and mAP, or $R^2$, MAE and RMSE per number, in the numbers' own units. The Train tab's chart draws each output's validation score.

## 9.11 Object detection (boxes)

To find and count objects (ships, cars, buildings, trees, animals) with a box around each, choose **Task: Object detection**. The data are an **Images folder** and a **Labels folder** with one text file per image of the same name (`im17.png` and `im17.txt`): one line per box, `class cx cy w h`, the class number (0 = the first line of the class list) and the box's centre, width and height as fractions of the image. This is the YOLO format that Roboflow, CVAT and most converted datasets offer; an image without a label file has no object. Pictures and multiband files both work; 10 % of the images are the test data; horizontal and vertical flips move the boxes with the image.

The network ends in a **Detection output** on a feature map, typically a quarter or an eighth of the image size (two or three $2 \times 2$ pools or stride-2 convolutions; the image must be a whole number of cells). For every cell it gives a score per class (is an object's centre here?), the box's width and height in cells and the centre's offset inside the cell: the anchor-free CenterNet design [49]. Training draws every box as a Gaussian peak on its class's map and uses CenterNet's loss (a focal loss on the peaks, L1 on size and offset). The boxes found are the peaks of the scores; the run reports mAP@0.5 (a box counts when it overlaps a true one by at least half, IoU $\geq 0.5$) and mAP@0.5:0.95 (stricter overlaps, as COCO), with the AP of every class, and Results draws the boxes found (score $\geq 30\,\%$) and the true ones on test images. Not yet: rotated boxes (DOTA's oriented format), COCO JSON labels, very large scenes cut into tiles with their boxes, and non-maximum suppression across classes.

## 9.12 Time series of satellite images

A field's crop shows in *when* it greens up and is harvested, more than in any single image. A time series gives the network the same place at several dates. In the Dataset tab, after choosing the bands, tick **a time series** and give the **dates per sample**. Each file then holds one place at all its dates: a `.npy` of shape (dates, bands, height, width) (or (dates, height, width, bands)), or a GeoTIFF whose bands go date by date: all bands of the first date, then of the second, ... (as Earth Engine exports a stack of monthly composites, `ee.ImageCollection.toBands()`). Class folders, a labels table (multi-label, regression, extra outputs) and images + masks (a crop-type map) all work; every band is normalised over all its dates, every date is resized like an image, and flips and turns apply to all dates alike. The result pictures show the median of the dates.

The Input is then $(C, T, H, W)$: bands, dates, height, width. Image blocks do not take it directly; the Time series blocks do:

- **Conv3D** convolves over neighbouring dates and pixels at once (*dates per window*, e.g. 3);
- **Temporal pool** combines the dates of every pixel into one image $(C, H, W)$: the mean, the maximum, or *attention*: a score for every date and pixel from the pixel's channels, a softmax over the dates, and the weighted sum (the idea of U-TAE [46]). After it every 2-D block works: convolutions, a U-Net decoder, a Segmentation output for a crop map;
- **To sequence** takes the mean of every date's pixels, one vector per date $(T, C)$: the time series of the whole patch. Then **Conv1D**, **LSTM** and **GRU** work along the dates, and so do the transformer blocks (Class token + position, Self-attention, Token pool), which gives a small temporal transformer (as in TSViT [47]). Token pool's *last* keeps the last step of a recurrent layer.

Both kinds of path can start from the same Input and be joined (`doc/examples/timeseries.nnb`). BatchNorm, Dropout, the activations, Add / Subtract / Multiply, Concat (equal dates and size), Flatten and Global pool work on a time series too. Not yet: a date per image (the dates are taken as evenly spaced), missing dates, a second band group with a time series, Hugging Face time series, the Earth Engine script for a time stack.

### 9.13 Shared weights: Siamese networks and change detection

A Siamese (twin) network looks at two images with the *same* layers, so that both are described in the same terms, and then compares the two descriptions: has this field been built on, flooded or cleared between two dates? NN Builder does it with:

- **the data as a pair in time:** the second band group with the *same bands* as the first, from the folder of the later images (same subfolders and file names): e.g. `before/changed/17.tif` and `after/changed/17.tif`. The second copy of each band is reported as `B04_2` etc.; flips and turns are applied to both images alike;
- **the Twin branch button** (Design tab): build one branch from the Input (convolutions, pools, ...), select its last block and press **Twin branch (shared weights)**. The same branch appears again from an Input for band group 2, made of **Shared weights** blocks (each runs the original layer with the original weights) and copies of the layers without weights. In text: `5 shared 2` uses block 2's weights (a label inside a function or repeat may name a block of the same call or pass);
- **a join that compares: Subtract** with **absolute difference** gives $|a-b|$, what changed (FC-Siam-diff); Concat keeps both (FC-Siam-conc). Then the head: Global pool + Output for "changed / not changed", or a decoder to a Segmentation output for a change map (images + masks).

The shared layers are trained by both images and counted once in the parameters. A Shared-weights block's input must have the shape of the original block's input, or the weights would not fit; the inspector lists the blocks whose weights can be used.

### 9.14 Sentinel-1 and Sentinel-2 together (fusion)

Radar sees through clouds and shows water and structure; optical bands show vegetation and colour. Under **Second band group (fusion)** choose the second sensor, its folder and its bands. Its files mirror the first group's: the same subfolders and file names, any extension (`s2/forest/x.tif` and `s1/forest/x.npy`); a different pixel size is resized to the image size. The two groups are stacked into one image (the first group's bands, then the second's), each band normalised with its own statistics, and the second Sentinel-1 group converted to dB when asked. Then, in the Design tab:

**Early fusion.** One Input takes all bands: nothing else to do.

**Mid fusion.** Select the Input, set **Bands** to band group 1, and press **Add an Input for band group 2**: each Input starts a branch of its own (e.g. a few convolutions per sensor), and a Concat (or Add) joins them. In text: `input 1` and `input 2`. The generated code takes each group's channels from the stacked image.

Fusion works for classification, multi-label (BigEarthNet: S1 and S2 per patch) and segmentation (images + masks, and scenes: the second group's scene files in its folder with the same names). With two groups the test data are 10 % of the images (no separate test folder).

**Each group at its own size.** Resizing the second group to the first group's grid invents pixels (Sentinel-1 at 20 m blown up to 10 m) or throws them away. Tick **its own image size** under the second band group and give its size, e.g. $32 \times 32$ next to Sentinel-2's $64 \times 64$: each group is then resized only to its own size and reaches the network as an input of its own. That needs mid fusion: one Input per group (**Bands** 1 and 2), each with its own branch, joined where their sizes match (here after a $2 \times 2$ pool or a stride 2 on the Sentinel-2 branch); an Input for all bands is refused, as the groups cannot be stacked. The second group may not be larger than the first (make the sharper sensor group 1), and scenes keep one grid. Flips and quarter turns are applied to both groups alike, and the band statistics are measured on each group's own pixels. (Inside, the second group fills the top-left corner of its channels in the batch tensor and the generated code cuts it

out: the Code tab shows `x[:, 4:, :32, :32]`.)

**The export script.** Under **Get Sentinel-1/2 data from Google Earth Engine** the Dataset tab writes a script that exports exactly what the tab expects: the ticked bands in their order, from the chosen dates, with clouds masked (Sentinel-2 SR: the scene classification band; with B10: the top-of-atmosphere collection and its QA60 band) or the median of Sentinel-1 IW scenes with VV and VH. For segmentation it is a Code Editor script: draw the region, **Run**, then run the two tasks; it exports the image and the ESA WorldCover 2021 map (11 classes, as class numbers 0–10; replace it by your own labelled image) on one grid into the Drive folders `nnb_image` and `nnb_label`. For classification it is a Python script (`earthengine-api`) that exports one patch per labelled point into a Drive folder per class, starting every export itself. **Set this tab to match the export** fills in the band order, the layout and (WorldCover) the class names. The app does not run the script: it needs your Earth Engine account, so check it there.

# 10 Kinds of data, step by step

For each kind of data: what such data looks like, the Dataset tab set up for it, and the steps. The data pictures are *generated* examples (a random land-cover map turned into typical Sentinel-2 reflectances and Sentinel-1 backscatter; `doc/screens/data_pictures.py`), not real images; the Dataset tab pictures are the application's own screen (`doc/screens/render.sh`). The folder paths in them are examples. After any of these, the Design tab's Input block shows the resulting shape, and the Output block must have as many classes as the class list.

## 10.1 Built-in sets (MNIST, Fashion-MNIST, CIFAR-10)

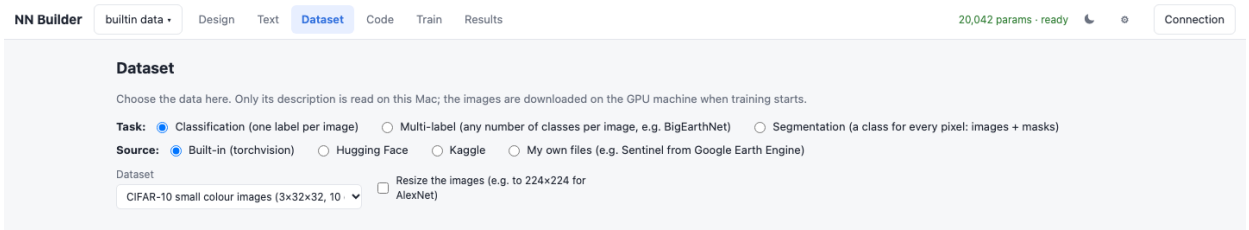



Figure 7: Dataset tab: built-in CIFAR-10.

1. **Task: Classification**, **Source: Built-in (torchvision)**.
2. Choose the set. The Input becomes its own size ($1 \times 28 \times 28$ or $3 \times 32 \times 32$); tick **Resize the images** for a network made for larger pictures and set the size and channels.

Nothing else: torchvision downloads the set on the GPU machine.

## 10.2 Pictures from Hugging Face

1. **Source: Hugging Face**; type the id (`owner/name`) and press **Read info**.
2. Check the splits, the **Image column** and the **Label column** (its class names fill the list); **Show 8 example images** checks the choice.
3. Set the image size and channels the network should receive.

## 10.3 Pictures in class folders (Kaggle or your own files)

1. **Source: Kaggle** (type `owner/name`, **Read files**) or **My own files** (**Choose...**, **Read folder**).

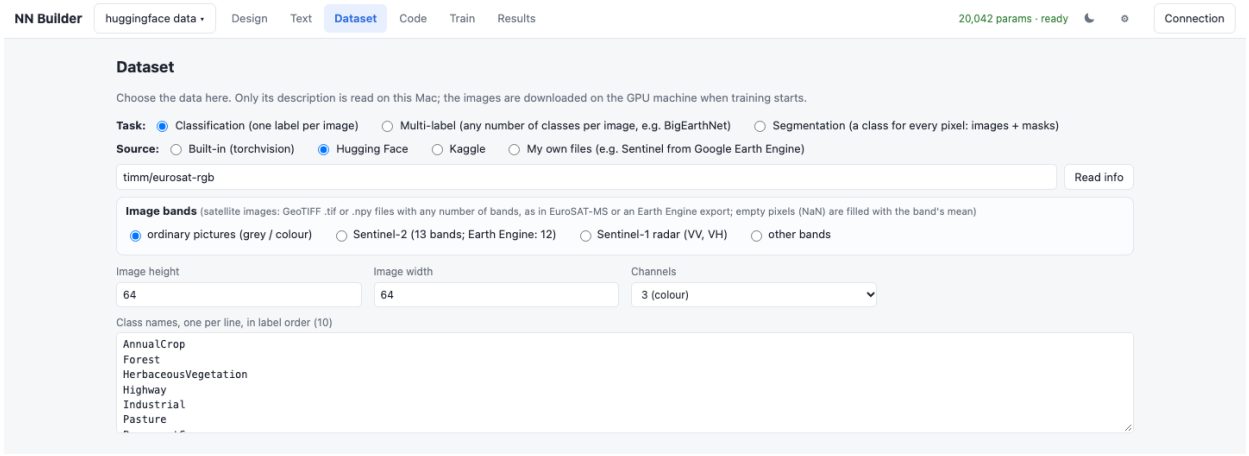



Figure 8: Dataset tab: a Hugging Face dataset (`timm/eurosat-rgb`, RGB satellite pictures in ten classes) before **Read info**.

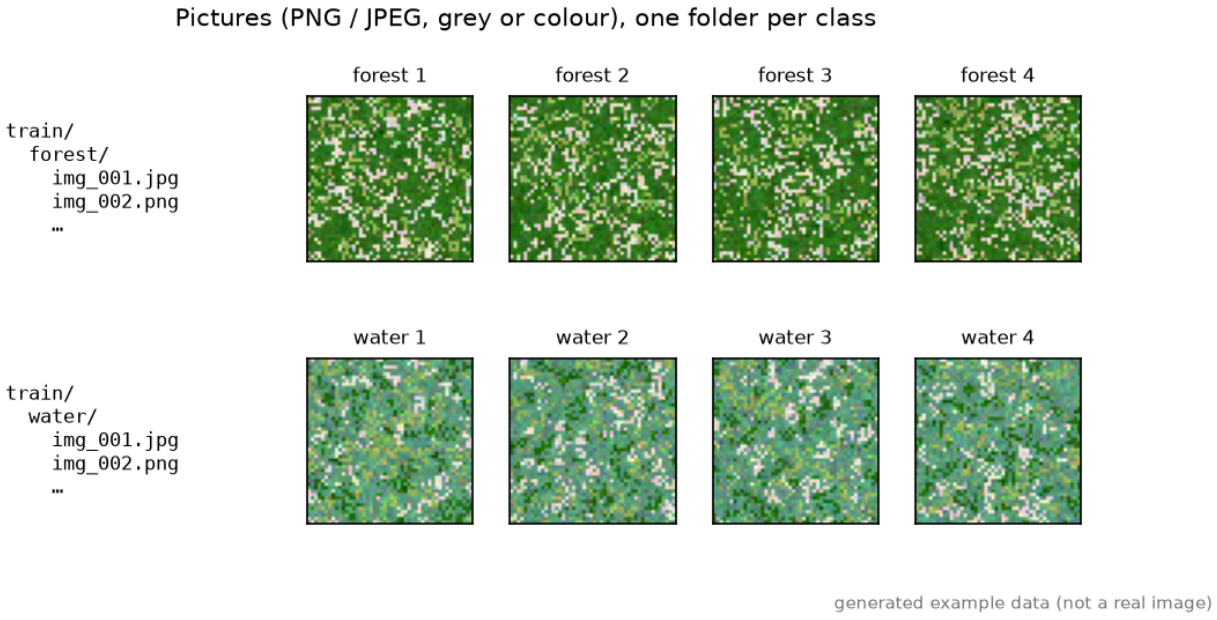



Figure 9: Pictures sorted into one folder per class (generated examples).

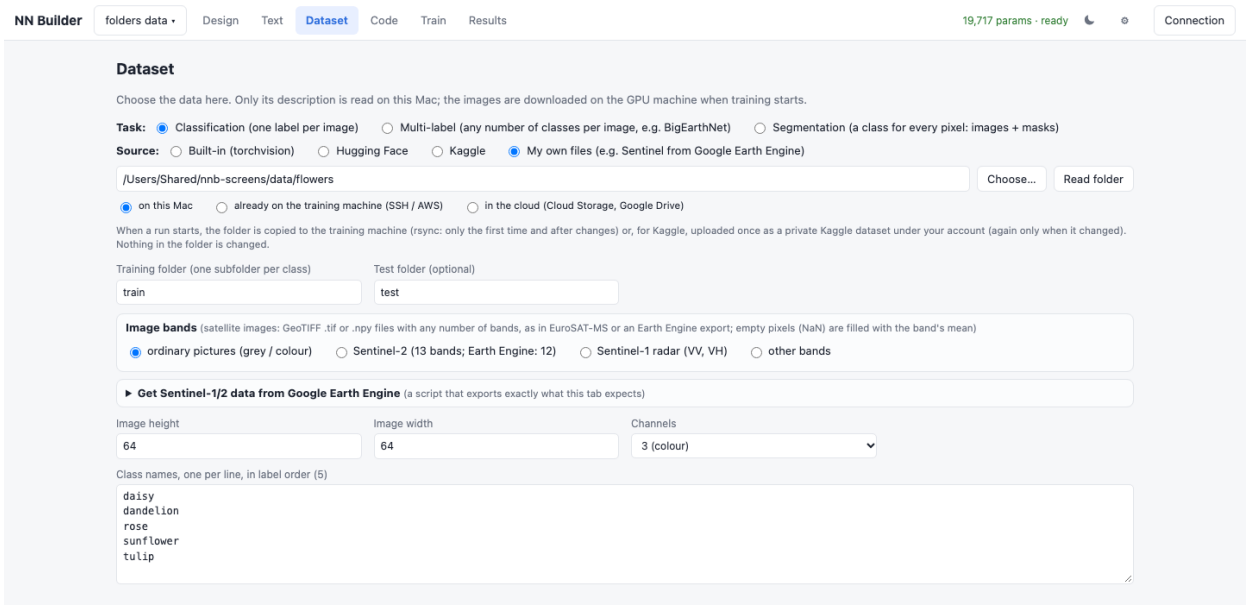



Figure 10: Dataset tab: your own folder with `train/` and `test/`, each with one subfolder per class.

2. **Training folder**: the folder whose subfolders are the classes (found by itself when there is one); optionally a **Test folder** with the same subfolders.
3. **Image bands**: **ordinary pictures**. Set the size and channels (1 grey, 3 colour); the class names are the folder names.

## 10.4 Segmentation: images and masks

1. **Task: Segmentation**. Hugging Face: choose the **Mask column**. Kaggle or own files: **Images folder** and **Masks folder** (a mask may add `_mask`; PNG, GeoTIFF or `.npy`).
2. Pictures: **ordinary pictures**; satellite images: choose the bands as below.
3. The class list: mask value 0 is the first line, 1 the second, ...; other values (no data, 255) are not scored.
4. The design must end in **Segmentation output**, at the input's size (e.g. a U-Net).

## 10.5 Satellite images: Sentinel-2, Sentinel-1, any bands

1. Kaggle or **My own files**; the files are GeoTIFF (`.tif`) or `.npy`, one per patch, in class folders (or images + masks, or a CSV for multi-label).
2. **Read folder** reads the band names from the first GeoTIFF when Earth Engine or GDAL wrote them; otherwise choose **Sentinel-2**, **Sentinel-1** or **other bands** and set **Bands in the files** (**EuroSAT-MS order** or **Earth Engine SR** fill it for Sentinel-2).
3. Tick the bands the network should see, or a preset (10 m + 20 m, true colour, colour + near infrared). Sentinel-1: tick **convert to decibels** only for linear values.
4. Set the image size; the channels are the ticked bands.

Details: Sections 9.6 and 9.7.

## 10.6 A large exported scene with a label image (Google Earth Engine)

1. **Task: Segmentation**, **Source: My own files**, the layout **large images + a label image**.

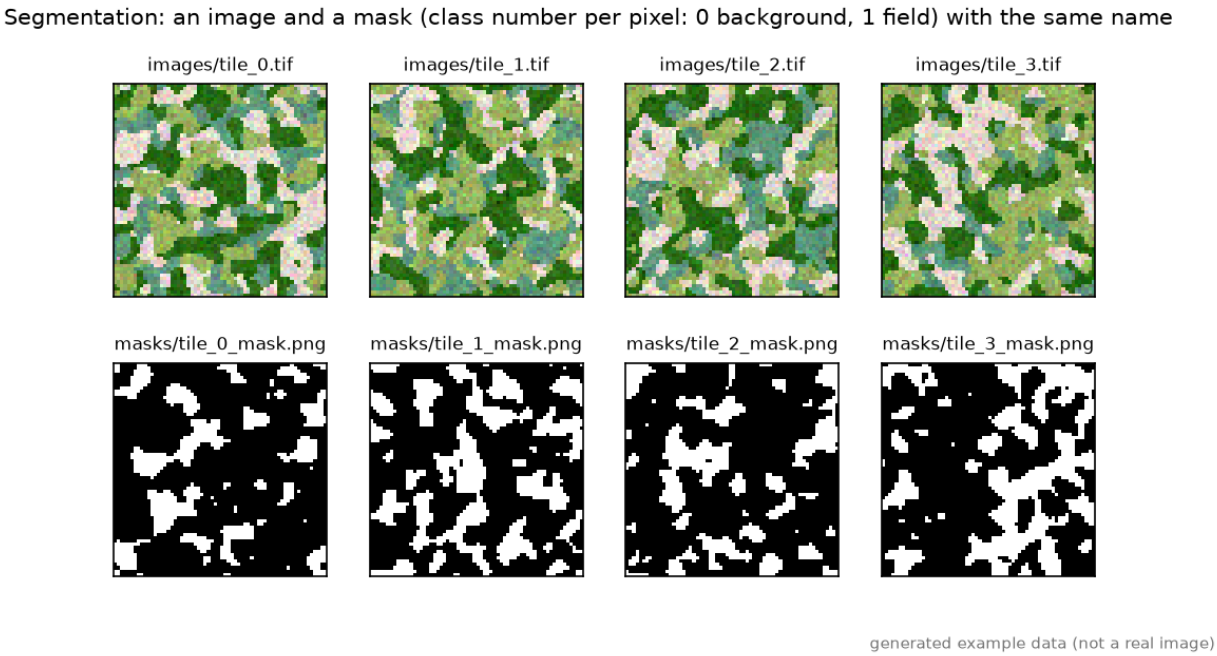



Figure 11: Images and masks with the same names; a mask holds the class number of every pixel (generated examples).

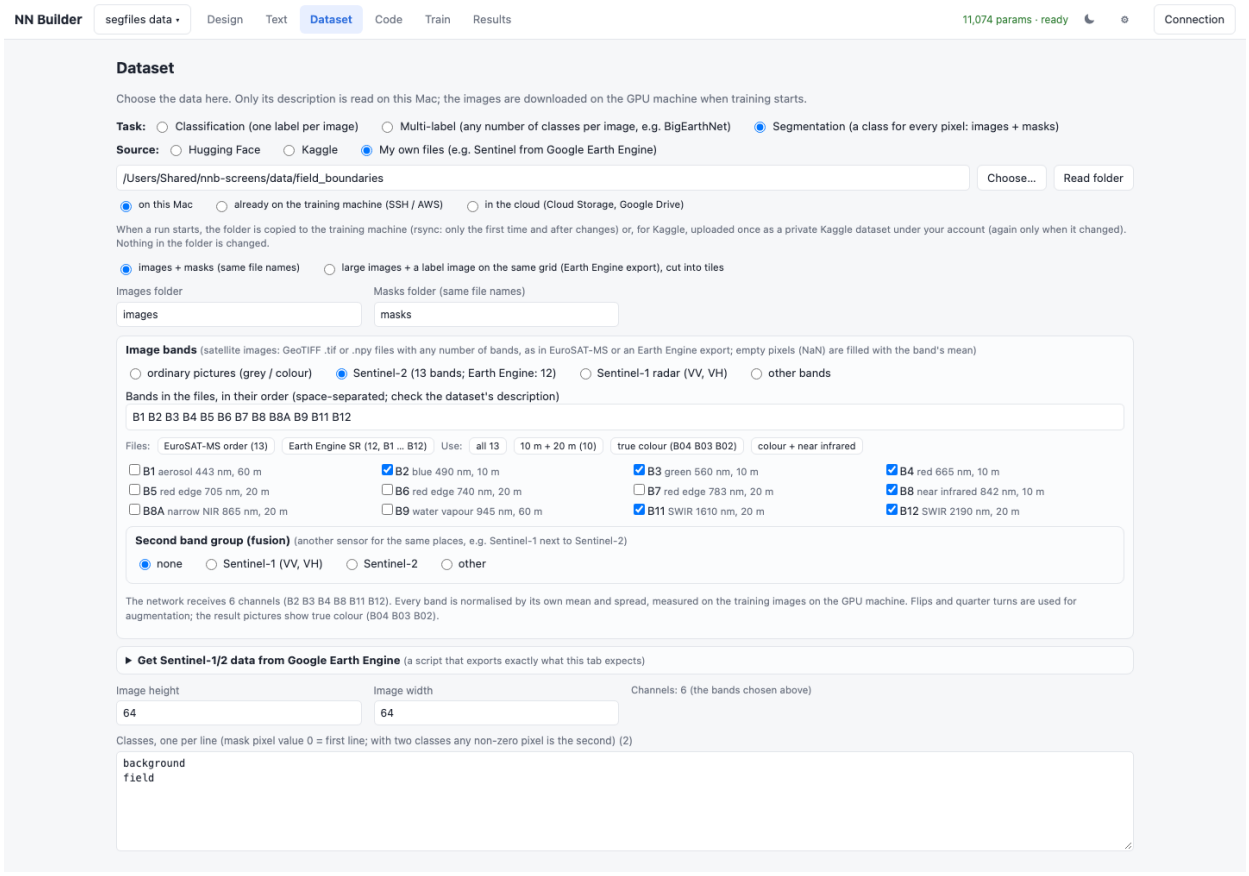



Figure 12: Dataset tab: multispectral images and masks in your own folder.

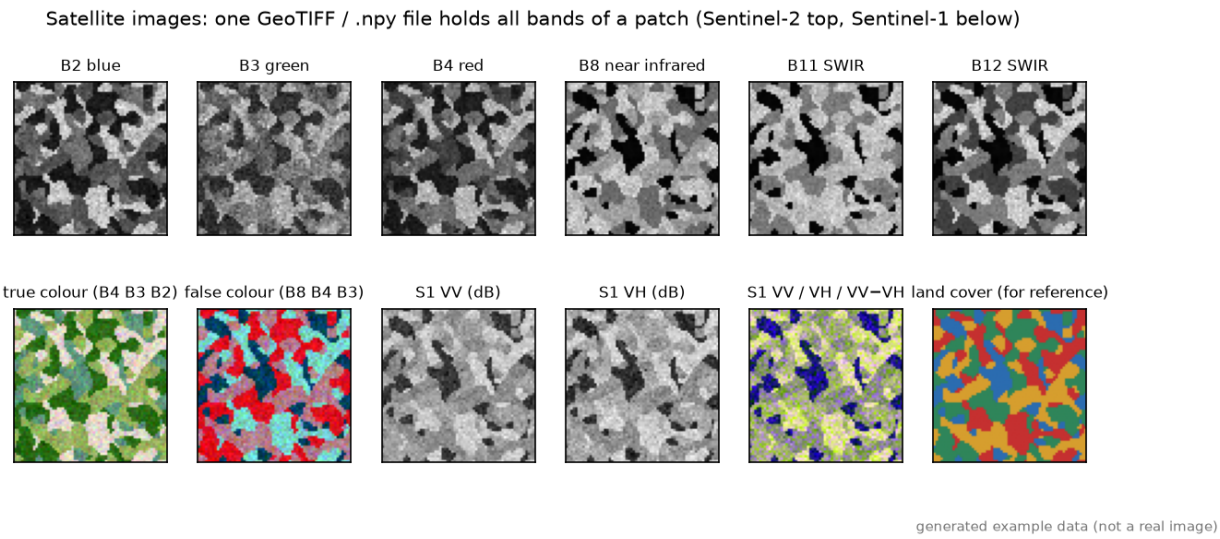



Figure 13: One patch, band by band: six Sentinel-2 bands, the true- and false-colour composites, and Sentinel-1 VV and VH in dB (generated example).

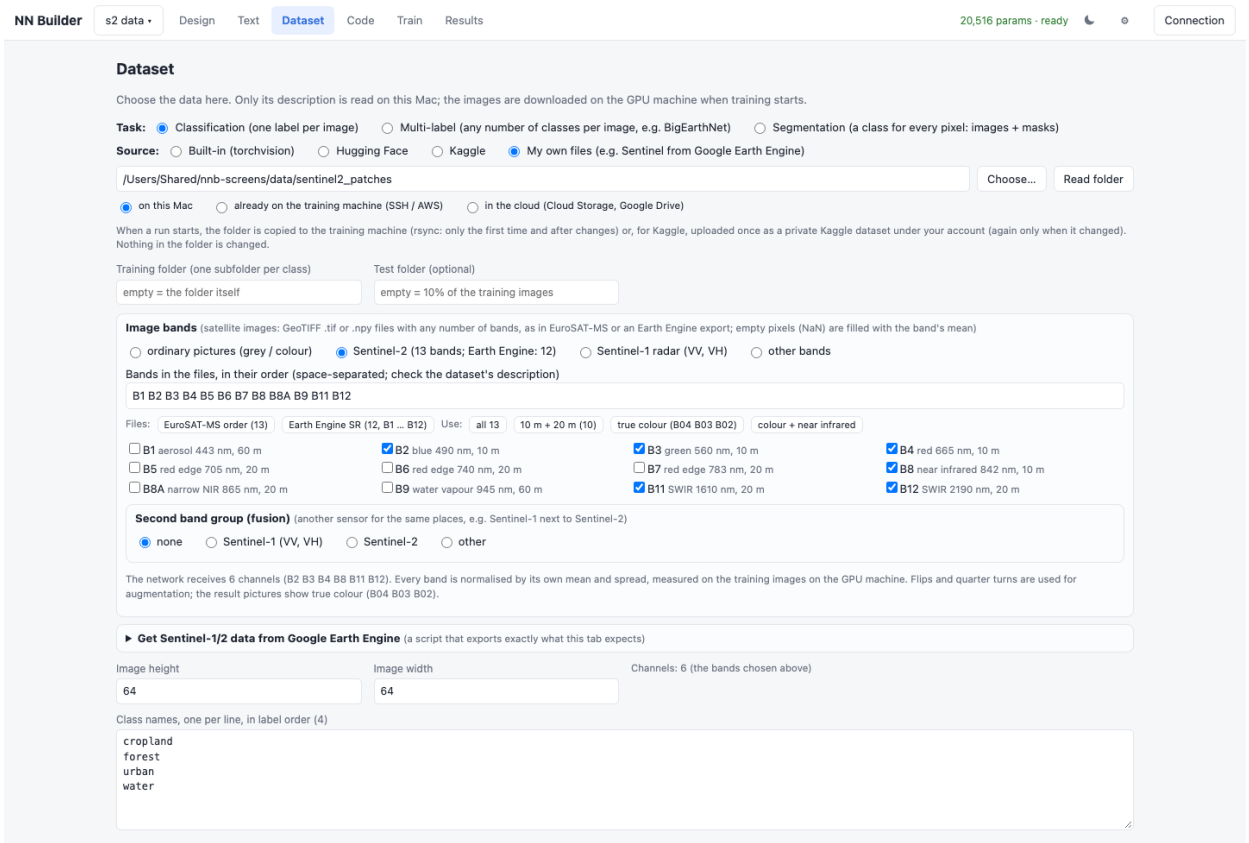



Figure 14: Dataset tab: Sentinel-2 patches exported from Earth Engine (band order B1 … B12), six bands chosen.

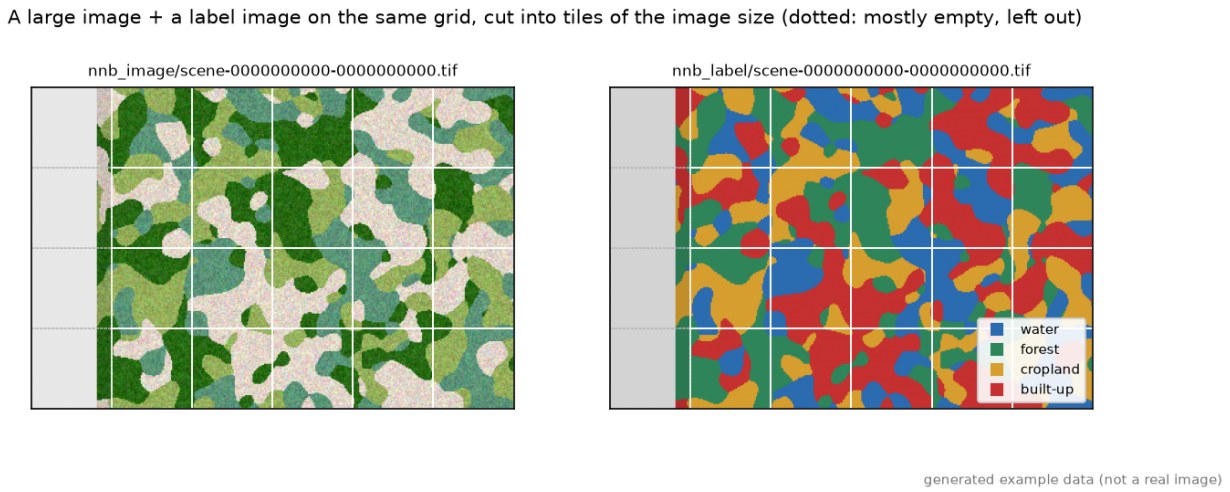



Figure 15: An export and its label image on the same grid, cut into tiles; mostly empty tiles (dotted) are left out (generated example).

2. **Image file or folder** and **Label file or folder** (the same file names when the export is split into several files).
3. The bands, and the image size = the tile size (e.g. 128 × 128). The class list: label value 0 first.
4. No data yet? Open **Get Sentinel-1/2 data from Google Earth Engine**, set the dates, copy the script into the Earth Engine Code Editor, draw the region, run both exports, download the two Drive folders into one folder, and press **Set this tab to match the export**.

### 10.7 Several classes per image (multi-label)

1. **Task: Multi-label**. Own files or Kaggle: the **Labels file (CSV)** and, if the images are not next to it, the **Images folder**; **Read labels** fills the class list. Hugging Face: a label column that is a list of classes.
2. Pictures or bands as above; the Output block gives one score per class.
3. The results are F1, mAP and exact match (Section 9.8).

### 10.8 Two sensors: Sentinel-2 and Sentinel-1 together (fusion)

1. Set up the first group (e.g. Sentinel-2) as above, with the **Training folder** pointing at its files (`s2`).
2. Under **Second band group (fusion)**: the sensor, the **Folder with these files** (`s1`, mirroring `s2`: the same subfolders and names) and its bands.
3. Design tab: keep one Input (early fusion), or select it, set **Bands** to group 1 and press **Add an Input for band group 2** (mid fusion): give each Input its own layers and join them with Concat. In text: `input 1` and `input 2`.

Section 9.14 has the details.

## 11 The Code tab: what the generated script does

The **Code** tab shows the complete `train.py`; **Copy** and **Export train.py** save it. It needs only PyTorch and torchvision (plus `datasets` or `kaggle` for those sources) and runs anywhere with `python train.py`. In order, it:

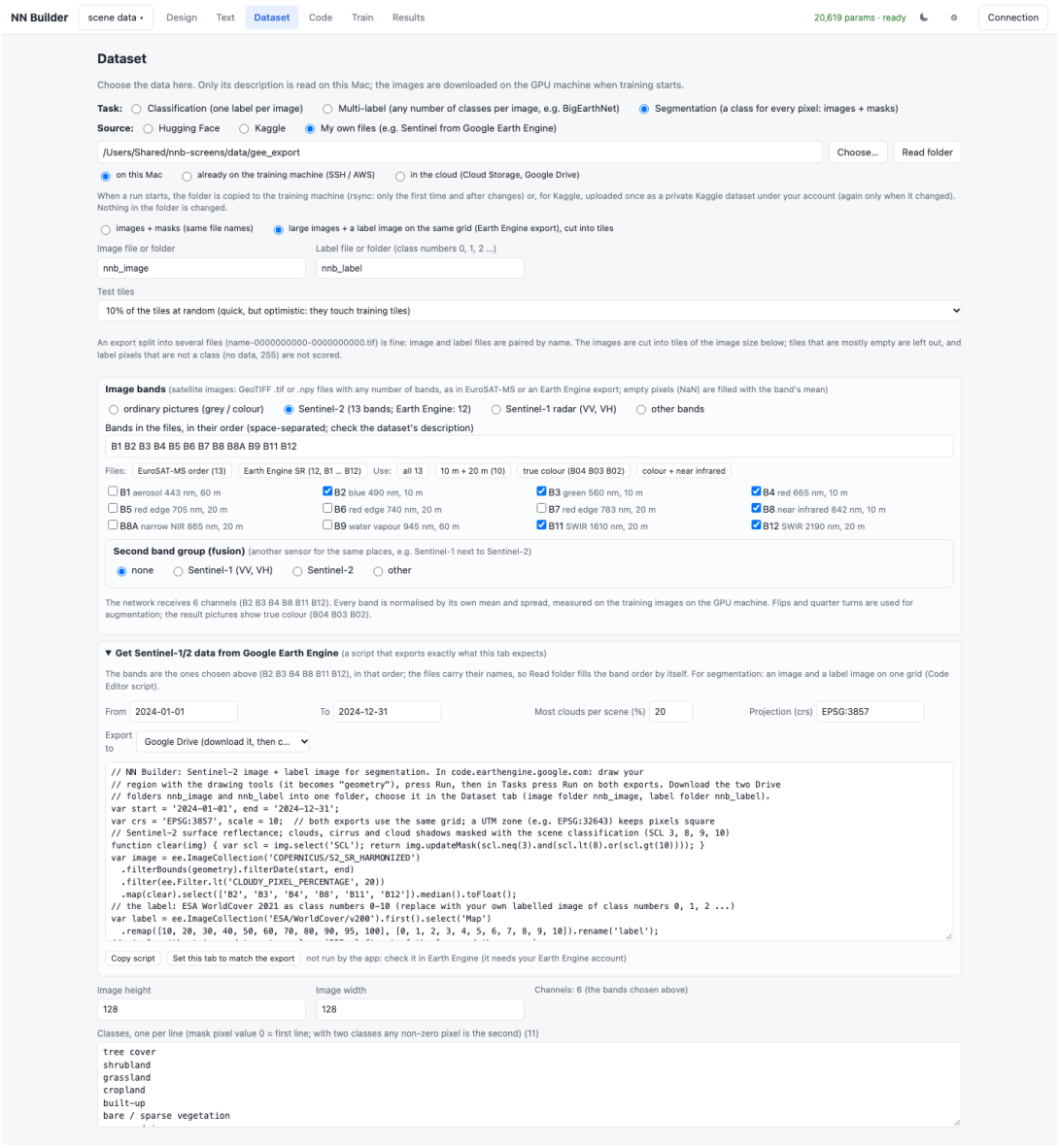



Figure 16: Dataset tab: a scene layout with the Earth Engine export script opened (it exports exactly these bands, and the WorldCover map as labels).

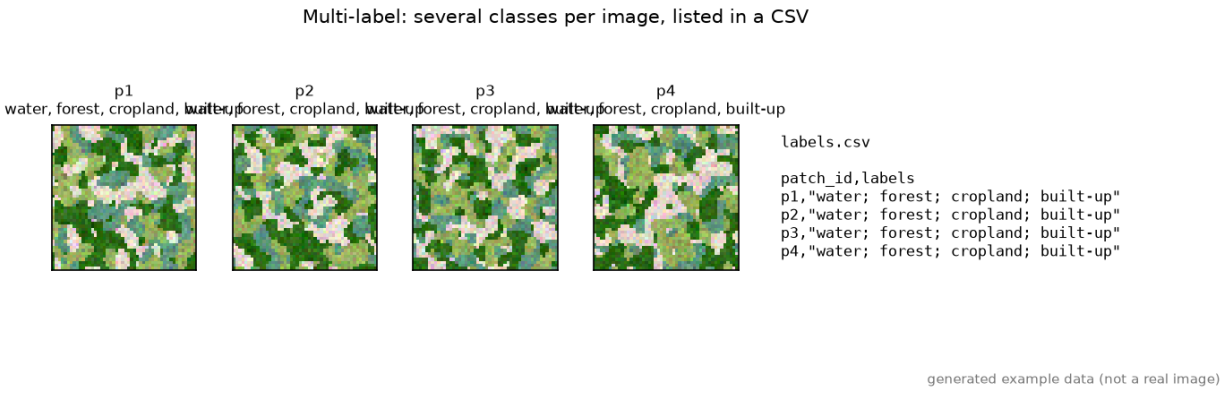



Figure 17: Patches with several classes each, and the CSV that lists them (generated examples).

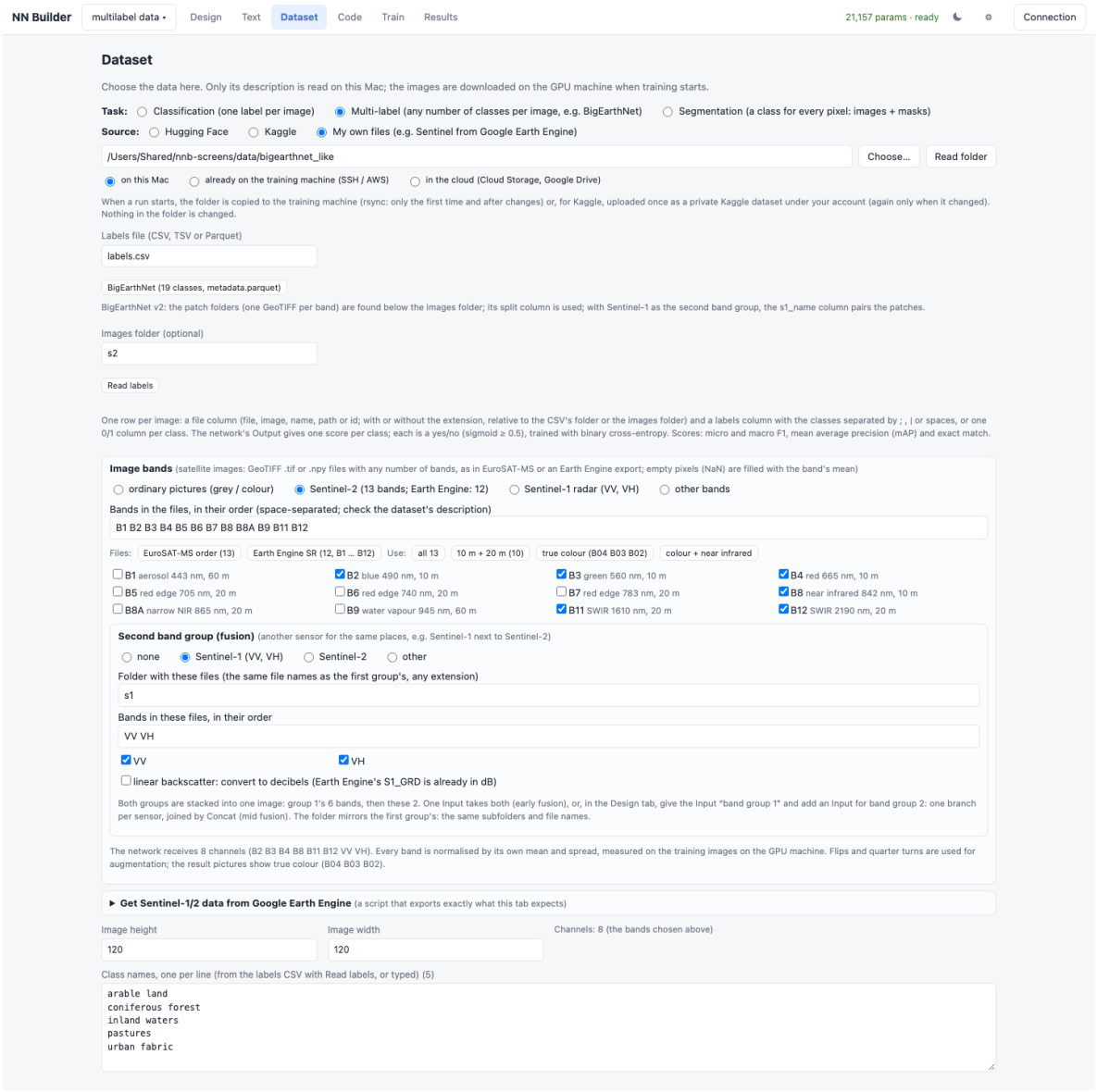



Figure 18: Dataset tab: a BigEarthNet-like folder: Sentinel-2 patches in `s2/`, Sentinel-1 in `s1/`, labels in `labels.csv`.

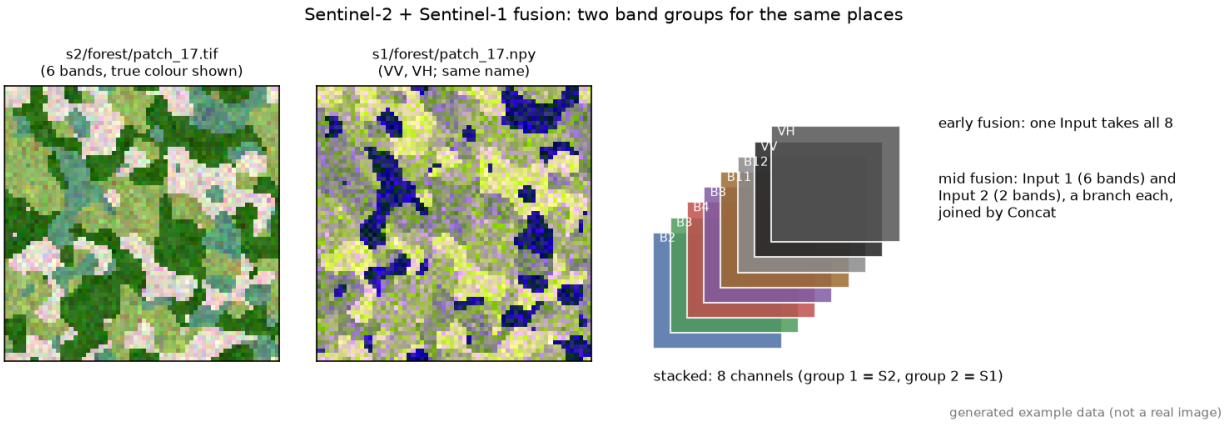



Figure 19: The same place from both sensors, in two folders with the same file names, stacked into one input of 8 channels (generated example).

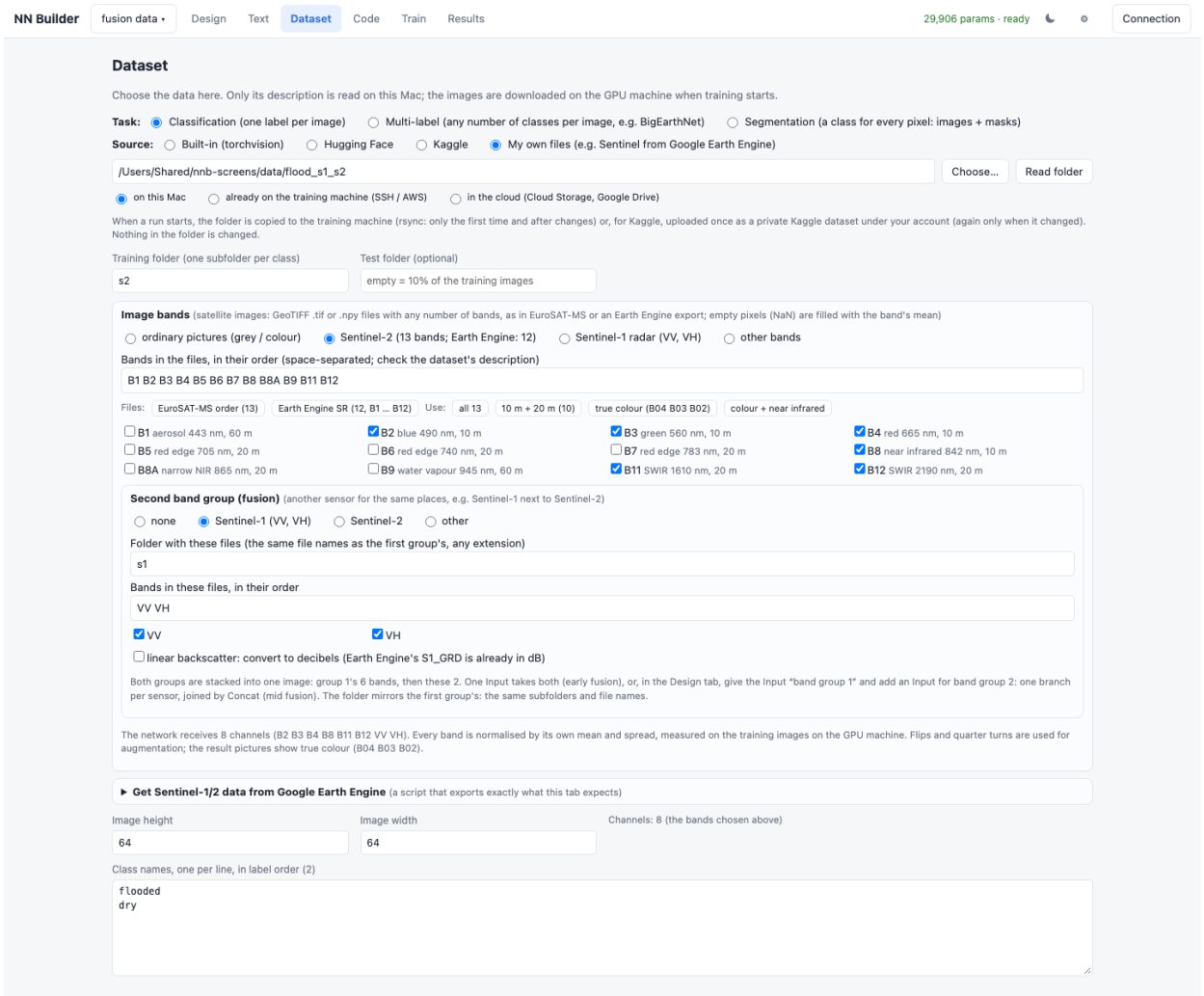



Figure 20: Dataset tab: Sentinel-2 (six bands, `s2/`) and, as the second band group, Sentinel-1 (VV, VH, `s1/`).

1. defines the network as an `nn.Module` whose `forward` follows the graph, and lists every layer's expected shape;
2. loads the data with the chosen transforms and augmentation (training images only), normalised with the dataset's statistics;
3. runs a **one-batch check** on the GPU machine: one batch forward and backward, comparing every layer's real shape with the application's, and stopping at once with a clear message on a difference;
4. trains with Adam [17], AdamW or SGD with momentum 0.9 and the chosen recipe (learning-rate schedule with warm-up, weight decay, label smoothing, gradient clipping), with automatic mixed precision on CUDA [27];
5. after each epoch evaluates on the test data and writes a **checkpoint** (`checkpoint.pt`; optionally also every few minutes inside an epoch); a restarted run **resumes** from it, in the middle of an epoch with the same batch order;
6. writes one JSON line per event to `metrics.jsonl` (start, batch, epoch, checkpoint, resume, interrupted, done, stopped, error) and finally `model.pt` (the weights) and `results.json` (test accuracy, confusion matrix, sample predictions, feature maps, first-layer filters, and per-class IoU for segmentation).

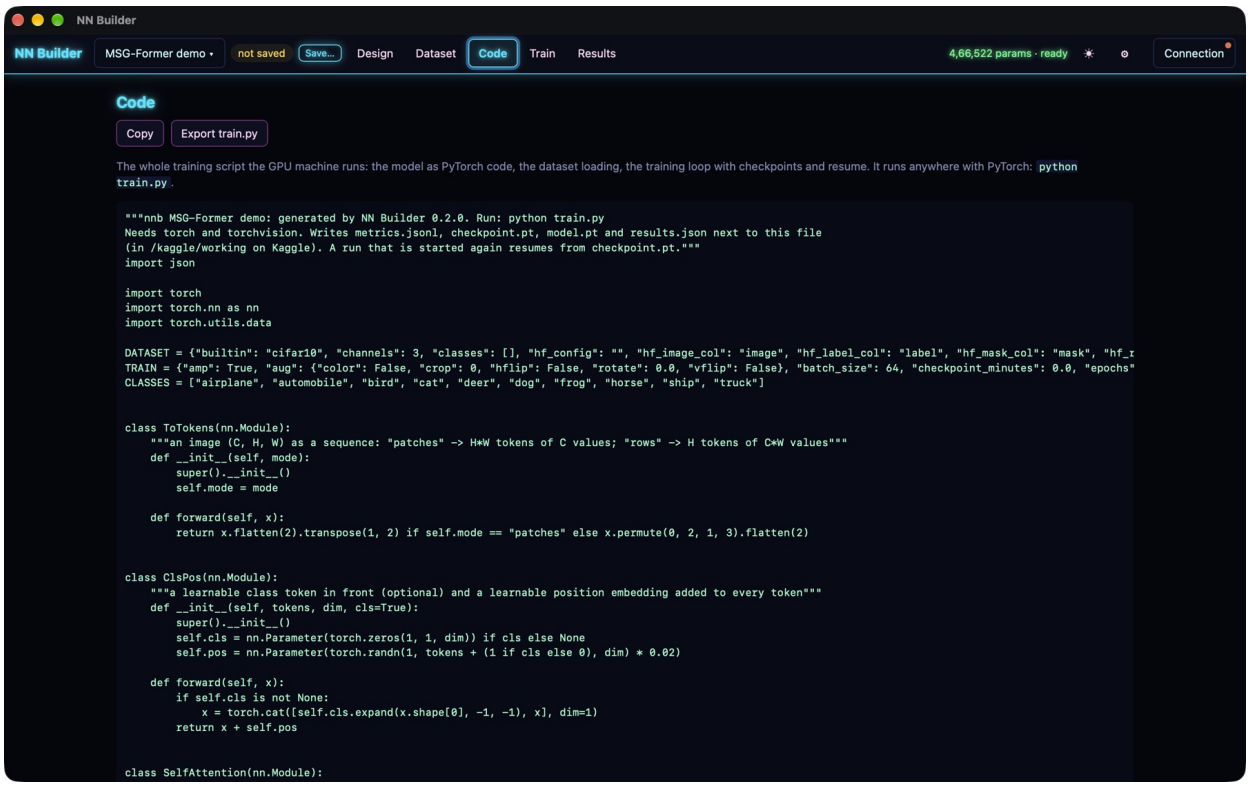



Figure 21: The Code tab: the generated `train.py` (here the start of MSG-Former's: the settings as Python literals and the transformer modules).

Signals: `TERM`/`INT` end the run cleanly (the model and results are still saved); on EC2, `USR1` means "AWS is taking the instance back" (checkpoint and exit to be resumed) and `USR2` means "checkpoint now" (Section 13.8).

# 12 The Connection tab

Tokens and keys are kept in the system keychain (macOS Keychain, Windows Credential Manager, Linux Secret Service) and handed to one call at a time; they are never written into projects, code or logs. When a run needs one on a remote machine, it is written there as a file readable only by its owner, through the SSH connection's input, never on a command line.

**Kaggle.** kaggle.com → profile → Settings → API → Create New Token. Paste the username and key (or the whole `kaggle.json`). GPU notebooks need a phone-verified account.

**Hugging Face.** Only for gated or private datasets: a "Read" token from huggingface.co → Settings → Access Tokens.

**AWS.** An IAM user's access key that may only start, stop and describe your instance (the page shows the policy to paste, with `ec2:DescribeSpotInstanceRequests` added for spot details).

**Tools.** **Install tools** makes a small private Python environment with the `kaggle` and `boto3` packages (about 60 MB) used to talk to Kaggle and AWS. No deep-learning library is installed.

## 12.1 Machines

**College / any SSH server.** Host (or an alias from `~/.ssh/config`), user, port, login by key or *password*, optional jump host, remote folder (default `.nn_builder`), optional existing Python with PyTorch. With a password, **Connect (log in once)** opens a Terminal window where you type it; the application then reuses that connection and never sees the password. **Test** checks the login; **Prepare** checks Python, the GPUs (`nvidia-smi`), the disk, and installs PyTorch in the remote folder (no administrator rights needed). **Keep runs going with** chooses how a run survives logging out: SLURM (`sbatch`, the right way on a shared cluster), `systemd-run` (needs "lingering"), `tmux`, or `nohup`+`setsid`; *automatic* picks the best the machine allows.

**AWS EC2 instance.** Region, instance id, user (`ubuntu`) and the instance's key file (`.pem`). **Check instance** reports its state, type and shutdown behaviour (it must be *Stop*, not *Terminate*, or a finished run would delete the instance), and for a spot instance its request type and interruption behaviour. The application starts the instance for a run and the run stops it at the end.

**Keep a copy of checkpoints on this Mac.** For either kind: every new checkpoint is copied back while the application is open, so a run can continue on another machine if its own machine is lost (essential for spot instances that may be terminated).

# 13 Training

## 13.1 Where to train

| | **Kaggle** | **College GPU (SSH)** | **AWS EC2 g4dn.xlarge** |
|---|---|---|---|
| GPU | T4 or P100 (Kaggle's choice) | whatever the server has (e.g. RTX A6000 48 GB) | NVIDIA T4 16 GB, 4 vCPU |
| Cost | free (about 30 GPU hours a week) | free | per hour; spot is cheaper but may be interrupted |
| Limits | about 12 h per run; runs may queue | the server's rules (SLURM time limits) | your quota for G instances |
| Progress | from the notebook's log, every 30 s | live (every few seconds) | live |
| Needs | Kaggle token | SSH access | AWS key, key file, quota |

### 13.2 Training settings

| Setting | Meaning and advice |
|---|---|
| Train on | Kaggle, or a machine from the Connection tab |
| GPU on the machine | which GPU(s) to use on a shared server (empty = any free one) |
| Epochs | passes over the training data |
| Batch size | images per step; larger is faster but needs more memory (halve it on "out of memory") |
| Learning rate | 0.001 suits Adam on small networks; very large networks (ViT-Base and bigger) need 0.0001 or less [37] |
| Optimizer | Adam [17], AdamW (weight decay applied separately [24]) or SGD with momentum 0.9 (often better for ResNet-style networks with a good schedule [8]) |
| Training recipe | learning-rate schedule (constant; cosine decay to 0; or halve when the validation loss stalls for 3 epochs), linear warm-up over the first epochs, weight decay, label smoothing, gradient clipping. **Use recommended** sets AdamW, weight decay 0.05, 5 warm-up epochs then cosine, label smoothing 0.1 and clipping at 1.0, the usual recipe for transformers and a good one for CNNs [37, 8]. The defaults are plain constant-rate training, as in earlier versions |
| Training images (0 = all) | a quick trial on a subset |
| End the run after (hours of running) | a time limit, summed over resumes (protects the budget) |
| Checkpoint every (minutes) | also checkpoint inside an epoch; 0 = only at the end of each epoch (spot instances then use 10 minutes) |
| Automatic resumes | spot instances: how many interruptions are resumed automatically (default 20) |
| Validation part of the training data | held out from the training images (default 0%: the curves' "validation" is then the test data). Use 10–20% whenever results are reported: the test data is then touched only once, at the end |
| Early stopping | stop when the validation loss (or accuracy, or mIoU) has not improved for *patience* epochs; the best epoch's weights become `model.pt` and the test accuracy is measured on them. Switching it on sets a 10% validation part if there is none. Epochs is then the maximum |
| Random seed | fixes the initial weights, the derived splits and the batch order; run several seeds to report a mean and spread |
| Mixed precision | faster on T4/A100-class GPUs, half the activation memory [27]. GPUs with bfloat16 (A100, A6000, RTX 30xx and newer) use it by themselves (no loss scaling needed); networks with convolutions use the faster channels-last memory layout on any NVIDIA GPU |
| Split a batch that does not fit | on by default: when the GPU runs out of memory, the batch is processed in 2, 4, 8 . . . parts whose gradients add up to the whole batch's, and the run continues instead of failing. Same batch, schedule and results (BatchNorm normalises over the smaller parts, the only difference); slower. The run card says when it happened |

| Setting | Meaning and advice |
|---|---|
| Save memory | activation checkpointing [3]: `train.py` cuts the network into about $\sqrt{n}$ parts at places where only one tensor passes, keeps only the parts' outputs and recomputes the inside of each part in the backward pass. The layer outputs then need roughly $2/\sqrt{n}$ of the memory, for about 30% more time; the results are the same |
| Stop the EC2 instance | stop the instance when the run ends (saves money) |
| Data augmentation | horizontal / vertical flip, random crop after padding, rotation, brightness/contrast; training images only [4]. Flips change the meaning of digits; vertical flips suit aerial images |
| Copy my token to the machine | needed for Kaggle data on a server/EC2 and for gated Hugging Face data |
| Start from ImageNet weights | transfer learning: torchvision's pretrained ResNet-18, VGG-16 or ViT-B/16 loaded into the matching demo, tensor by tensor (a new output head for your classes starts at random); optionally train only the new head. Images are then normalised with ImageNet's statistics. Usually far better than training from scratch on small datasets; use a smaller learning rate (e.g. 0.0001) when fine-tuning everything |

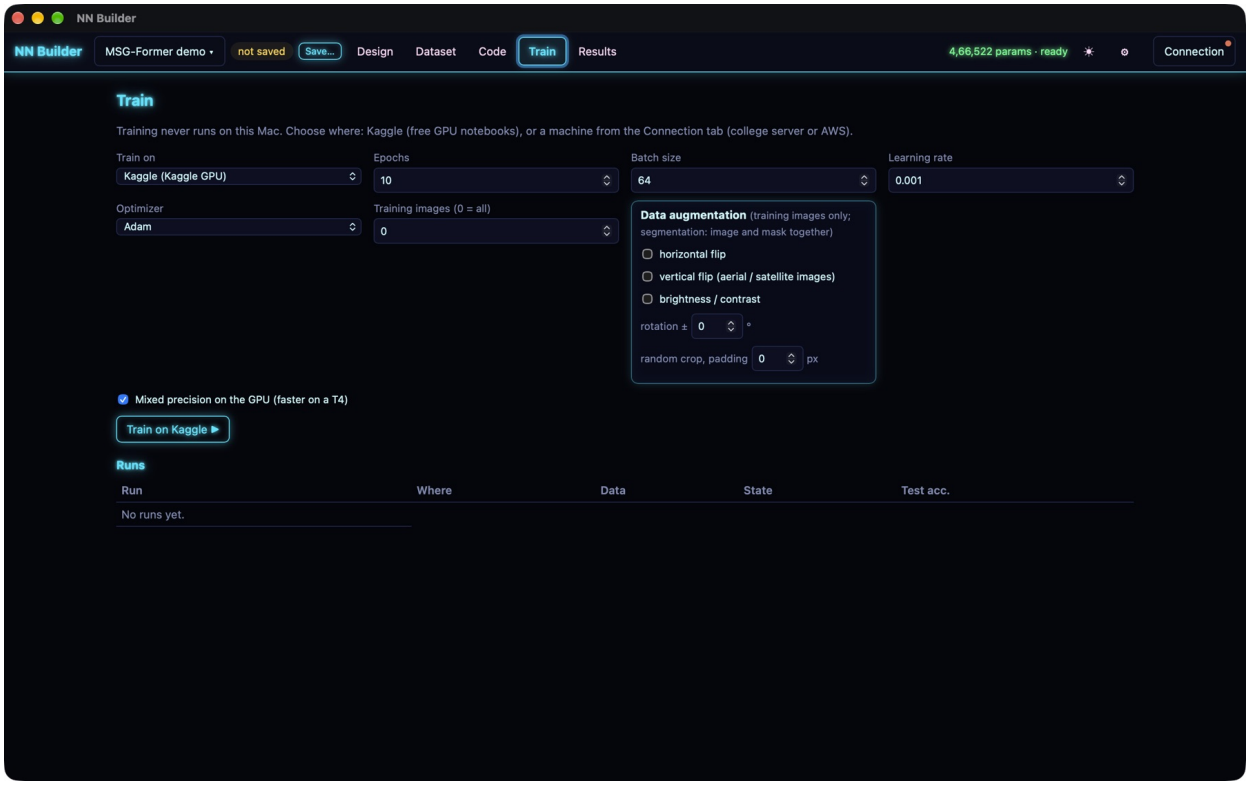



Figure 22: The Train tab: where to train, the main settings, augmentation, and the list of runs. Settings for SSH and EC2 machines (time limit, checkpoint interval, automatic resumes, stopping the instance) appear when such a machine is chosen.

Above the Train button the tab says what is still missing for the chosen target (tools, token, key) and gives a rough estimate of the GPU memory the training needs (16 bytes per parameter plus the layer outputs at the chosen batch size), compared with the target's GPU (Kaggle and EC2 g4dn: 16 GB; an SSH machine: as **Prepare** found it). It warns when the design is likely not to fit.

## 13.3 Unbalanced classes

Real land-cover data is rarely balanced (BigEarthNet's rarest labels are on under 1 % of the patches), and a network can then score well while ignoring the rare classes. The Dataset tab shows the images per class next to the class folders (Read folder / Read files), with a warning above a 3:1 ratio; **Check data** counts them on the machine for every source. Three settings in the Train tab's recipe help:

- **Unbalanced classes**: *weight by inverse frequency* gives class $c$ the weight $N/(K\,n_c)$ in the loss ($N$ training images, $K$ classes, $n_c$ of class $c$), so every class counts as much in total; *by its square root* is gentler. For multi-label, each class's positive examples weigh $(N - n_c)/n_c$ (its square root) against its negatives. Segmentation counts pixels. The weights are computed on the training machine and listed in the log and in Results.
- **multi-label: a threshold per class**: after training, each class gets the threshold (5 %–95 %) with the highest F1 on the validation data instead of 50 %; the test scores use them, Results lists them, and `thresholds.json` is saved next to the model. It needs validation data, so it sets the validation share to 10 % if it was 0.
- **Loss**: *cross-entropy* (binary cross-entropy for multi-label) is the default. *Focal* multiplies each image's (pixel's) term by $(1 - p)^\gamma$, where $p$ is the probability given to the right answer, so the images the network already gets right count little and the rare, hard ones count more; $\gamma = 0$ is plain cross-entropy, 2 is usual. For segmentation and multi-label there is also *Dice*: $1 - \frac{1}{K}\sum_c \frac{2|P_c \cap T_c|}{|P_c| + |T_c|}$ on the batch's predicted probabilities $P_c$ and true pixels (images) $T_c$ of class $c$, so a small class counts as much as a large one; *Dice + cross-entropy* adds the two and learns more steadily at the start. Unlabelled pixels are left out of every loss, and class weights apply to all of them (Dice: a weighted mean over the classes). Dice for one class per image is refused before the run starts. Results names the loss used.

Look at the per-class scores (precision, recall, F1) rather than the accuracy alone.

## 13.4 Check data before training

**Check data on ...**, next to the Train button, starts a short run on the same target that loads the data exactly as training would, reports it, and sends one batch through the network, but does not train: minutes instead of a GPU hour lost to a wrong band order or a bad labels file. The run is listed as a *data check* ("looks right", or how many things to look at), and the Results tab shows:

- the number of training, validation and test images, and the training images per class (pixels per class for segmentation, counted on a sample), with a bar per class;
- for satellite images, every band's raw values in the files (minimum, maximum, mean, spread, empty and zero pixels) from a sample of training images;
- twelve test images as the network receives them (composites for satellite images) with their labels;
- warnings: a band that is empty or constant (often a wrong band order), many empty pixels, Sentinel-1 in linear units without **convert to decibels**, Sentinel-2 values far above 10 000, a class without any images, strongly unbalanced classes, very few images, rows of a labels CSV without a file, and anything the data loading itself reported.

Nothing is saved in the project's settings: the next **Train on ...** trains as usual.

## 13.5 Following a run

**Train on ...** starts a background task whose log appears at the bottom right. The run card then shows the state (starting, running with the epoch, interrupted, done, stopped, crashed, failed), the GPU, the training set size, the loss curves (per batch, training and validation per epoch) and the accuracy curves (and mIoU for segmentation), the last checkpoint, and the log. **Stop** ends the run

cleanly (the current epoch ends early, the model and results are saved). **Fetch results** copies the outputs back (for a stopped EC2 instance it starts the instance, copies, and stops it again). The **Runs** table lists every run of the project with its seed and test accuracy.

### 13.6 Several seeds, comparing runs, exporting results

A single training run is one sample: the same design and settings with another random seed give a slightly different accuracy. Next to the Train button, **seeds** starts 3 or 5 runs at once with seeds $s, s{+}1, \ldots$ (one group; on Kaggle one notebook per seed, which Kaggle may queue; on a server they share its GPU). Tick runs in the **Runs** table (or select a whole group of seeds) and press **Compare selected**: their validation curves are overlaid, and a table gives each run's recipe, epochs, best epoch, test accuracy (and top-5), training time, and for each group the mean ± sample standard deviation. **Export CSV + LaTeX** writes the table to the project's `exports` folder as `runs-<time>.csv` and `runs-<time>.tex` (a `booktabs` tabular with a "Mean ± s.d." row, ready to `\input` into a paper).

### 13.7 Independence from the connection

On a server or EC2, the run folder is copied with `rsync` and started detached (SLURM, systemd, tmux or nohup). The application only looks at it: every few seconds while the run is open it copies the new metric lines and the end of the log. Closing the application, sleeping the laptop or losing the network does not affect the run; the application catches up when it can reach the machine again.

### 13.8 EC2 spot instances

Spot instances use spare EC2 capacity at a lower price, but AWS may take one back with a two-minute notice. NN Builder recognises a spot instance by itself. The recommended setup is a *persistent* spot request with interruption behaviour *stop* (AWS restarts the instance when capacity returns, with its disk), plus **Keep a copy of checkpoints on this Mac**.

| Situation | What NN Builder does |
|---|---|
| Two-minute interruption notice | a watcher on the instance sees it in the instance metadata; the script saves a mid-epoch checkpoint at once and exits to be resumed |
| Rebalance recommendation (earlier warning) | a checkpoint is saved now; training continues |
| Interruption without notice | at most the checkpoint interval of training is lost (10 minutes by default on spot) |
| Interrupted while writing a checkpoint, or disk full | checkpoints are written to a temporary file and swapped in: the previous one is never damaged |
| Instance restarted while the laptop is off | a boot-time job on the instance resumes the run: no laptop needed |
| Instance back while the application is open | the application follows the instance's new address and restarts the run if needed |
| Two starters at once | a lock lets only one copy of a run train |
| Resumed in the middle of an epoch | the remaining batches of that epoch, in the same order; optimiser, mixed-precision scaler and random state restored |
| Interrupted again and again | at most the set number of automatic resumes; the time limit counts running time over all resumes |

| Situation | What NN Builder does |
|---|---|
| Instance terminated | shown as "terminated"; with the copy on the Mac, **Continue on another machine** starts a new run from it (for example on a college GPU) |
| Old instance returns after moving | when that run is opened and the instance answers, the run there is stopped (cancel its spot request in the AWS console to be sure) |
| **Stop** while the instance is down | remembered: the run is stopped instead of resumed when the instance answers |

### 13.9 Can my GPU train this network?

Use the 16-bytes-per-parameter rule of Section 6 plus room for activations. A 48 GB RTX A6000 can train ViT-g/14 (about 18 GB of fixed state) with batches of roughly 16 to 32 images at 224×224; a 16 GB T4 cannot. Large networks trained from scratch on small datasets overfit: ViT-g was trained on billions of images [45], and even ViT-Base needs strong augmentation and regularisation, or pre-training, to do well on small data [37, 39].

With **Save memory** the layer outputs shrink to a small fraction (the estimate on the Train tab takes it into account), so a network whose weights and optimizer state fit can usually train with a reasonable batch; the weights and optimizer state themselves (16 bytes per parameter) do not shrink. After the first epoch the run card shows the memory really used (the peak of the GPU allocator), the precision, and whether batches were split; it also warns when the GPU waited for data more than 30% of an epoch (loading and augmenting images is then the bottleneck: the data workers already use one CPU core each, up to 8).

### 13.10 How large a design the application handles

The application never holds weights, so the number of parameters does not matter to it; the number of *blocks* does. Measured on a Mac (designs written as text: ViT-style encoders, 224×224 input):

| Design | parameters | blocks | window memory | analysis per edit |
|---|---|---|---|---|
| none open | – | – | 56 MB | – |
| ViT-g/14 (40 layers, width 1 408) | 1.0 B | 287 | 164 MB | 1–2 ms |
| width 8 192, 32 layers | 25.8 B | 231 | 151 MB | 1 ms |
| width 12 288, 55 layers | 99.7 B | 392 | 192 MB | 1 ms |
| width 4 608, 100 layers | 25.5 B | 707 | 287 MB | 3 ms |
| width 2 048, 500 layers | 25.2 B | 3 507 | 965 MB | 21 ms |

The canvas needs about 0.3 MB per block drawn. **Settings** → **Design size** sets the most blocks a design may have: recommended from the computer's memory (a quarter of it at 0.3 MB per block, e.g. 13 000 blocks with 16 GB, 105 000 with 124 GB) and editable; it concerns the computer running NN Builder, not the training machine. Above 500 blocks the canvas says so and offers **Collapse repeated blocks (×N)**, which draws each repeated group once; the Text tab edits a design of any size without drawing it. Conv2D filters and attention heads go up to 65 536 and 512; a setting outside its range is an error on its block (it is never silently changed).

# 14 The Results tab

For a finished run (open it in the Train tab): the test accuracy (and top-5 accuracy with more than five classes, and mIoU), precision, recall and F1 of every class with their macro averages and the hardest classes, the confusion matrix (rows: true class, columns: predicted; the diagonal is correct), sample predictions with their confidence (wrong ones marked), the feature maps of the first test image (first eight channels of early layers), the first layer's filters as small pictures, and for segmentation the true and predicted masks and the IoU of every class; **Grad-CAM** heat maps on the sample images (where the network looked: the last image-shaped layer's feature maps weighted by how strongly the predicted class depends on them [34]); and a gallery of up to 16 **misclassified** test images with their true and predicted class. **Show files** reveals `model.pt`, `train.py` and `results.json`, and with **Also save the trained model as TorchScript and ONNX** also `model.ts` (load with `torch.jit.load`, no NN Builder code needed) and `model.onnx` (for ONNX Runtime and other frameworks). The weights load in PyTorch with `Net().load_state_dict(torch.load("model.pt"))` using the `Net` class from the same `train.py`.

# 15 The demo architectures

Each demo opens unsaved on the canvas (Projects page, **File** → **New from Demo**, or **Famous architectures** in the Design tab) at the input size it was made for. Parameter counts below are the application's, with a 10-class output (2 classes for the segmentation networks); they change with the number of classes. Where a demo differs from the paper, the difference is stated.

| Demo | Year | Task | Input | Main idea | Parameters |
|---|---|---|---|---|---|
| LeNet-5 | 1998 | classify | 1×28×28 | convolution + subsampling | 61 706 |
| Small CNN (CIFAR-10) | 2015 | classify | 3×32×32 | VGG-style blocks + BatchNorm | 551 018 |
| AlexNet | 2012 | classify | 3×224×224 | deep CNN on GPUs, ReLU, dropout | 58 322 314 |
| VGG-16 | 2014 | classify | 3×224×224 | only 3×3 convolutions | 134 301 514 |
| GoogLeNet | 2014 | classify | 3×224×224 | Inception modules | 5 983 802 |
| ResNet-18 | 2015 | classify | 3×224×224 | residual connections | 11 181 642 |
| MobileNet v1 | 2017 | classify | 3×224×224 | depthwise separable convolutions | 3 217 226 |
| DenseNet (small) | 2017 | classify | 3×32×32 | dense connectivity | 76 810 |
| Transformer encoder | 2017 | classify | 1×28×28 | self-attention on image rows | 71 498 |
| ViT-Base/16 | 2020 | classify | 3×224×224 | image patches as tokens | 85 806 346 |
| ViT-Tiny (CIFAR-10) | 2020 | classify | 3×32×32 | a small ViT | 1 806 538 |
| ViT-g/14 | 2022 | classify | 3×224×224 | scaling a ViT to 1 B parameters | 1 011 216 522 |
| U-Net | 2015 | segment | 3×128×128 | encoder–decoder with skips | 7 760 130 |
| Attention U-Net | 2018 | segment | 3×128×128 | attention gates on the skips | 7 847 894 |
| MSG-Former | 2026 | classify | 3×32×32 | NN Builder original (hybrid) | 466 522 |

## 15.1 LeNet-5 (1998)

**Background.** LeCun et al. trained LeNet-5 to read handwritten digits on bank cheques [21]; it established the pattern every CNN since has followed: convolutions that detect local features, subsampling that makes them tolerant to small shifts, and fully connected layers that classify.

**As built in NN Builder.**

| Block | Settings | Output | Parameters |
|---|---|---|---|
| Conv2D C1 | 6 filters, 5×5, padding 2, tanh | 6×28×28 | 156 |
| Pool S2 | average 2×2 | 6×14×14 | 0 |
| Conv2D C3 | 16 filters, 5×5, valid, tanh | 16×10×10 | 2 416 |
| Pool S4 | average 2×2 | 16×5×5 | 0 |
| Flatten | | 400 | 0 |
| Dense C5, F6 | 120 then 84 units, tanh | 84 | 48 120 + 10 164 |
| Output | 10 classes | 10 | 850 |

**Differences from the paper.** The paper used 32×32 inputs with no padding (padding 2 on 28×28 MNIST gives the same 28×28 maps), trainable subsampling coefficients, a hand-designed partial connection table in C3, and Gaussian output units; the demo uses plain average pooling, full connections and a softmax output.

**Training it.** Dataset: built-in MNIST (no resizing needed). Adam, learning rate 0.001, batch 64, 5–10 epochs; it reaches about 99% test accuracy on any GPU in a few minutes. Look at the first-layer filters in Results: they become stroke and edge detectors.

### 15.2 Small CNN for CIFAR-10

**Background.** Not one paper but the standard "VGG-style" baseline for 32×32 colour images: stacks of 3×3 convolutions [35] with BatchNorm [16] and dropout [36].

**As built.** Three blocks with 32, 64 and 128 filters; each block is two times (Conv2D 3×3 without bias → BatchNorm → ReLU), then max pooling 2×2 and dropout (0.2, 0.3, 0.4). The image goes 32 → 16 → 8 → 4; Flatten gives 2 048 values, then Dense 128 (ReLU), Dropout 0.5 and the output. The classifier holds 263 562 of the 551 018 parameters.

**Training it.** Built-in CIFAR-10; Adam 0.001, batch 128, 30–50 epochs, with horizontal flips and a random crop with 4 pixels of padding: such networks commonly reach the mid to high 80s in test accuracy. Without augmentation the training accuracy runs far ahead of the validation accuracy (overfitting): compare the two curves in the Train tab.

### 15.3 AlexNet (2012)

**Background.** Krizhevsky, Sutskever and Hinton won the ImageNet challenge [33] of 2012 by a wide margin with a deep CNN trained on two GPUs [19]; its ingredients (ReLU instead of tanh, dropout, data augmentation, GPU training) started the deep-learning era in computer vision.

**As built.** Conv2D 96 filters 11×11 stride 4 padding 2 (224 → 55) → max pool 3×3 stride 2 (overlapping, 55 → 27) → Conv2D 256 5×5 → pool (13) → Conv2D 384, 384, 256 (3×3) → pool (6) → Flatten (9 216) → Dropout 0.5 → Dense 4 096 → Dropout 0.5 → Dense 4 096 → Output. Of the 58.3 million parameters, 37.8 million sit in the first Dense layer: the layer table's Classifier group shows this at a glance.

**Differences from the paper.** No local response normalisation, and no split of the layers into two groups (one per GPU); with 1 000 classes the demo has about 62 million parameters (the paper reports 60 million). A later note by Krizhevsky describes the single-GPU variant [20].

**Training it.** AlexNet expects 224×224 images: use CIFAR-10 with **Resize** (slow and not ideal: tiny photos enlarged) or a Hugging Face dataset of larger photos. Use a smaller learning rate (Adam 0.0001) and flips. It fits a T4 at batch 64–128.

## 15.4 VGG-16 (2014)

**Background.** Simonyan and Zisserman showed that depth matters and that small filters suffice: two 3×3 convolutions see as far as one 5×5, three as far as one 7×7, with fewer parameters and more non-linearities [35].

**As built.** Five blocks of 3×3 convolutions (ReLU, padding same) with 64, 64 | 128, 128 | 256 ×3 | 512 ×3 | 512 ×3 filters, each block followed by 2×2 max pooling (224 → 7); Flatten (25 088), two Dense 4 096 with dropout 0.5, Output. 134 301 514 parameters, exactly torchvision's `vgg16` with a 10-class head; 102.8 million of them are in the first Dense layer.

**Training it.** Like AlexNet: 224×224 images, Adam 0.0001 or SGD 0.01 with momentum, batch 32–64 on a T4. Watch the receptive field grow in the inspector: 3×3 convolutions and poolings reach 212×212 pixels at the last convolution.

## 15.5 GoogLeNet / Inception v1 (2014)

**Background.** Szegedy et al. asked which filter size to use, and answered "all of them, side by side": the Inception module runs 1×1, 3×3 and 5×5 convolutions and a pooling branch in parallel and concatenates them [38]. 1×1 convolutions first reduce the channels (the idea comes from Network in Network [23]), keeping the cost low: GoogLeNet has twelve times fewer parameters than AlexNet and was more accurate.

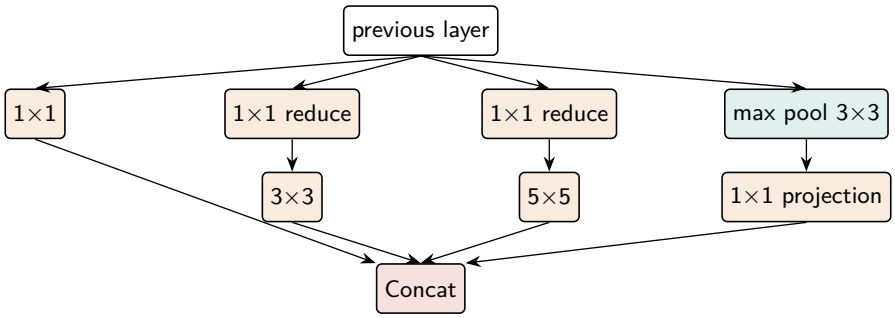



Figure 23: An Inception module: four branches side by side (all keep the height and width); Concat stacks their channels.

**As built.** Stem: Conv2D 64 7×7 stride 2, max pool, Conv2D 1×1 64, Conv2D 3×3 192, max pool (224 → 28). Then nine Inception modules with the paper's channel numbers: 3a (output 256 channels), 3b (480), max pool, 4a–4e (512, 512, 512, 528, 832), max pool, 5a (832), 5b (1 024); global average pooling, dropout 0.4, output. 5 983 802 parameters.

**Differences from the paper.** The two auxiliary classifiers used during training are left out (later work found them of little use), and there is no local response normalisation.

**Training it.** 224×224 inputs; without BatchNorm, deep plain networks like this train slowly: use Adam 0.0001–0.0003 and patience, or add BatchNorm after the convolutions yourself.

## 15.6 ResNet-18 (2015)

**Background.** Deeper plain networks became *harder to train*, not just to generalise: adding layers raised the training error. He et al. let every block learn a correction to its input, $y = x + F(x)$, through a shortcut [9]; gradients then flow through the shortcuts, and networks of 50, 101, even 1 000 layers train well [10].

**As built.** Stem: Conv2D 64 7×7 stride 2 + BatchNorm + ReLU, max pool 3×3 stride 2 (224 → 56). Four stages of two basic blocks with 64, 128, 256 and 512 filters; the first block of stages 2–4 has stride 2 and a 1×1 stride-2 convolution with BatchNorm on the shortcut (56 → 28 → 14 →

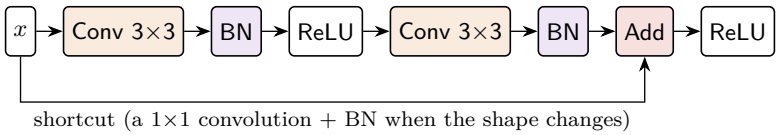



Figure 24: A basic residual block as built on the canvas: $y = \mathrm{ReLU}(x + F(x))$.

7). Global average pooling and the output. 11 181 642 parameters, exactly torchvision's `resnet18` with a 10-class head.

**Training it.** The most forgiving of the large demos thanks to BatchNorm and the shortcuts. At 224×224 it fits a T4 at batch 64–128. For CIFAR-10 at 32×32 the stem shrinks the image too early (32 → 8 before the first block); the usual CIFAR variant replaces the stem with a 3×3 stride-1 convolution and drops the max pooling, an easy edit on the canvas.

### 15.7 MobileNet v1 (2017)

**Background.** For phones, Howard et al. factorised a convolution into a *depthwise* 3×3 convolution (one filter per channel, spatial only) and a *pointwise* 1×1 convolution (mixing channels) [13]. The cost drops by a factor of about $1/N + 1/k^2$ ($N$ output channels, $k = 3$): roughly eight to nine times fewer computations.

**As built.** Conv2D 32 3×3 stride 2 + BN + ReLU, then 13 depthwise separable blocks (Depthwise Conv2D 3×3 + BN + ReLU, Conv2D 1×1 + BN + ReLU) with 64, 128, 128, 256, 256, 512 (×6), 1 024, 1 024 channels; five of them have stride 2 (224 → 7); global average pooling and the output. 3 217 226 parameters (4.2 million with 1 000 classes, as in the paper). Width and resolution multipliers of the paper can be imitated by changing the filter counts and the input size.

**Training it.** Light and fast; SGD or Adam 0.001 with BatchNorm works well. Depthwise convolutions are memory-bound on GPUs, so it is not proportionally faster to *train*; its advantage is at inference on small devices.

### 15.8 DenseNet, small CIFAR version (2017)

**Background.** Huang et al. connected each layer to *all* earlier layers of its block by concatenation [15]: features are reused instead of re-learnt, so each layer can be narrow (the "growth rate" $k$, here 12 channels).

**As built.** Conv2D 24 3×3; three dense blocks of four layers (BatchNorm → ReLU → Conv2D 12 3×3, concatenated to everything before); between blocks a transition (BatchNorm → 1×1 Conv2D halving the channels → average pool 2×2). The channels go 24 → 72 → 36 → 84 → 42 → 90 while the image goes 32 → 16 → 8; BatchNorm, ReLU, global pool and output. Only 76 810 parameters.

**Differences from the paper.** The paper's CIFAR models are much deeper (for example 100 layers with bottleneck 1×1 convolutions inside each layer, about 0.8 million parameters); the demo is a small teaching version. The paper's transitions have a ReLU after their BatchNorm; the demo's do not.

**Training it.** CIFAR-10 built in, Adam 0.001, flips and crops, 50+ epochs; small but deep, so batch 64–128 fits easily.

### 15.9 Transformer encoder on image rows (2017)

**Background.** The Transformer of Vaswani et al. replaced recurrence with self-attention: every token computes a weighted mix of all tokens, the weights coming from query–key similarities [40].

Its encoder is a stack of attention and two-layer MLP blocks with residual connections and layer normalisation [1].

**As built.** A teaching version on MNIST: To tokens (*rows*) makes each of the 28 rows a token of 28 values; Dense 64 embeds them; Class token + position gives 29 tokens of 64; two pre-norm encoder blocks with 4 heads, an MLP of 128 (GELU [11]) and dropout 0.1; LayerNorm, the class token, output. 71 498 parameters.

**Training it.** MNIST, Adam 0.001, 10–20 epochs: it reaches high-90s accuracy and shows that attention can learn image structure from a sequence of rows. Change the number of heads (64 must stay divisible by it) or blocks and compare.

### 15.10 ViT-Base/16 (2020)

**Background.** Dosovitskiy et al. showed that a plain Transformer encoder on image patches, "an image is worth 16×16 words", matches the best CNNs when pre-trained on enough data [6].

**As built.** Conv2D 768 filters 16×16 stride 16 (the patch embedding: 224 → 14×14), To tokens (196 tokens of 768), Class token + position (197 tokens), 12 encoder blocks (12 heads, MLP 3 072), LayerNorm, class token, output. 85 806 346 parameters, exactly torchvision's `vit_b_16` with a 10-class head (86.6 million with 1 000 classes).

**Training it.** ViTs lack the built-in assumptions of convolutions (locality, translation equivariance) and need much data or strong augmentation and regularisation [37, 39]. From scratch on a small dataset it does worse than ResNet-18. Use a low learning rate (0.0001 or less; AdamW with warm-up is the usual recipe [24, 8], not built in), batch 32–64 on a T4 with mixed precision.

### 15.11 ViT-Tiny for CIFAR-10

**Background.** Small ViTs in the spirit of DeiT-Ti [39] (embedding 192, 3 heads) make transformers trainable on small images in minutes.

**As built.** Conv2D 192 4×4 stride 4 (32 → 8×8 = 64 patches), To tokens, Class token + position (65 tokens), 6 encoder blocks with 3 heads, MLP 384 and dropout 0.1, LayerNorm, class token, output. 1 806 538 parameters. (DeiT-Ti itself has 12 blocks and an MLP of 768.)

**Training it.** CIFAR-10 with flips and crops (augmentation matters a lot for ViTs), Adam 0.001 or a little lower, 50+ epochs. It usually stays below the Small CNN on CIFAR-10 when trained from scratch for a short time: a useful experiment on the data hunger of transformers.

### 15.12 ViT-g/14, one billion parameters (2022)

**Background.** Zhai et al. studied how ViTs scale with model size, data and compute, training up to two billion parameters on billions of images [45]. ViT-g/14 ("giant") is their one-billion-parameter model.

**As built.** Conv2D 1 408 filters 14×14 stride 14 (224 → 16×16 = 256 patches), Class token + position (257 tokens), 40 encoder blocks with 16 heads and MLP 6 144, LayerNorm, class token, output: 1 011 216 522 parameters. Each encoder block alone has 25 250 176 parameters (attention $4d^2 + 4d = 7\,935\,488$; MLP $2 \cdot 1408 \cdot 6144 + 6144 + 1408 = 17\,309\,056$; two LayerNorms 5 632). The canvas holds about 290 blocks; the layer table groups them into Embedding, 40 encoder blocks, Final norm and Classifier.

**Training it.** About 16 GB of fixed state for Adam (Section 6): it does *not* fit a 16 GB T4 (Kaggle, g4dn.xlarge). A 48 GB RTX A6000 fits it with batches of roughly 16–32 at 224×224 (estimate), at around 1.5 TFLOP of compute per training image; a checkpoint with the optimiser state is about 12 GB and the final `model.pt` about 4 GB, both copied over the network. From scratch on a small

dataset it overfits badly and scores below ResNet-18; its value here is to see how a transformer scales (compare it block by block with ViT-Base) and to measure memory and speed.

### 15.13 U-Net (2015)

**Background.** Ronneberger et al. designed U-Net for biomedical image segmentation with few training images [32]: a contracting path captures context, an expanding path recovers resolution, and skip connections hand the fine detail of each encoder level to the decoder level of the same size.

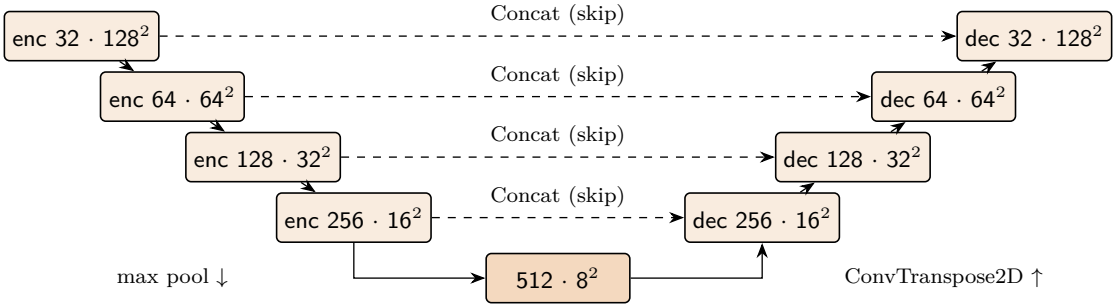



Figure 25: The U shape as built: channels · image size per level, 128 → 8 pixels and back, with a skip at every level.

**As built.** Input 3×128×128. Encoder: four levels of two 3×3 convolutions (ReLU) with 32, 64, 128, 256 filters, each followed by 2×2 max pooling; bottleneck: two convolutions with 512 filters at 8×8. Decoder: at each level a 2×2 stride-2 ConvTranspose2D halving the channels, Concat with the encoder output of the same size, two 3×3 convolutions; Segmentation output (1×1) with two classes. 7 760 130 parameters.

**Differences from the paper.** The paper starts at 64 filters (1 024 at the bottom, about 31 million parameters) and uses unpadded convolutions, cropping the encoder maps before concatenating; the demo uses padding "same", so sizes match without cropping.

**Training it.** Dataset → Segmentation, with an image column and a mask column (Hugging Face) or images and masks folders (Kaggle); the class list maps mask values to classes. Flips and rotations are applied to the image and its mask together. Watch the mIoU curve and the predicted masks in Results.

### 15.14 Attention U-Net (2018)

**Background.** Oktay et al. added *attention gates* to U-Net's skip connections: the decoder's coarser signal decides which regions of the encoder features matter, suppressing irrelevant background [28].

**As built.** U-Net as above, but before each Concat the skip features $x$ pass a gate: $g$ (the up-sampled decoder signal) and $x$ are each projected by a 1×1 convolution to half the channels, added, passed through ReLU and a 1×1 convolution to one channel with a sigmoid; the resulting map $\psi \in [0,1]^{H\times W}$ multiplies $x$ (Multiply, one-channel map times image). 7 847 894 parameters.

**Difference from the paper.** The paper takes the gating signal from the coarser decoder level and resamples it; the demo uses the up-sampled decoder features, which already have the skip's size.

## 15.15 MSG-Former: an NN Builder original

MSG-Former (*Multi-Scale Gated transFormer*) is the one demo that does not come from a paper: it was designed for NN Builder as a small, fast network for 32×32 colour images that combines three published ideas in one design. It is an untested proposal, not a benchmark result: every number below (shapes, parameters, receptive fields, compute) is exact, but how well it classifies is for its first training runs to show.

### 15.15.1 The idea

Convolutional networks and transformers are good at different things. A convolution looks at a small neighbourhood and is very efficient at finding local texture and edges, but needs many layers before one output can relate two distant parts of the image. Self-attention relates every position to every other in one step, but on raw pixels it is expensive and, without the built-in assumptions of convolutions, needs much more data [6, 39]. MSG-Former therefore works in two phases:

1. **Local, multi-scale, gated** (32×32 and 16×16 pixels): convolution stages that look at three scales at once, weigh their channels by how useful they are, and keep a residual path;
2. **Global** (an 8×8 grid): the feature map becomes 64 tokens, and two Transformer encoder blocks let every region attend to every other before the class token makes the prediction.

The same convolution-first, attention-later order is used by hybrid models such as the ResNet–ViT hybrid of the ViT paper [6], CoAtNet [5] and MobileViT [26]; MSG-Former's own contribution is the form of its convolution stage.

### 15.15.2 The multi-scale gated residual block

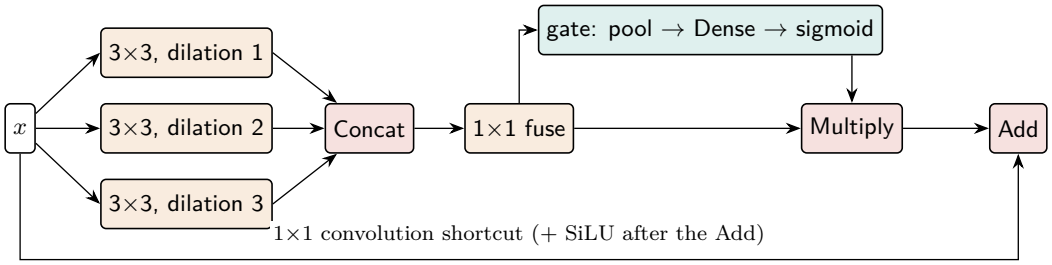



Figure 26: The multi-scale gated residual block. Each 3×3 branch is Conv2D (no bias) → BatchNorm → SiLU; the fuse is a 1×1 Conv2D → BatchNorm → ReLU; the gate is Global pool → Dense ($C/4$, ReLU) → Dense ($C$, sigmoid).

For an input $x$ with $C_{\text{in}}$ channels, a branch width $b$ and an output width $C$, the block computes

$$u_r = \mathrm{SiLU}\big(\mathrm{BN}(W_r *_r x)\big), \qquad r \in \{1,2,3\} \quad (*_r :\ 3{\times}3 \text{ convolution with dilation } r), \tag{4}$$

$$v = \mathrm{ReLU}\big(\mathrm{BN}(W_f * [u_1; u_2; u_3])\big) \qquad (1{\times}1 \text{ fuse of the } 3b \text{ concatenated channels to } C), \tag{5}$$

$$g = \sigma\big(W_2\ \mathrm{ReLU}(W_1\ \mathrm{GAP}(v) + c_1) + c_2\big) \in (0,1)^C, \tag{6}$$

$$y = \mathrm{SiLU}\big(g \odot v + W_s * x\big) \qquad (W_s :\ 1{\times}1 \text{ shortcut, } C_{\text{in}} \to C). \tag{7}$$

Each part has a reason:

**Three dilations, same cost.** A 3×3 kernel with dilation $r$ covers $(2r+1) \times (2r+1)$ pixels with the same nine weights: 3×3, 5×5 and 7×7 here [44]. The block sees fine detail and wider context at the same resolution and the price of three ordinary 3×3 convolutions, like an Inception module [38] without the cost of large kernels, or the atrous spatial pyramid of

DeepLab [2] inside a classifier. Mixing the rates 1, 2 and 3 (rather than only large ones) keeps every pixel covered and avoids the "gridding" gaps that stacked dilated convolutions can leave [41].

**1×1 fuse.** Concatenation gives $3b$ channels; the 1×1 convolution mixes the scales and sets the block's width.

**Squeeze-and-excitation gate.** Global average pooling summarises each channel over the whole image; two small Dense layers (reduction 4) turn that summary into one weight in $(0, 1)$ per channel, and Multiply scales the channels with it, so the block can emphasise the scales and features that matter for this image [14]. The original SE networks use reduction 16; with 64 and 128 channels, 4 keeps the bottleneck from becoming too narrow (16 and 32 units).

**Residual shortcut.** As in ResNet [9], the block learns a correction to its input; because the channel count changes ($32 \to 64$, $64 \to 128$), the shortcut is a 1×1 projection.

**SiLU.** A smooth activation [31] in the branches and after the sum; the fuse keeps ReLU.

### 15.15.3 The whole network

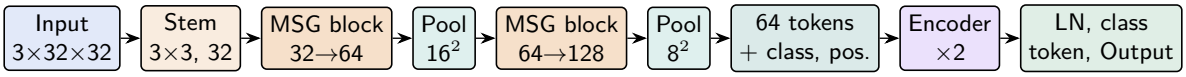



Figure 27: MSG-Former from left to right: local multi-scale stages at 32×32 and 16×16, then global attention on an 8×8 grid.

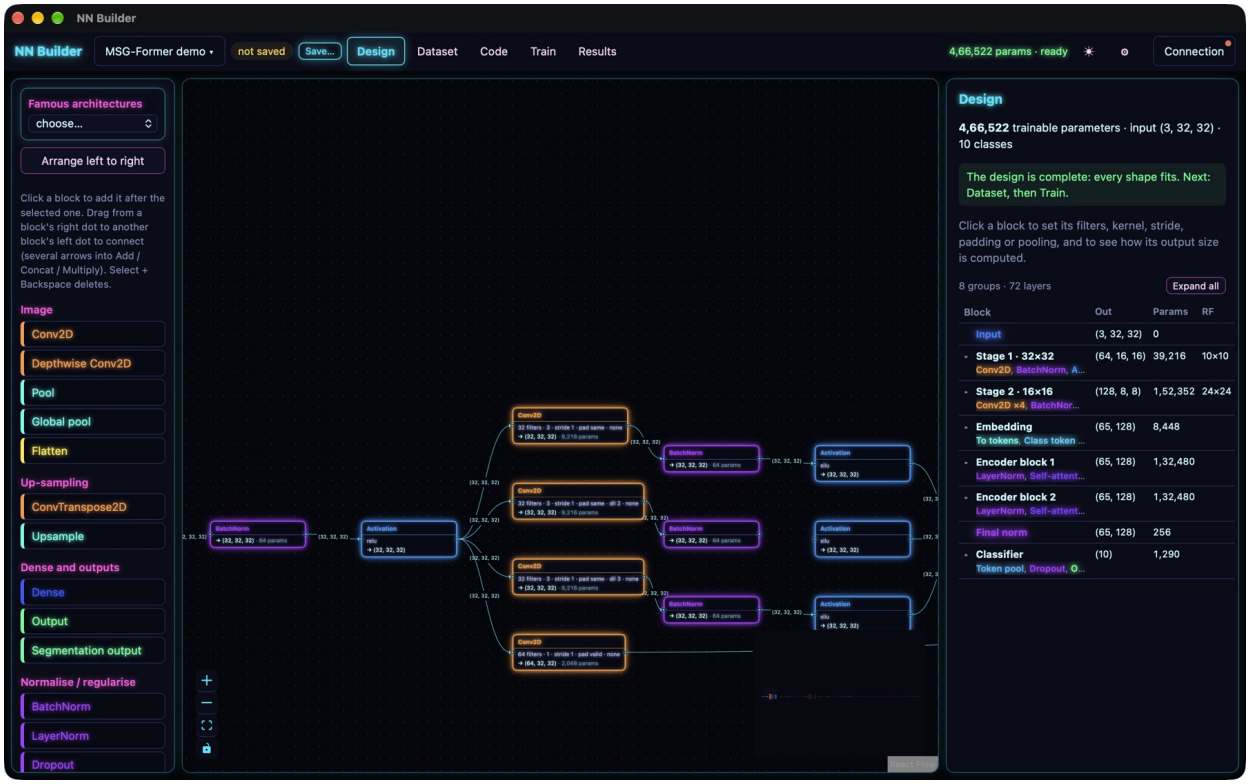



Figure 28: MSG-Former on the canvas, zoomed in on the first block: the stem's activation feeds the three dilated 3×3 convolutions (dilation 1, 2, 3, each with BatchNorm and SiLU) and, at the bottom, the 1×1 shortcut; the branches meet in Concat (96 channels). Every arrow shows its tensor shape.

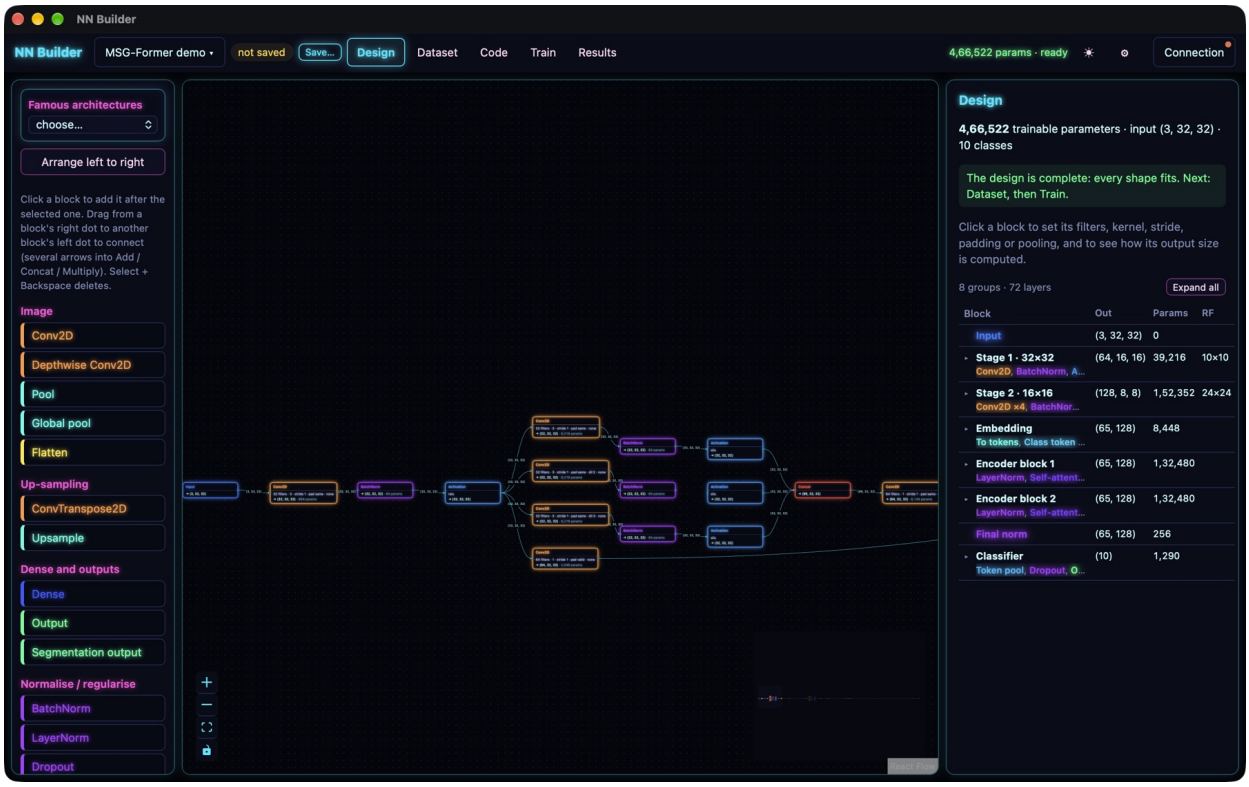



Figure 29: The whole design opened from the demo: 72 blocks, 466 522 parameters, "ready". The layer table groups them as Input, Stage 1 · 32×32, Stage 2 · 16×16, Embedding, Encoder block 1 and 2, Final norm and Classifier, each with its output, parameters and receptive field.

Table 13 lists every layer as the application computes it (with 10 classes). The receptive field (RF) is given in input pixels.

Table 13: MSG-Former layer by layer.

| Stage | Layer | Output | Parameters | RF |
|---|---|---|---|---|
| Input | image | 3×32×32 | 0 | 1 |
| Stem | Conv2D 32, 3×3, no bias; BatchNorm; ReLU | 32×32×32 | 864 + 64 | 3 |
| Block 1 | branch $r = 1$: Conv2D 32 3×3; BatchNorm; SiLU | 32×32×32 | 9 216 + 64 | 5 |
| | branch $r = 2$: Conv2D 32 3×3 dilation 2; BatchNorm; SiLU | 32×32×32 | 9 216 + 64 | 7 |
| | branch $r = 3$: Conv2D 32 3×3 dilation 3; BatchNorm; SiLU | 32×32×32 | 9 216 + 64 | 9 |
| | Concat | 96×32×32 | 0 | 9 |
| | fuse: Conv2D 64 1×1; BatchNorm; ReLU | 64×32×32 | 6 144 + 128 | 9 |
| | gate: Global pool; Dense 16 ReLU; Dense 64 sigmoid | 64 | 1 040 + 1 088 | |
| | Multiply (gate × fused map) | 64×32×32 | 0 | |
| | shortcut: Conv2D 64 1×1, no bias | 64×32×32 | 2 048 | |
| | Add; SiLU | 64×32×32 | 0 | |
| | Pool max 2×2 | 64×16×16 | 0 | 10 |
| Block 2 | branches $r = 1, 2, 3$: Conv2D 64 3×3; BatchNorm; SiLU | 3 × 64×16×16 | 3 × (36 864 + 128) | 14, 18, 22 |
| | Concat | 192×16×16 | 0 | 22 |

| Stage | Layer | Output | Parameters | RF |
|---|---|---|---|---|
| | fuse: Conv2D 128 1×1; BatchNorm; ReLU | 128×16×16 | 24 576 + 256 | 22 |
| | gate: Global pool; Dense 32 ReLU; Dense 128 sigmoid | 128 | 4 128 + 4 224 | |
| | Multiply; shortcut Conv2D 128 1×1; Add; SiLU | 128×16×16 | 8 192 | |
| | Pool max 2×2 | 128×8×8 | 0 | 24 |
| Embedding | To tokens (patches): every cell of the 8×8 map | 64×128 | 0 | |
| | Class token + position embeddings | 65×128 | 8 448 | |
| Encoder 1 | LayerNorm; Self-attention (4 heads, dropout 0.1); Dropout; Add | 65×128 | 256 + 66 048 | global |
| | LayerNorm; Dense 256 GELU; Dropout; Dense 128; Dropout; Add | 65×128 | 256 + 33 024 + 32 896 | |
| Encoder 2 | the same | 65×128 | 132 480 | |
| Head | LayerNorm; Token pool (class token); Dropout 0.2; Output | 10 | 256 + 1 290 | |
| | **Total** | | **466 522** | |

**Where the parameters are.**

| Part | Parameters | Share |
|---|---|---|
| Stem | 928 | 0.2% |
| Block 1 (branches 27 840, fuse 6 272, gate 2 128, shortcut 2 048) | 38 288 | 8.2% |
| Block 2 (branches 110 976, fuse 24 832, gate 8 352, shortcut 8 192) | 152 352 | 32.7% |
| Class token and positions | 8 448 | 1.8% |
| Two encoder blocks (attention 66 048, MLP 65 920, norms 512 each) | 264 960 | 56.8% |
| Final LayerNorm and output | 1 546 | 0.3% |
| Total | 466 522 | 100% |

These counts were derived by hand, agree with the application's engine, are pinned in its automatic tests, and are confirmed against a real PyTorch build of the network on every change (the comparison on GitHub, Section 6).

**Where the computation is.** Counting multiply–accumulate operations (MACs) of the convolutions, the Dense layers and attention for one 32×32 image (BatchNorm, activations and pooling are negligible):

| Part | MACs per image |
|---|---|
| Stem ($9 \cdot 3 \cdot 32 \cdot 32^2$) | 0.88 M |
| Block 1: branches $3 \cdot (9 \cdot 32 \cdot 32 \cdot 32^2)$ 28.3 M, fuse 6.3 M, shortcut 2.1 M | 36.7 M |
| Block 2: branches $3 \cdot (9 \cdot 64 \cdot 64 \cdot 16^2)$ 28.3 M, fuse 6.3 M, shortcut 2.1 M | 36.7 M |
| Encoders: per block projections $4Nd^2$ 4.3 M, attention $2N^2d$ 1.1 M, MLP $2Nd \cdot 256$ 4.3 M ($N = 65$, $d = 128$) | 19.2 M |
| Total | about 93.5 M |

About 94 million multiply–accumulates (roughly 0.19 GFLOP) per image: the convolution stages do about 80% of the work with about 41% of the parameters, the encoders the reverse (21% of the work, 57% of the parameters). Because pooling halves the image before block 2 while the width

doubles, both blocks cost almost exactly the same. Attention is cheap here because there are only 65 tokens: its $N^2$ term is 1.1 M MACs per block. Training costs about three times the forward pass, so one epoch of CIFAR-10 (50 000 images) is roughly $1.4 \times 10^{13}$ MACs, a few minutes on a T4.

**Receptive field.** Before the transformer each token sees a 24×24 pixel region of the 32×32 image (the RF column of Table 13), and neighbouring tokens are 4 pixels apart, so their regions overlap heavily. The first encoder block then makes the receptive field global: every token, and the class token, can draw on every part of the image.

### 15.15.4 Training it

1. Open MSG-Former from the Projects page and save it (Cmd+S). It opens with built-in CIFAR-10 at its own size.
2. Train tab: Adam, learning rate 0.001 (0.0005 if the loss is unstable), batch 128, 30–60 epochs; augmentation: horizontal flip and random crop with 4 pixels of padding (transformer layers in particular profit from augmentation [37]).
3. Train on Kaggle or any GPU: by a rough estimate it needs about 1 GB of GPU memory at batch 128 (mostly the activations of the two convolution stages kept for the backward pass), so even small GPUs are enough.
4. Compare with the Small CNN (551 018 parameters, convolution only) and ViT-Tiny (1 806 538 parameters, attention only), trained with the same settings, in the Runs table.

### 15.15.5 Experiments to run (ablations)

Each is a small edit on the canvas, and together they show which part of the design earns its place:

| | **Change on the canvas** | **Parameters** |
|---|---|---|
| A | Remove both gates (delete Global pool, the two Dense layers and Multiply; connect the fuse to Add) | 456 042 |
| B | All dilations 1 (three identical-scale branches) | 466 522 |
| C | No transformer: Global pool after the second pooling, then Output | 192 858 |
| D | A third encoder block | 598 970 |
| E | Token pool *mean* instead of the class token | 466 522 |
| F | Wider: branch filters 64 and 128 | larger |

(Counts for A, C and D follow from the tables above: A removes 2 128 + 8 352; C removes the embedding, encoders, final norm and the 128-input head and adds a 128-input head; D adds one encoder block of 132 480. The application shows the exact count after each edit.) If B matches the full model, the multi-scale branches are not helping on this data; if C matches it, the attention is not; and so on.

### 15.15.6 Adapting it

- **Larger images** (for example 64×64 or 224×224): add a stride-2 stem (or a third block with pooling) so that the grid reaching the transformer stays around 8×8 to 14×14; otherwise the number of tokens, and the $N^2$ attention cost, grows with the image.
- **More classes:** only the output layer changes ($128 \cdot K + K$ parameters).

- **Segmentation:** the two convolution stages can serve as an encoder in a U-Net-style design, with ConvTranspose2D decoders and skips from the blocks.

#### 15.15.7 Status

The design, its shapes and its parameter count are verified (by hand, by the application's engine and against PyTorch). Its accuracy is not: no training results exist yet. Treat it as a starting point for experiments, and record the results of the comparisons above.

## 16 Troubleshooting

**"design has problems".** Open Design; the red block says why (Section 5.1). Blocks after it are greyed out until it is fixed.

**"dataset incomplete" or no classes.** The class list is empty (Hugging Face: **Read info** and a ClassLabel column; Kaggle: type the class folder names). Text datasets cannot be used.

**The one-batch check stops the run.** A layer's real shape differed from the application's (for example images of an unexpected size in a custom dataset). The message names the layer; check the image size and channels in the Dataset tab.

**"CUDA out of memory".** With **Split a batch that does not fit** on (the default) the run handles this itself and continues more slowly. If even one image at a time does not fit, the run stops with a message: switch on **Save memory**, keep mixed precision on, or use a smaller input size, network or a larger GPU (Section 6).

**Kaggle run fails to start.** The account must be phone-verified for GPU and internet access; the token must be current.

**SSH machine does not answer.** Password machines: press **Connect** again (the login lasts a few hours); key machines: **Test**. A running training is not affected.

**EC2: "no public address" / ssh never answers.** Give the instance a public IP (or an Elastic IP), allow port 22 in its security group, and use its key file. A new account may have a vCPU quota of 0 for G instances: request an increase.

**EC2 spot: the run says "interrupted".** AWS took the capacity back; with a persistent request and behaviour *stop* it resumes by itself. To continue now, use **Continue on another machine** (needs the checkpoint copy on the Mac).

**A demo disappeared.** Unsaved demos are discarded when another demo or project is opened or the application quits; save them with Cmd+S.

**Training is unstable (loss jumps or becomes NaN).** Lower the learning rate (by 10×), keep mixed precision on (the script scales the loss), add BatchNorm / LayerNorm, check that the labels match the class list.

**High training accuracy, low validation accuracy.** Overfitting: augmentation, dropout, fewer parameters (Global pool instead of Flatten + large Dense), more data, fewer epochs.

## 17 Privacy and security

- Tokens and keys live only in the system keychain; they are passed to one call at a time, and written on a remote machine only when a run needs them, as files readable by their owner.
- The design computer never downloads datasets or runs training; it reads dataset descriptions only.

- Unsaved demos are kept in the application's own folder and deleted when no longer needed; projects live where you choose.
- Updates come only from the project's GitHub releases and are checked against the published SHA-256 checksum.

# A Glossary

**Activation function** a non-linear function applied element-wise (ReLU, GELU, ...).

**Batch** the images processed together in one training step.

**Channel** one "layer" of an image or feature map (3 for RGB; a convolution with 64 filters produces 64).

**Checkpoint** a saved training state (weights, optimiser, position) from which a run can resume.

**Epoch** one pass over the whole training set.

**Feature map** the output of a convolution: one image per filter.

**Kernel / filter** the small window of weights a convolution slides over the image.

**mIoU** mean over the classes of (overlap / union) of predicted and true masks.

**Padding** pixels added around the image so the window can reach the borders.

**Parameter** a trainable number (weight or bias).

**Receptive field** the input region one output value depends on.

**Spot instance** spare cloud capacity at a lower price that the provider can take back.

**Stride** how far the window moves at each step.

**Token** one element of a sequence a transformer works on (here: an image patch or row).

# References


[1] J. L. Ba, J. R. Kiros and G. E. Hinton. Layer normalization. 2016. arXiv:1607.06450.

[2] L.-C. Chen, G. Papandreou, F. Schroff and H. Adam. Rethinking atrous convolution for semantic image segmentation. 2017. arXiv:1706.05587.

[3] T. Chen, B. Xu, C. Zhang and C. Guestrin. Training deep nets with sublinear memory cost. 2016. arXiv:1604.06174.

[4] E. D. Cubuk, B. Zoph, D. Mané, V. Vasudevan and Q. V. Le. AutoAugment: learning augmentation policies from data. 2018. arXiv:1805.09501.

[5] Z. Dai, H. Liu, Q. V. Le and M. Tan. CoAtNet: marrying convolution and attention for all data sizes. 2021. arXiv:2106.04803.

[6] A. Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. 2020. arXiv:2010.11929.

[7] V. Dumoulin and F. Visin. A guide to convolution arithmetic for deep learning. 2016. arXiv:1603.07285.

[8] P. Goyal et al. Accurate, large minibatch SGD: training ImageNet in 1 hour. 2017. arXiv:1706.02677.

[9] K. He, X. Zhang, S. Ren and J. Sun. Deep residual learning for image recognition. 2015. arXiv:1512.03385.

[10] K. He, X. Zhang, S. Ren and J. Sun. Identity mappings in deep residual networks. 2016. arXiv:1603.05027.

[11] D. Hendrycks and K. Gimpel. Gaussian error linear units (GELUs). 2016. arXiv:1606.08415.

[12] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. 2012. arXiv:1207.0580.

[13] A. G. Howard et al. MobileNets: efficient convolutional neural networks for mobile vision applications. 2017. arXiv:1704.04861.

[14] J. Hu, L. Shen, S. Albanie, G. Sun and E. Wu. Squeeze-and-excitation networks. 2017. arXiv:1709.01507.

[15] G. Huang, Z. Liu, L. van der Maaten and K. Q. Weinberger. Densely connected convolutional networks. 2016. arXiv:1608.06993.

[16] S. Ioffe and C. Szegedy. Batch normalization: accelerating deep network training by reducing internal covariate shift. 2015. arXiv:1502.03167.

[17] D. P. Kingma and J. Ba. Adam: a method for stochastic optimization. 2014. arXiv:1412.6980.

[18] A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009 (not on arXiv). `https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf`.

[19] A. Krizhevsky, I. Sutskever and G. E. Hinton. ImageNet classification with deep convolutional neural networks. NeurIPS 2012 (not on arXiv). `https://papers.nips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html`.

[20] A. Krizhevsky. One weird trick for parallelizing convolutional neural networks. 2014. arXiv:1404.5997.

[21] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner. Gradient-based learning applied to document recognition. Proceedings of the IEEE 86(11), 1998 (not on arXiv). doi:10.1109/5.726791.

[22] Q. Lhoest et al. Datasets: a community library for natural language processing. 2021. arXiv:2109.02846.

[23] M. Lin, Q. Chen and S. Yan. Network in network. 2013. arXiv:1312.4400.

[24] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. 2017. arXiv:1711.05101.

[25] W. Luo, Y. Li, R. Urtasun and R. Zemel. Understanding the effective receptive field in deep convolutional neural networks. 2017. arXiv:1701.04128.

[26] S. Mehta and M. Rastegari. MobileViT: light-weight, general-purpose, and mobile-friendly Vision Transformer. 2021. arXiv:2110.02178.

[27] P. Micikevicius et al. Mixed precision training. 2017. arXiv:1710.03740.

[28] O. Oktay et al. Attention U-Net: learning where to look for the pancreas. 2018. arXiv:1804.03999.

[29] A. Paszke et al. PyTorch: an imperative style, high-performance deep learning library. 2019. arXiv:1912.01703.

[30] S. Rajbhandari, J. Rasley, O. Ruwase and Y. He. ZeRO: memory optimizations toward training trillion parameter models. 2019. arXiv:1910.02054.

[31] P. Ramachandran, B. Zoph and Q. V. Le. Searching for activation functions. 2017. arXiv:1710.05941.

[32] O. Ronneberger, P. Fischer and T. Brox. U-Net: convolutional networks for biomedical image segmentation. 2015. arXiv:1505.04597.

[33] O. Russakovsky et al. ImageNet large scale visual recognition challenge. 2014. arXiv:1409.0575.

[34] R. R. Selvaraju et al. Grad-CAM: visual explanations from deep networks via gradient-based localization. 2016. arXiv:1610.02391.

[35] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. 2014. arXiv:1409.1556.

[36] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. JMLR 15, 2014 (not on arXiv; see also [12]). `https://jmlr.org/papers/v15/srivastava14a.html`.

[37] A. Steiner et al. How to train your ViT? Data, augmentation, and regularization in Vision Transformers. 2021. arXiv:2106.10270.

[38] C. Szegedy et al. Going deeper with convolutions. 2014. arXiv:1409.4842.

[39] H. Touvron et al. Training data-efficient image transformers & distillation through attention. 2020. arXiv:2012.12877.

[40] A. Vaswani et al. Attention is all you need. 2017. arXiv:1706.03762.

[41] P. Wang et al. Understanding convolution for semantic segmentation. 2017. arXiv:1702.08502.

[42] H. Xiao, K. Rasul and R. Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. 2017. arXiv:1708.07747.

[43] R. Xiong et al. On layer normalization in the Transformer architecture. 2020. arXiv:2002.04745.

[44] F. Yu and V. Koltun. Multi-scale context aggregation by dilated convolutions. 2015. arXiv:1511.07122.

[45] X. Zhai, A. Kolesnikov, N. Houlsby and L. Beyer. Scaling Vision Transformers. 2021. arXiv:2106.04560.

[46] V. Sainte Fare Garnot and L. Landrieu. Panoptic segmentation of satellite image time series with convolutional temporal attention networks. ICCV, 2021. arXiv:2107.07933.

[47] M. Tarasiou, E. Chavez and S. Zafeiriou. ViTs for SITS: Vision transformers for satellite image time series. CVPR, 2023. arXiv:2301.04944.

[48] S. Hochreiter and J. Schmidhuber. Long short-term memory. Neural Computation 9(8), 1997.

[49] X. Zhou, D. Wang and P. Krähenbühl. Objects as points. 2019. arXiv:1904.07850.

[50] E. J. Hu, Y. Shen, P. Wallis et al. LoRA: Low-rank adaptation of large language models. 2021. arXiv:2106.09685.